

SARCINI LA TEMA: GESTIUNEA FISIERELOR IN PYTHON

I. PARTE(30 puncte)

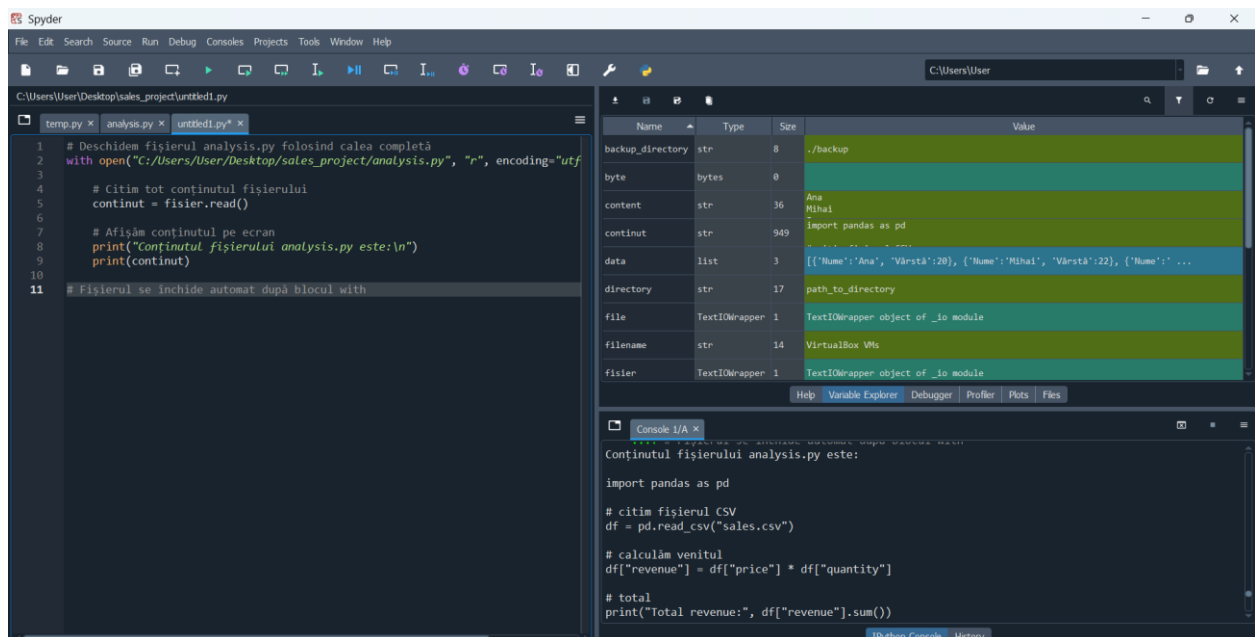
1. Citirea unui fișier text și afișarea conținutului

Scrie un program care citește un fișier text și afișează conținutul acestuia pe ecran.

Raspuns COD

```
# Deschidem fișierul analysis.py folosind calea completă
with open("C:/Users/User/Desktop/sales_project/analysis.py", "r", encoding="utf-8") as fisier:
    # Citim tot conținutul fișierului
    continut = fisier.read()
    # Afișăm conținutul pe ecran
    print("Conținutul fișierului analysis.py este:\n")
    print(continut)
# Fișierul se închide automat după blocul with
```

CAPTURA



2. Numărarea numărului de cuvinte dintr-un fișier text

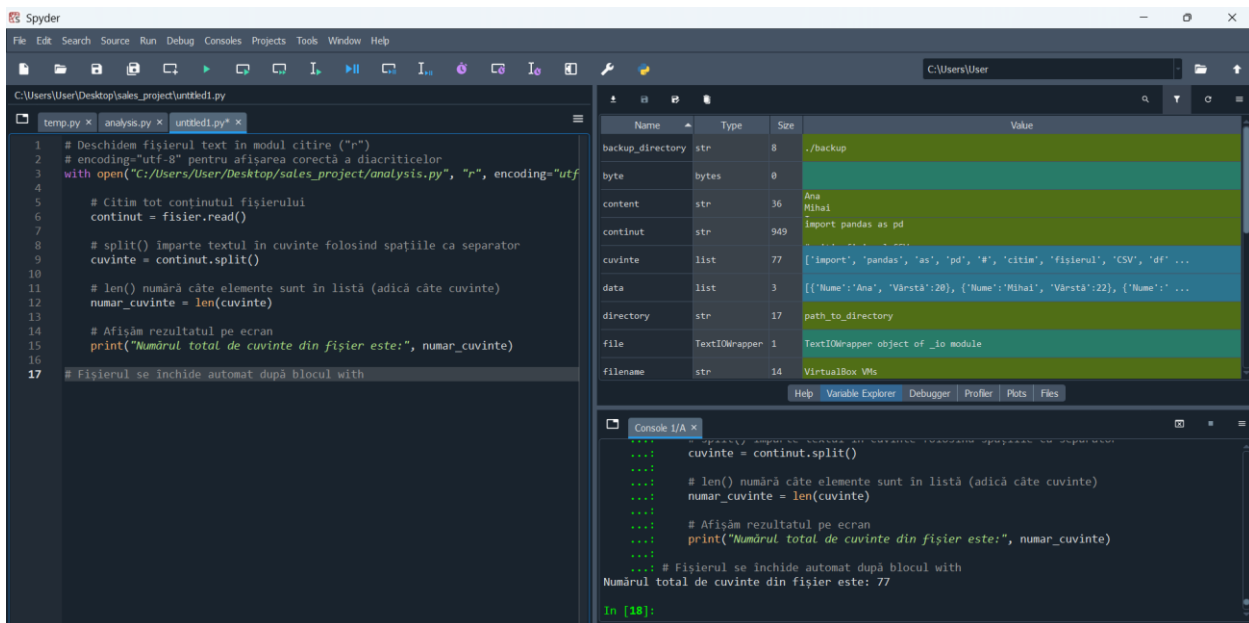
Scrie un program care citește un fișier text și numără câte cuvinte sunt în fișier.

```
# Deschidem fișierul text în modul citire ("r")
# encoding="utf-8" pentru afișarea corectă a diacriticelor
with open("C:/Users/User/Desktop/sales_project/analysis.py", "r", encoding="utf-8") as fisier:
    # Citim tot conținutul fișierului
    continut = fisier.read()
    # split() împarte textul în cuvinte folosind spațiile ca separator
```

```

cuvinte = continut.split()
# len() numără câte elemente sunt în listă (adică câte cuvinte)
numar_cuvinte = len(cuvinte)
# Afișăm rezultatul pe ecran
print("Numărul total de cuvinte din fișier este:", numar_cuvinte)
# Fișierul se închide automat după blocul with

```



3. Numărarea liniilor dintr-un fișier

Scrie un program care citește un fișier și numără câte linii conține.

```

# Deschidem fișierul în modul citire ("r")
# encoding="utf-8" pentru afișarea corectă a caracterelor
with open("C:/Users/User/Desktop/sales_project/analysis.py", "r", encoding="utf-8") as fisier:

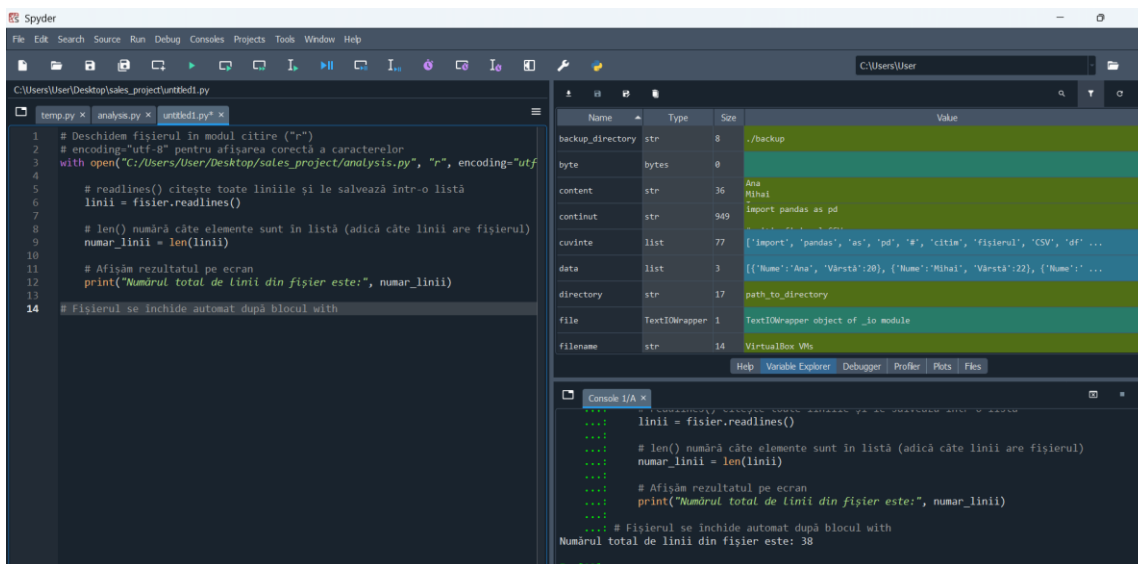
    # readlines() citește toate liniile și le salvează într-o listă
    linii = fisier.readlines()

    # len() numără câte elemente sunt în listă (adică câte linii are fișierul)
    numar_linii = len(linii)

    # Afișăm rezultatul pe ecran
    print("Numărul total de linii din fișier este:", numar_linii)

# Fișierul se închide automat după blocul with

```



Iată o **reformulare** a textului și explicației codului:

Căutarea unui cuvânt într-un fișier

Scrie un program care caută un cuvânt specific într-un fișier și afișează toate liniile în care apare acel cuvânt.

Cuvântul pe care dorim să îl găsim

cuvant_cautat = "import"

Deschidem fișierul în modul citire ("r")

encoding="utf-8" asigură afișarea corectă a caracterelor

with open("C:/Users/User/Desktop/sales_project/analysis.py", "r", encoding="utf-8") as fisier:

Parcurgem fișierul linie cu linie

for numar_linie, linie in enumerate(fisier, start=1):

Verificăm dacă cuvântul căutat apare în linie

if cuvant_cautat in linie:

Afișăm numărul liniei și textul acesteia

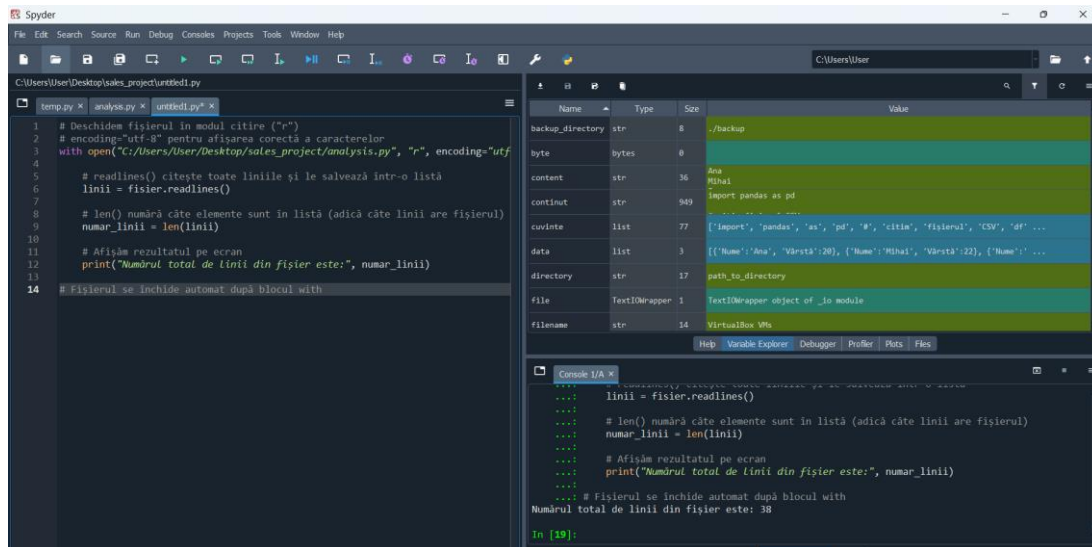
print(f"Linia {numar_linie}: {linie.strip()}")

Fișierul se închide automat după terminarea blocului with

Explicație:

- Variabila `cuvant_cautat` conține cuvântul pe care dorim să îl găsim.
- Funcția `open()` deschide fișierul în modul citire.
- `enumerate()` parcurge fiecare linie și oferă și numărul acesteia.
- Dacă linia conține cuvântul căutat, programul afișează numărul liniei și conținutul ei.
- Blocul `with` închide automat fișierul după terminarea citirii.

Dacă vrei, îți pot face și o **versiune mai scurtă pentru caiet / raport de laborator** sau o **explicație pas cu pas mai simplă pentru începători**.



4. Scrierea unui fișier cu date citite de la utilizator

Scrie un program care primește de la utilizator un text și îl salvează într-un fișier text.

Cerem utilizatorului să introducă un text

`text_utilizator = input("Introdu un text pe care vrei să îl salvezi în fișier: ")`

Deschidem (sau creăm) fișierul în modul scriere ("w")

Dacă fișierul există deja, conținutul lui va fi șters

`with open("C:/Users/User/Desktop/sales_project/text_salvat.txt", "w", encoding="utf-8") as f:`

Scriem textul introdus de utilizator în fișier

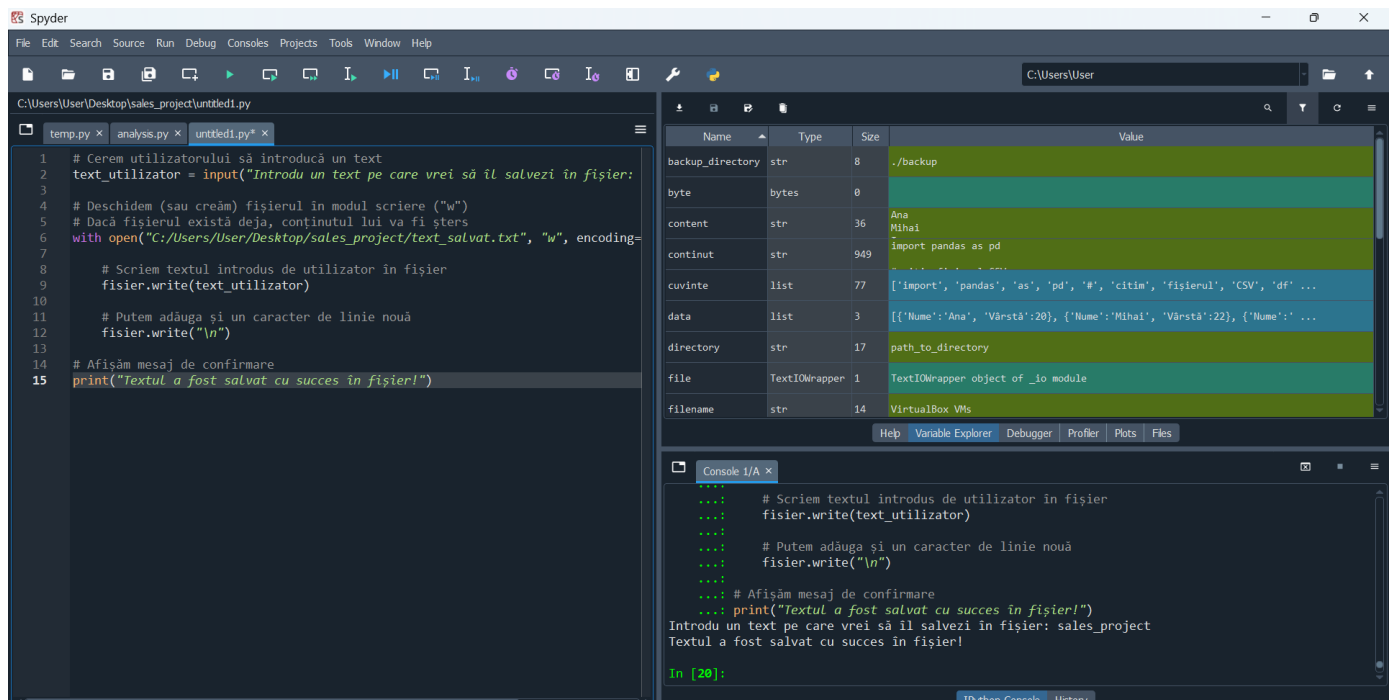
`f.write(text_utilizator)`

Putem adăuga și un caracter de linie nouă

`f.write("\n")`

Afișăm mesaj de confirmare

`print("Textul a fost salvat cu succes în fișier!")`



5. Adăugarea unui text într-un fișier existent

Scrie un program care adaugă un text introdus de utilizator la sfârșitul unui fișier existent.

Cerem utilizatorului să introducă textul care va fi adăugat
`text_utilizator = input("Introdu textul pe care vrei să îl adaugi: ")`

Deschidem fișierul în modul adăugare ("a")

Modul "a" înseamnă: adaugă la sfârșit fără să ștergă conținutul existent

`with open("C:/Users/User/Desktop/sales_project/text_salvat.txt", "a", encoding="utf-8") as f:` as fișier:

Scriem textul introdus în fișier

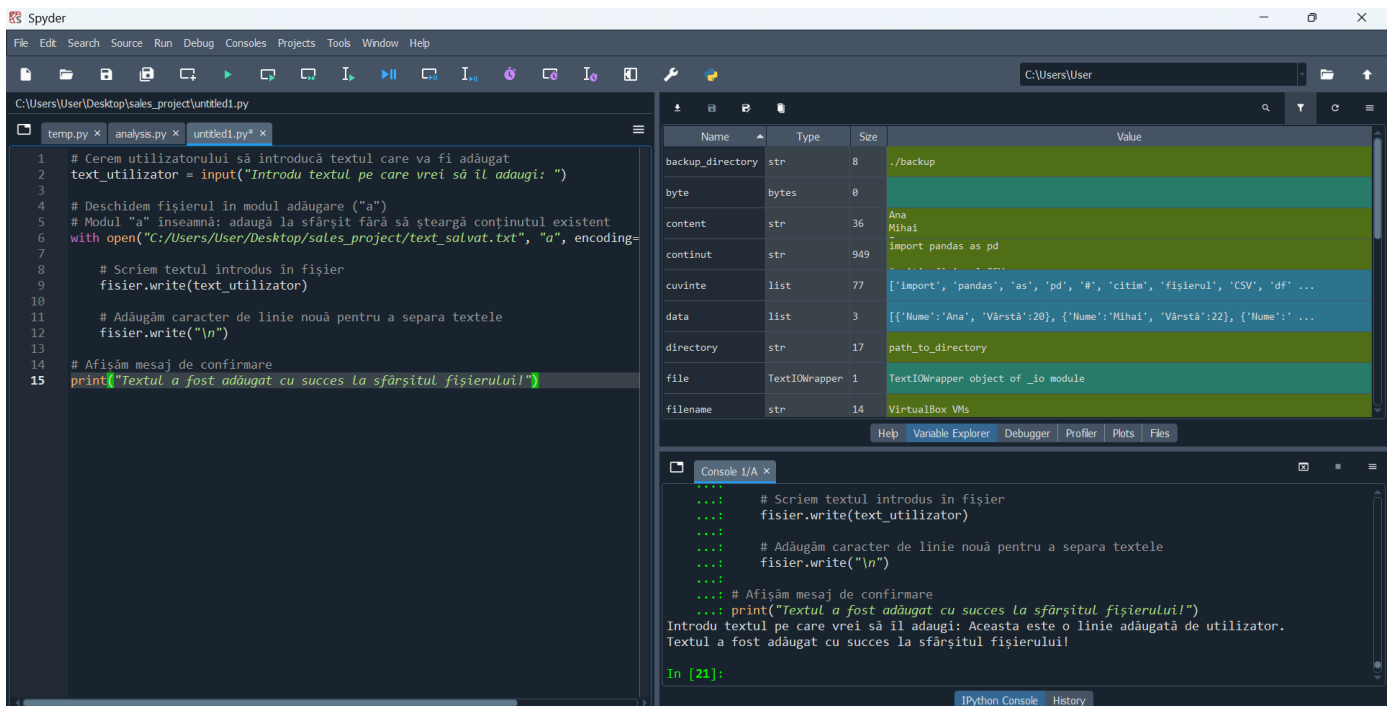
`f.write(text_utilizator)`

Adăugăm caracter de linie nouă pentru a separa textele

`f.write("\n")`

Afișăm mesaj de confirmare

`print("Textul a fost adăugat cu succes la sfârșitul fișierului!")`



6. Crearea unui fișier cu datele unui dicționar

Scrie un program care creează un fișier text în care salvează datele unui dicționar (cheie: valoare), fiecare pe o linie.

Creăm un dicționar cu date (cheie: valoare)

```

date_student = {
    "Nume": "Irina",
    "Vârsta": 20,
    "Specializare": "Securitatea Telecomunicațiilor Electronice",
    "Universitate": "UTM"
}

```

Deschidem (sau creăm) fișierul în modul scriere ("w")

Dacă fișierul există deja, conținutul va fi șters

with open("C:/Users/User/Desktop/sales_project/date_student.txt", "w", encoding="utf-8") as fisier:

Parcurgem dicționarul folosind metoda items()

items() returnează perechi (cheie, valoare)

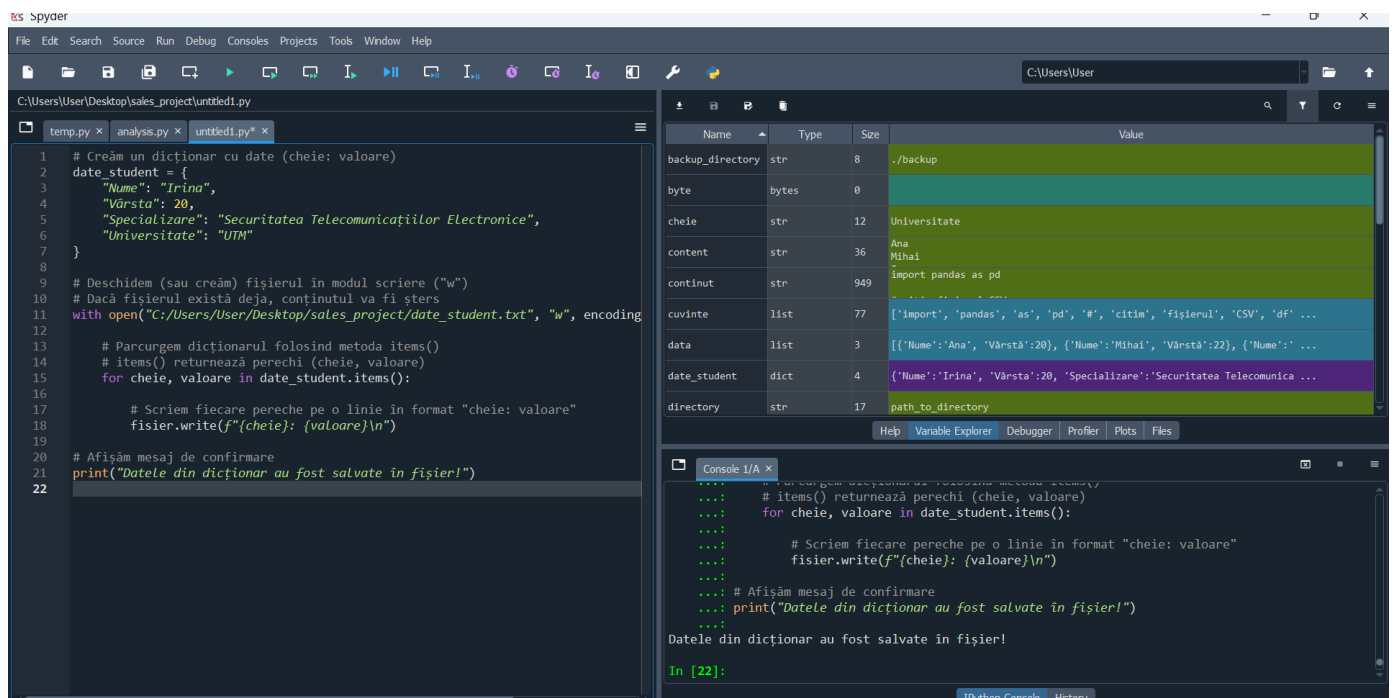
for cheie, valoare in date_student.items():

Scriem fiecare pereche pe o linie în format "cheie: valoare"

fisier.write(f'{cheie}: {valoare}\n')

Afișăm mesaj de confirmare

print("Datele din dicționar au fost salvate în fișier!")



7. Citirea unui fișier CSV și afisarea conținutului

Scrie un program care citește un fișier CSV și afișează conținutul acestuia într-un format ușor de citit.

Importăm modulul csv pentru a lucra cu fișiere CSV

```
import csv
```

Deschidem fișierul sales.csv în modul citire

```
with open("C:/Users/User/Desktop/sales_project/sales.csv", "r", newline="", encoding="utf-8") as fisier_csv:
```

Creăm un obiect reader care citește fiecare rând

```
reader = csv.reader(fisier_csv)
```

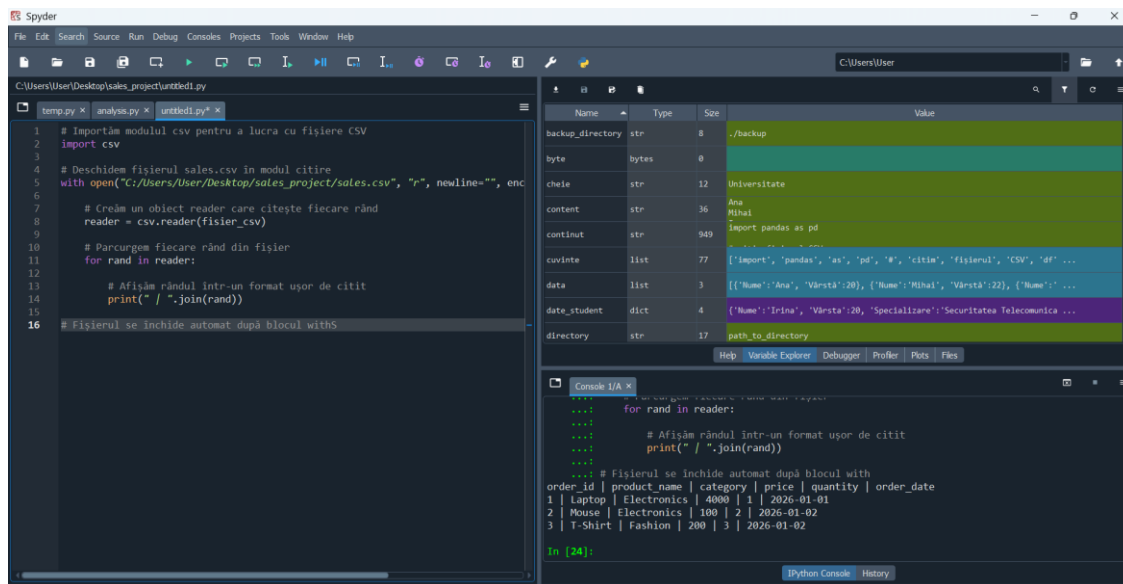
Parcurgem fiecare rând din fișier

```
for rand in reader:
```

Afișăm rândul într-un format ușor de citit

```
print(" | ".join(rand))
```

Fișierul se închide automat după blocul with



8. Ștergerea unui fișier

Scrie un program care șterge un fișier specificat de utilizator.

Importăm modulul os pentru a lucra cu fișierele sistemului

`import os`

Afișăm directorul curent ca utilizatorul să știe unde caută fișierul

`print("Director curent:", os.getcwd())`

Cerem utilizatorului să introducă numele fișierului

`nume_fisier = input("Introdu numele fișierului pe care vrei să îl ștergi (ex: text_salvat.txt): ")`

Verificăm dacă fișierul există în directorul curent

`if os.path.exists(nume_fisier):`

Ștergem fișierul folosind funcția os.remove()

`os.remove(nume_fisier)`

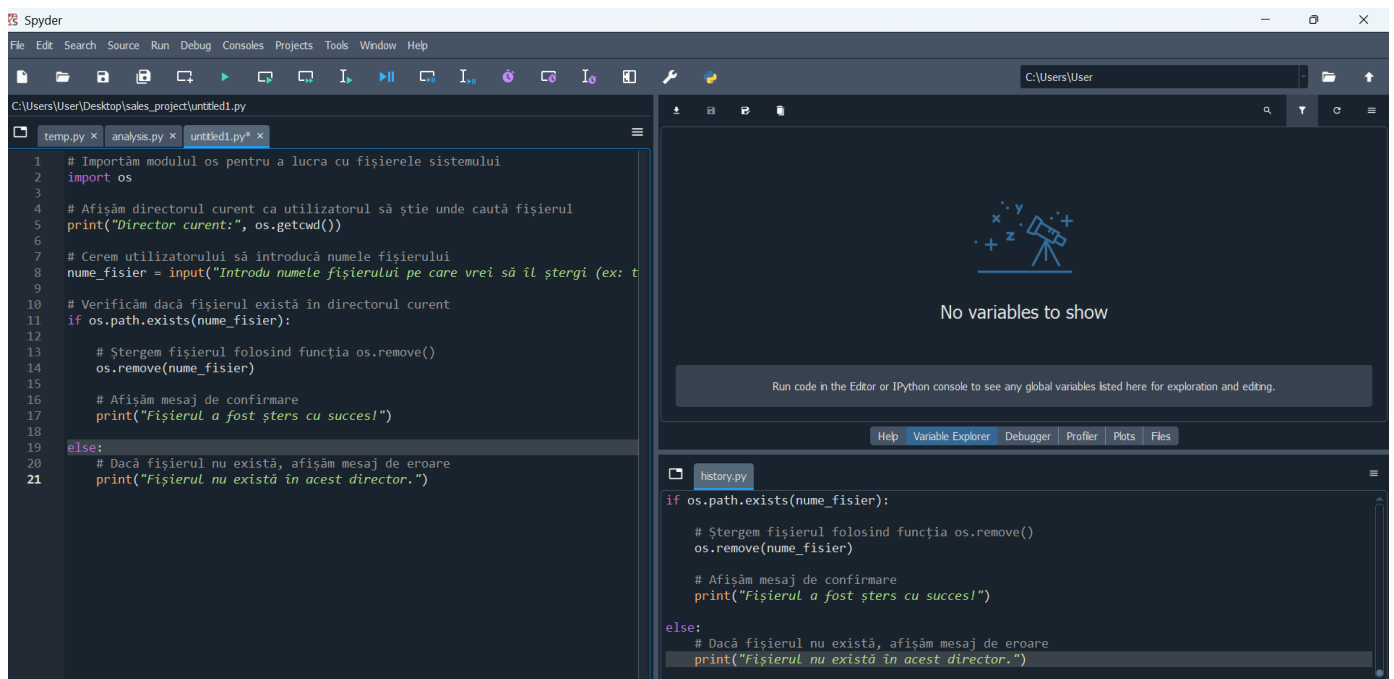
Afișăm mesaj de confirmare

`print("Fișierul a fost șters cu succes!")`

`else:`

Dacă fișierul nu există, afișăm mesaj de eroare

`print("Fișierul nu există în acest director.")`



9. Calcularea sumelor dintr-un fișier CSV

Scrie un program care citește un fișier CSV în care fiecare linie conține mai multe numere și calculează suma valorilor din fiecare coloană.

```
import csv
```

```
with open("C:/Users/User/Desktop/sales_project/sales.csv", "r", newline="", encoding="utf-8") as fisier:
```

```
    reader = csv.reader(fisier)
```

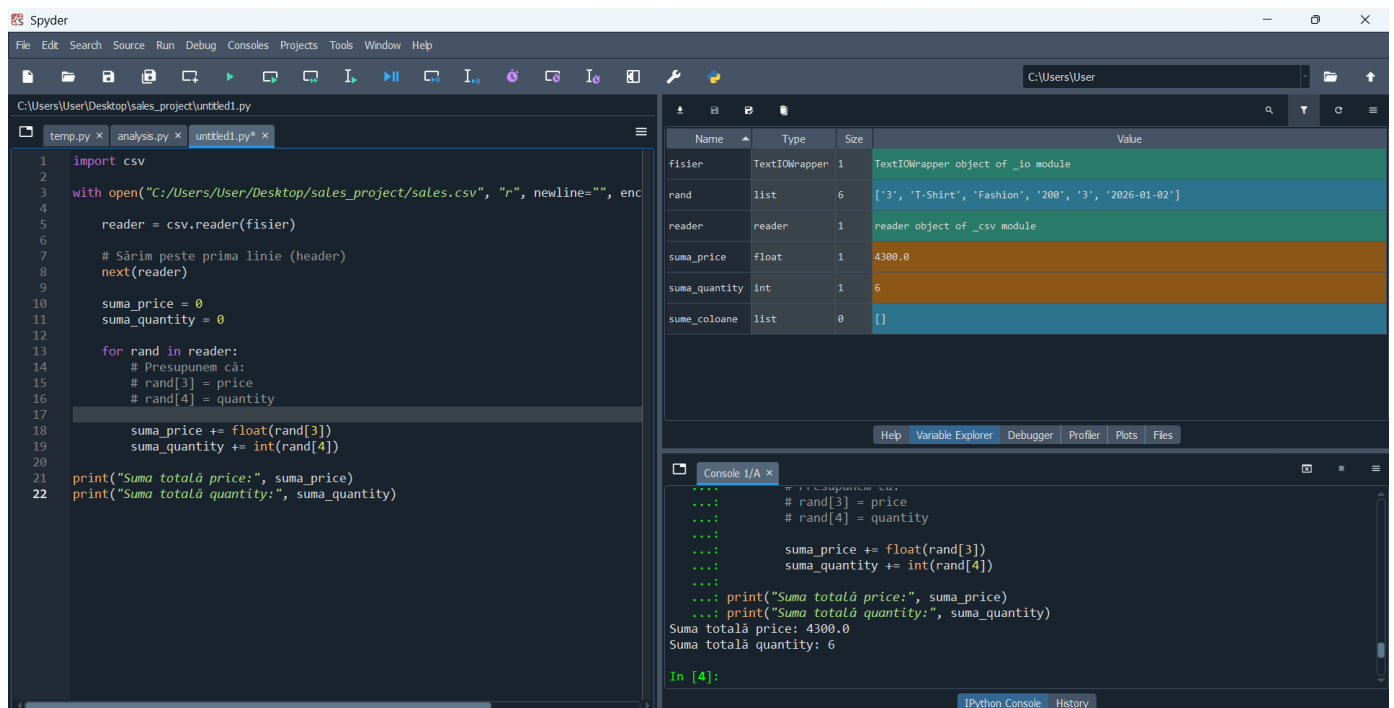
```
    # Sărim peste prima linie (header)
    next(reader)
```

```
    suma_price = 0
    suma_quantity = 0
```

```
    for rand in reader:
        # Presupunem că:
        # rand[3] = price
        # rand[4] = quantity
```

```
        suma_price += float(rand[3])
        suma_quantity += int(rand[4])
```

```
print("Suma totală price:", suma_price)
print("Suma totală quantity:", suma_quantity)
```



10. Căutarea unui nume într-un fișier CSV cu date despre angajați

Scrie un program care caută un nume de angajat într-un fișier CSV și afișează informațiile asociate acelui angajat.

Importăm modulul csv pentru a lucra cu fișiere CSV

```
import csv
```

Cerem utilizatorului să introducă numele angajatului pe care vrea să îl caute

```
nume_cautat = input("Introdu numele angajatului pe care vrei să îl cauți: ")
```

Deschidem fișierul angajati.csv în modul citire

```
with open("C:/Users/User/Desktop/sales_project/angajati.csv", "r", newline="", encoding="utf-8") as fisier:
```

Folosim DictReader pentru a lucra mai ușor cu coloanele

```
reader = csv.DictReader(fisier)
```

```
gasit = False # Variabilă pentru a verifica dacă angajatul a fost găsit
```

Parcurgem fiecare rând din fișier

```
for rand in reader:
```

Verificăm dacă numele introdus există în coloana "Nume"

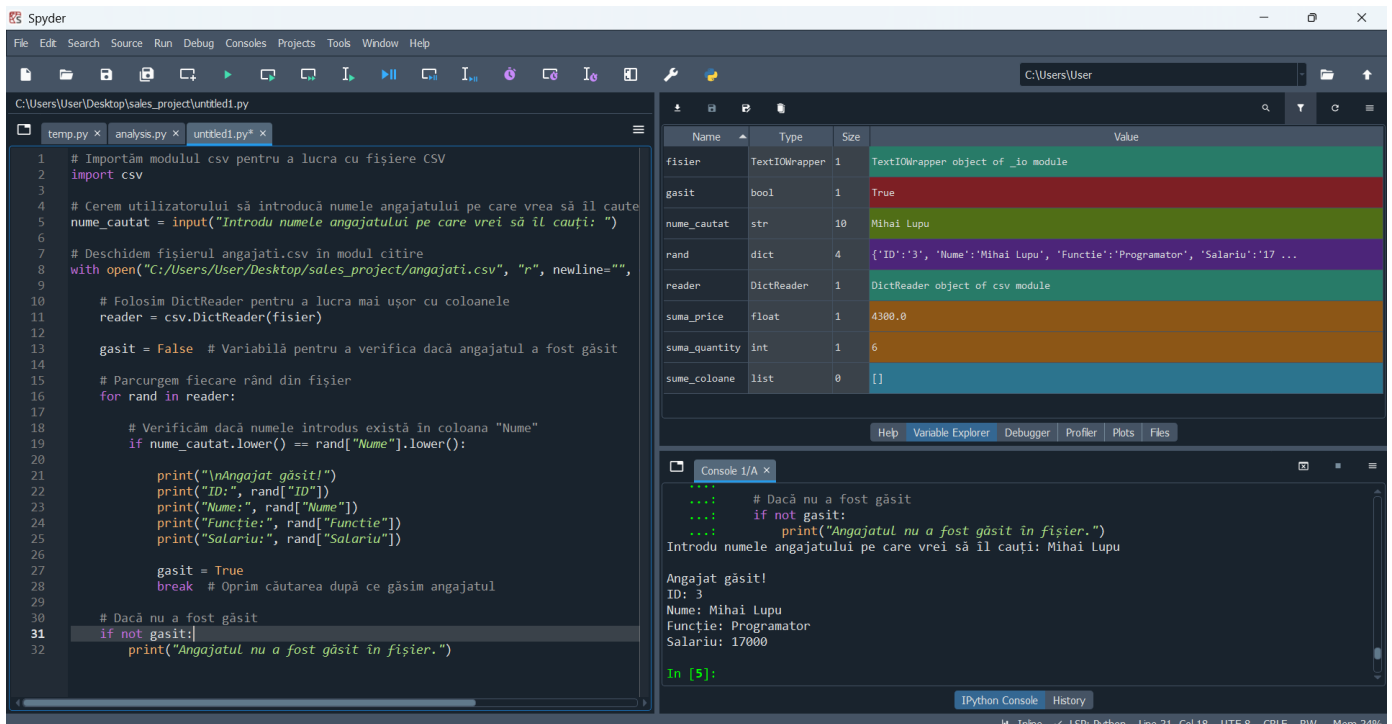
```
if nume_cautat.lower() == rand["Nume"].lower():
```

```
    print("\nAngajat găsit!")
    print("ID:", rand["ID"])
    print("Nume:", rand["Nume"])
    print("Funcție:", rand["Funcție"])
    print("Salariu:", rand["Salariu"])
```

```
    gasit = True
```

```
    break # Oprim căutarea după ce găsim angajatul
```

```
# Dacă nu a fost găsit
if not gasit:
    print("Angajatul nu a fost găsit în fișier.")
```



11. Găsirea liniei cu cele mai multe cuvinte

Scrie un program care citește un fișier text și găsește linia cu cele mai multe cuvinte. Afișează acea linie.

```
# Deschidem fișierul text în modul citire
with open("C:/Users/User/Desktop/sales_project/date_student.txt", "r", encoding="utf-8") as fisier:
```

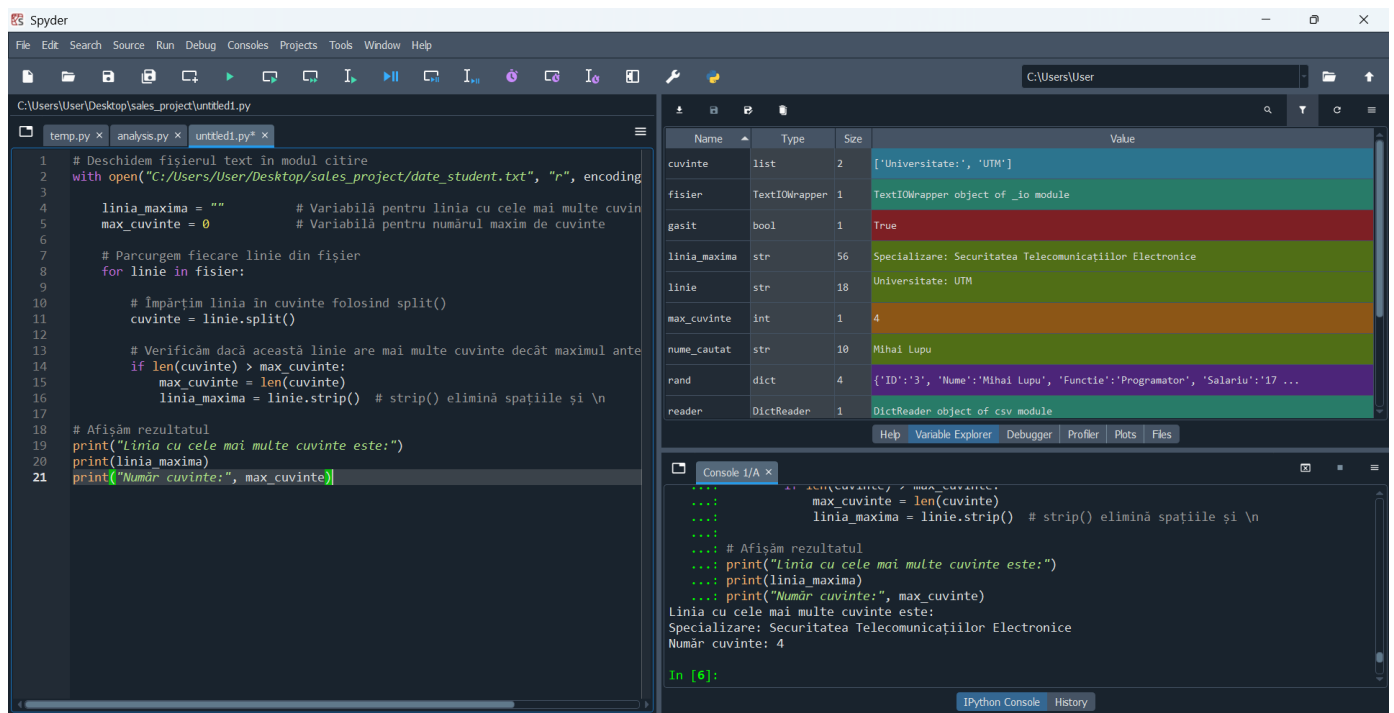
```
linia_maxima = ""      # Variabilă pentru linia cu cele mai multe cuvinte
max_cuvinte = 0        # Variabilă pentru numărul maxim de cuvinte
```

```
# Parcurgem fiecare linie din fișier
for linie in fisier:
```

```
    # Împărțim linia în cuvinte folosind split()
    cuvinte = linie.split()
```

```
    # Verificăm dacă această linie are mai multe cuvinte decât maximul anterior
    if len(cuvinte) > max_cuvinte:
        max_cuvinte = len(cuvinte)
        linia_maxima = linie.strip() # strip() elimină spațiile și \n
```

```
# Afișăm rezultatul
print("Linia cu cele mai multe cuvinte este:")
print(linia_maxima)
print("Număr cuvinte:", max_cuvinte)
```



12. Scrierea unui fișier JSON

Scrie un program care creează un fișier JSON dintr-un dicționar și îl salvează pe disc.

Importăm modulul json pentru a lucra cu fișiere JSON

import json

Creăm un dicționar cu date

```

date_student = {
    "nume": "Irina Popa",
    "varsta": 20,
    "specializare": "Securitatea Telecomunicațiilor Electronice",
    "an_studiu": 2,
    "medie": 9.5
}

```

Deschidem (sau creăm) fișierul JSON în modul scriere ("w")

encoding="utf-8" pentru caractere speciale

with open("C:/Users/User/Desktop/sales_project/student.json", "w", encoding="utf-8") as f: # f is the file object

dump() scrie dicționarul în fișier în format JSON

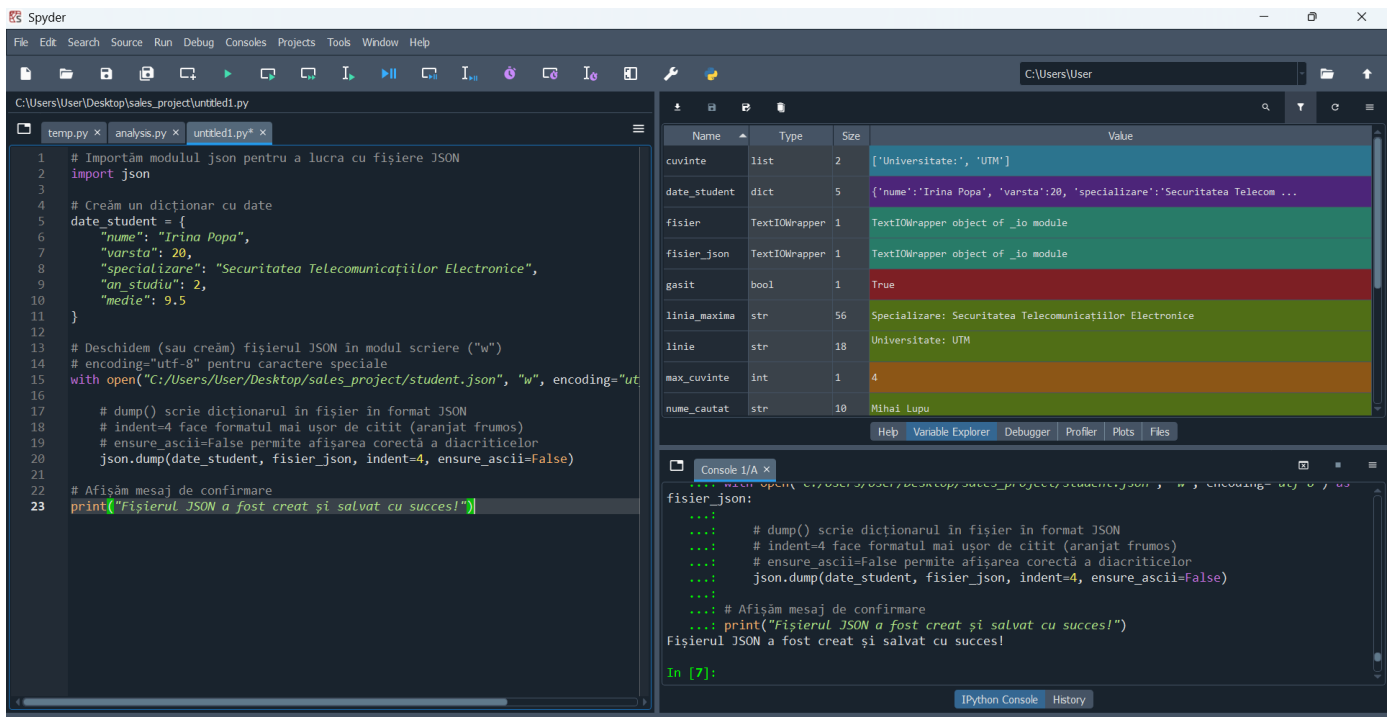
indent=4 face formatul mai ușor de citit (aranjat frumos)

ensure_ascii=False permite afișarea corectă a diacriticelor

json.dump(date_student, f, indent=4, ensure_ascii=False)

Afișăm mesaj de confirmare

print("Fișierul JSON a fost creat și salvat cu succes!")



13. Citirea unui fișier JSON și afișarea datelor

Scrie un program care citește un fișier JSON și afișează conținutul acestuia într-un format ușor de citit.

Importăm modulul json pentru a lucra cu fișiere JSON

```
import json
```

Deschidem fișierul JSON în modul citire ("r")

```
with open("C:/Users/User/Desktop/sales_project/student.json", "r", encoding="utf-8") as fisier_json:
```

load() citește conținutul JSON și îl transformă într-un dicționar Python

```
date = json.load(fisier_json)
```

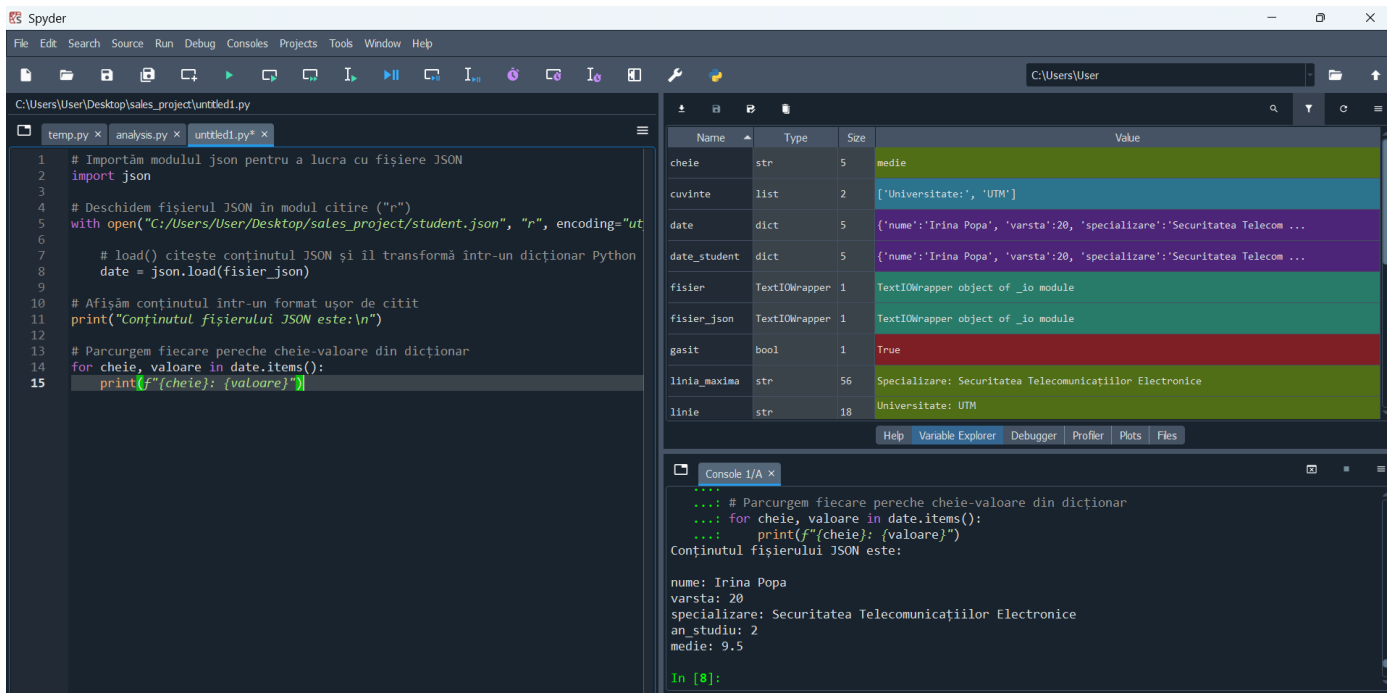
Afișăm conținutul într-un format ușor de citit

```
print("Conținutul fișierului JSON este:\n")
```

Parcurgem fiecare pereche cheie-valoare din dicționar

```
for cheie, valoare in date.items():
```

```
    print(f'{cheie}: {valoare}')
```



14. Compararea a două fișiere text

Scrie un program care compară două fișiere text și afișează liniile care sunt diferite între ele.

Deschidem primul fișier în modul citire

with open("C:/Users/User/Desktop/sales_project/fisier1.txt", "r", encoding="utf-8") as f1:

readlines() citește toate liniile și le salvează într-o listă

linii1 = f1.readlines()

Deschidem al doilea fișier

with open("C:/Users/User/Desktop/sales_project/fisier2.txt", "r", encoding="utf-8") as f2:

linii2 = f2.readlines()

print("Liniile diferite între cele două fișiere:\n")

Determinăm numărul maxim de linii

max_linii = max(len(linii1), len(linii2))

Parcurgem fiecare poziție

for i in range(max_linii):

Luăm linia din fiecare fișier dacă există

linie1 = linii1[i].strip() if i < len(linii1) else ""

linie2 = linii2[i].strip() if i < len(linii2) else ""

Comparăm liniile

if linie1 != linie2:

print(f'Linia {i+1} este diferită:')

print("Fisier1:", linie1)

print("Fisier2:", linie2)

print("-" * 40)

II. PARTE (30 puncte)

SARCINI LA TEMA "GESTIUNEA FIȘIERELOR ÎN PYTHON"

Sarcina 1: Crearea și citirea unui fișier de text

Scrie un program care:

1. Creează un fișier `students.txt` și scrie în el numele unor studenți (un nume pe linie).
2. Apoi, citește și afișează pe ecran toate liniile din fișier.

1 Scriem numele în fișierul students.txt

with open("students.txt", "w", encoding="utf-8") as fisier:

```
fisier.write("Barduc Iulian\n")
fisier.write("Ciutac Olga\n")
fisier.write("Grișca Ivan\n")
fisier.write("Pușcaș Roman\n")
fisier.write("Rotari Bogdan\n")
fisier.write("Țărnă Irina\n")
fisier.write("Țepușel Dan\n")
fisier.write("Vîntu Marius\n")
```

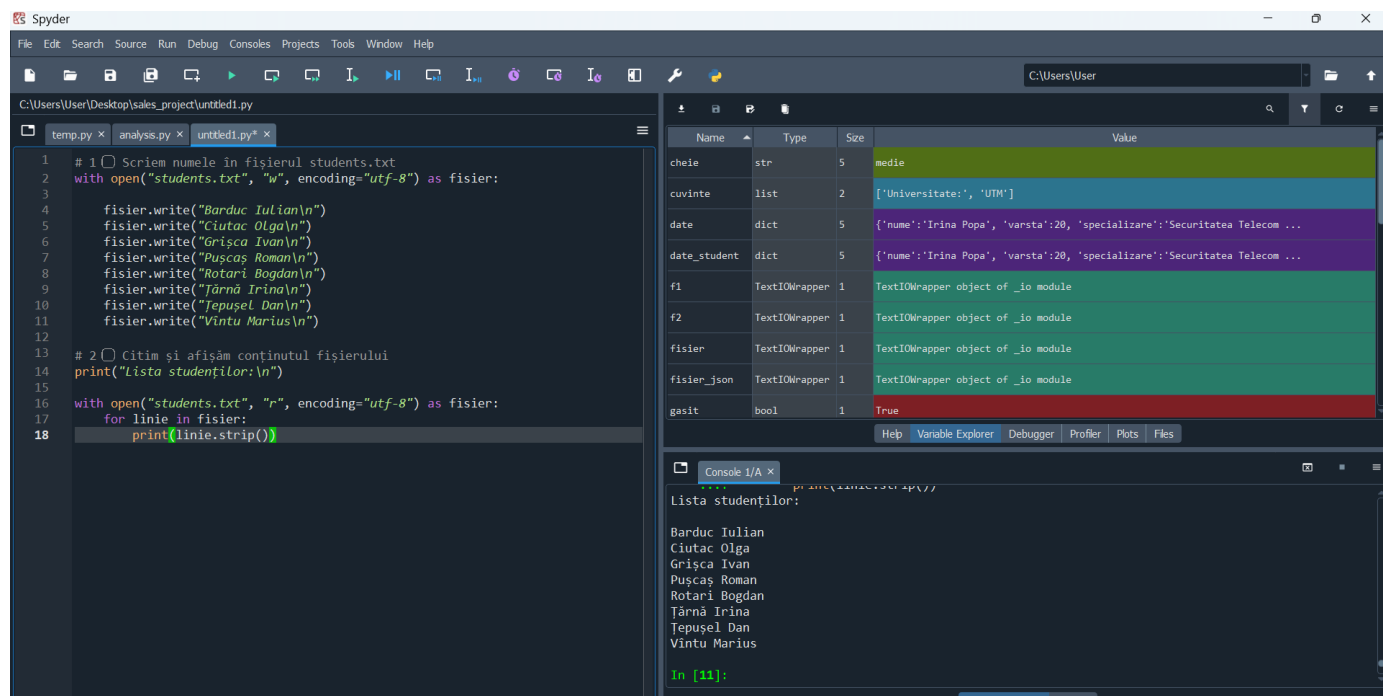
2 Citim și afișăm conținutul fișierului

print("Lista studenților:\n")

with open("students.txt", "r", encoding="utf-8") as fisier:

for linie in fisier:

print(linie.strip())



Sarcina 2: Adăugarea de date într-un fișier

Scrie un program care:

1. Deschide fișierul `students.txt` (dacă nu există, îl creează).
2. Adaugă noi nume de studenți în fișier fără a șterge datele existente.
3. Citește și afișează conținutul fișierului după adăugarea noilor date.

```
# 1 Deschidem fișierul în modul adăugare ("a")
# Nu șterge conținutul existent
with open("students.txt", "a", encoding="utf-8") as fisier:

    # Adăugăm studenți noi
    fisier.write("Popescu Andrei\n")
    fisier.write("Marin Alexandra\n")

# 2 Citim și afișăm conținutul actualizat
print("Lista actualizată a studenților:\n")

with open("students.txt", "r", encoding="utf-8") as fisier:
    for linie in fisier:
        print(linie.strip())
```

Sarcina 3: Numărarea liniilor dintr-un fișier

Scrie un program care:

1. Citește fișierul `test.txt`.
2. Numără și afișează numărul de linii din fișier.

```
import os

nume_fisier = "test.txt"

# Dacă fișierul nu există, îl creăm și scriem ceva în el
if not os.path.exists(nume_fisier):
    with open(nume_fisier, "w", encoding="utf-8") as fisier:
        fisier.write("Prima linie\n")
        fisier.write("A doua linie\n")
        fisier.write("A treia linie\n")

# Acum citim fișierul și numărăm liniile
with open(nume_fisier, "r", encoding="utf-8") as fisier:
    numar_linii = 0
    for linie in fisier:
        numar_linii += 1

print("Numărul total de linii din fișier este:", numar_linii)
```

Sarcina 4: Citirea unui fișier linie cu linie

Scrie un program care:

1. Deschide fișierul `data.txt`.
2. Citește și afișează doar prima linie din fișier.

```
# Importăm modulul os (opțional, doar pentru verificare)
import os
```

```
# Afișăm directorul curent (pentru verificare)
print("Directorul curent este:", os.getcwd())
```

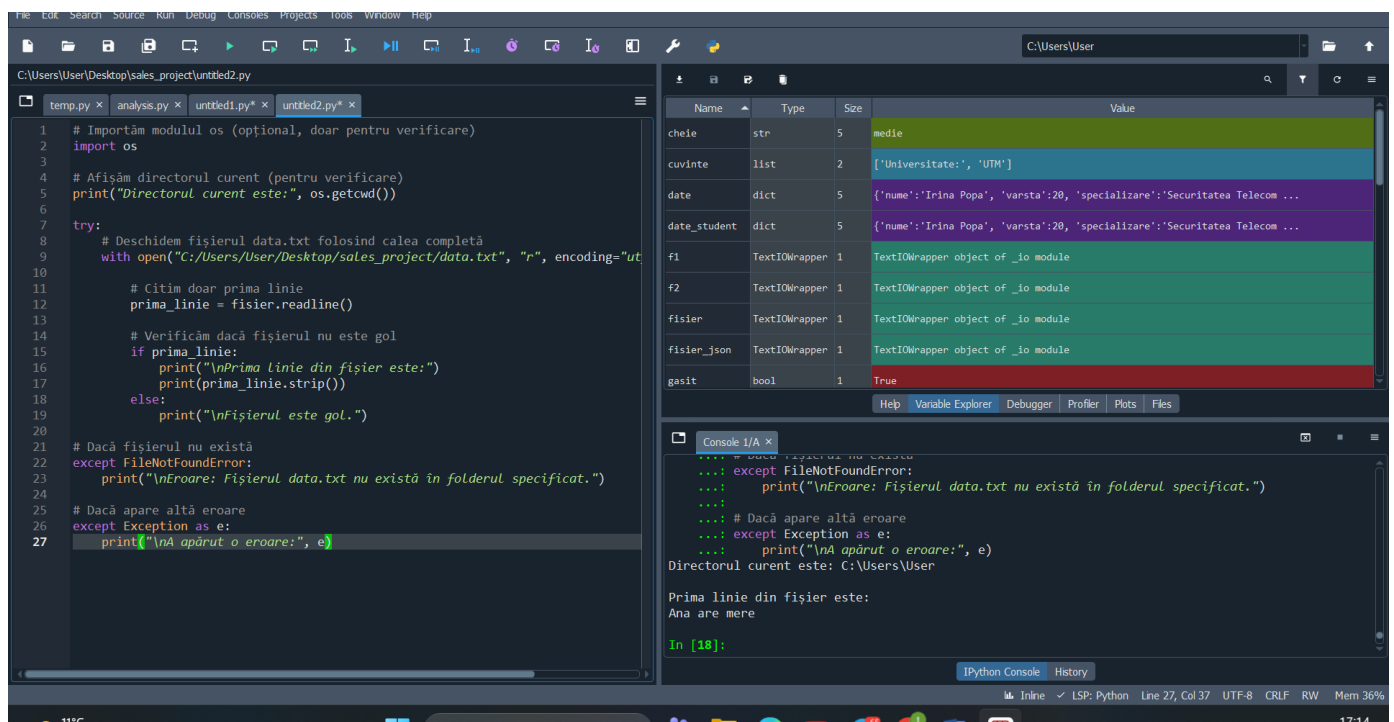
```
try:
    # Deschidem fișierul data.txt folosind calea completă
    with open("C:/Users/User/Desktop/sales_project/data.txt", "r", encoding="utf-8") as fisier:
```

```
    # Citim doar prima linie
    prima_linie = fisier.readline()
```

```
    # Verificăm dacă fișierul nu este gol
    if prima_linie:
        print("\nPrima linie din fișier este:")
        print(prima_linie.strip())
    else:
        print("\nFișierul este gol.")
```

```
# Dacă fișierul nu există
except FileNotFoundError:
    print("\nEroare: Fișierul data.txt nu există în folderul specificat.")
```

```
# Dacă apare altă eroare
except Exception as e:
    print("\nA apărut o eroare:", e)
```



Sarcina 5: Căutarea unui cuvânt într-un fișier

Scrie un program care:

1. Citește fișierul `data.txt`.
2. Căută un cuvânt specificat într-un fișier și afișează toate liniile care conțin acel cuvânt.

```
# Cerem utilizatorului să introducă cuvântul căutat
cuvant = input("Introdu cuvântul pe care vrei să îl cauți: ")

try:
    # Deschidem fișierul folosind calea completă
    with open("C:/Users/User/Desktop/sales_project/data.txt", "r", encoding="utf-8") as fisier:

        print("\nLiniile care conțin cuvântul căutat:\n")

        gasit = False # Variabilă pentru a verifica dacă găsim cuvântul

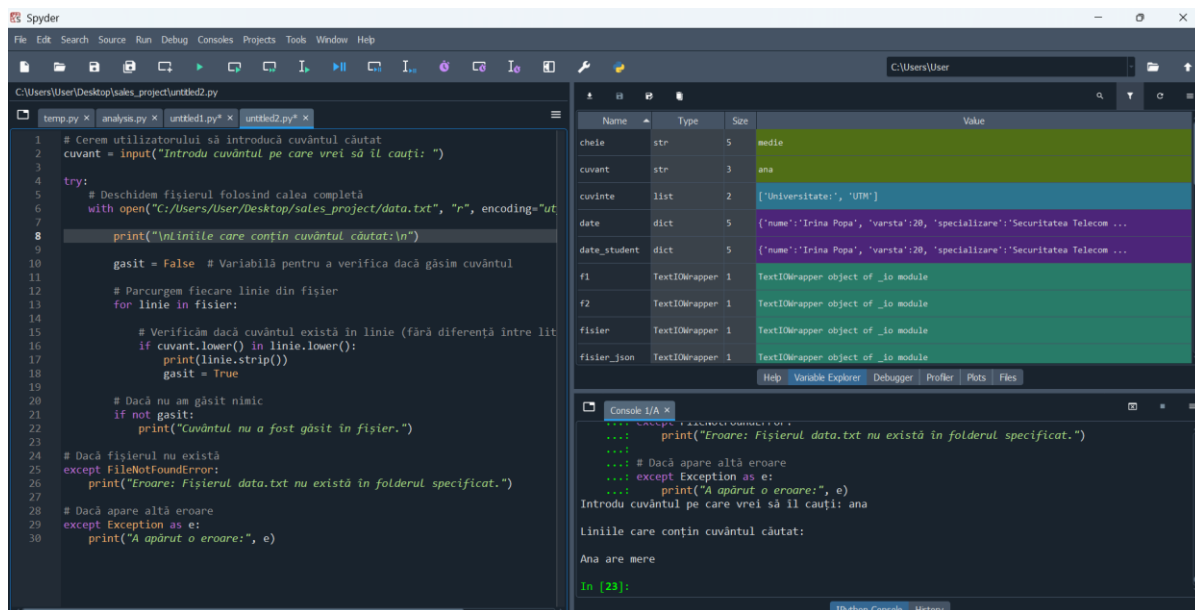
        # Parcurgem fiecare linie din fișier
        for linie in fisier:

            # Verificăm dacă cuvântul există în linie (fără diferență între litere mari/mici)
            if cuvant.lower() in linie.lower():
                print(linie.strip())
                gasit = True

        # Dacă nu am găsit nimic
        if not gasit:
            print("Cuvântul nu a fost găsit în fișier.")

# Dacă fișierul nu există
except FileNotFoundError:
    print("Eroare: Fișierul data.txt nu există în folderul specificat.")

# Dacă apare altă eroare
except Exception as e:
    print("A apărut o eroare:", e)
```



Sarcina 6: Ștergerea liniilor dintr-un fișier

Scrie un program care:

1. Deschide fișierul **data.txt**.
2. Șterge toate liniile care conțin cuvântul „eroare”.
3. Scrie înapoi fișierul fără liniile respective.

```
# 1 Deschidem fișierul în modul citire
with open("data.txt", "r", encoding="utf-8") as fișier:
    linii = fișier.readlines() # Citim toate liniile într-o listă
```

```
# 2 Filtrăm liniile care NU conțin cuvântul "eroare"
linii_filtrate = []
```

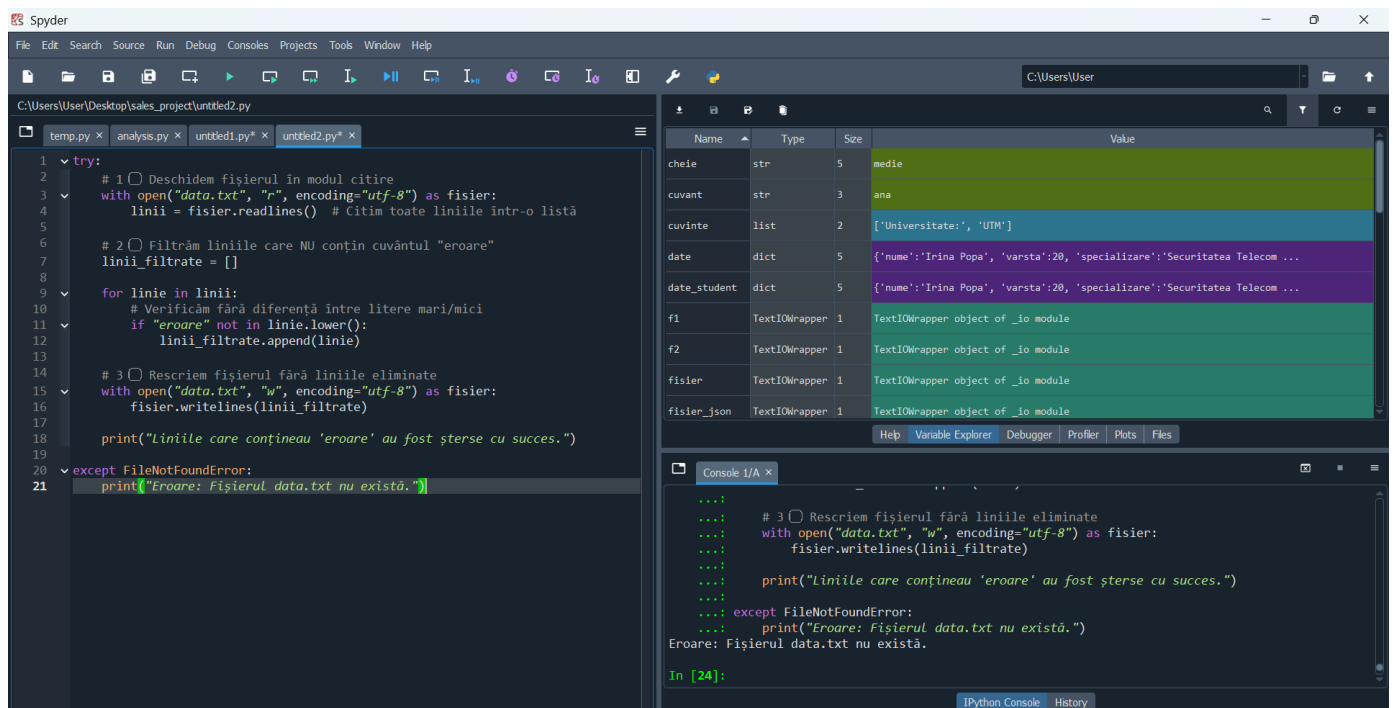
```
for linie in linii:
    # Verificăm fără diferență între litere mari/mici
    if "eroare" not in linie.lower():
        linii_filtrate.append(linie)
```

```
# 3 Rescriem fișierul fără liniile eliminate
with open("data.txt", "w", encoding="utf-8") as fișier:
    fișier.writelines(linii_filtrate)
```

```
print("Liniile care conțineau 'eroare' au fost șterse cu succes.")
```

```
except FileNotFoundError:
    print("Eroare: Fișierul data.txt nu există.")
```

Creciun Dumitrita GR TSRC-251,PYTHON



Sarcina 7: Sortarea liniilor dintr-un fișier

Scrie un program care:

1. Citește fișierul `data.txt`.
2. Sortează liniile în ordine alfabetică și salvează rezultatul într-un fișier nou numit `sorted_data.txt`.

Deschidem fișierul folosind calea completă

```
with open("C:/Users/User/Desktop/sales_project/data.txt", "r", encoding="utf-8") as fisier:
    linii = fisier.readlines()
```

Sortăm liniile

```
linii_sortate = sorted(linii)
```

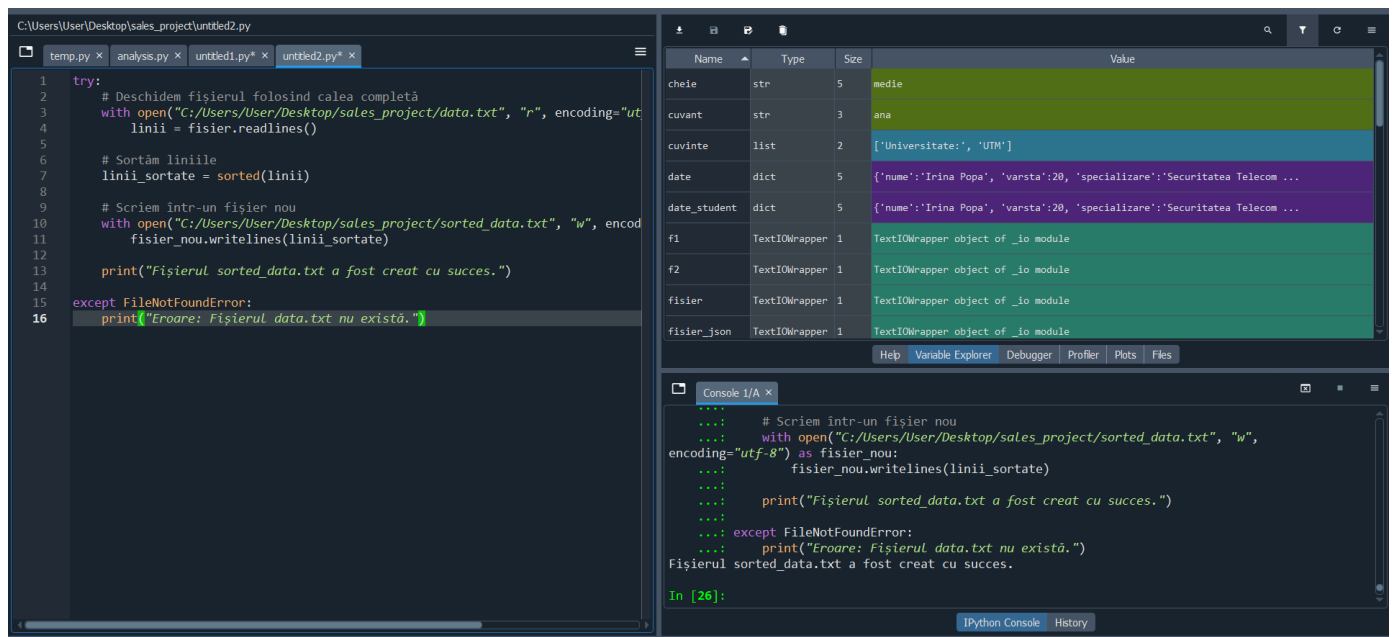
Scriem într-un fișier nou

```
with open("C:/Users/User/Desktop/sales_project/sorted_data.txt", "w", encoding="utf-8") as fisier_nou:
    fisier_nou.writelines(linii_sortate)
```

```
print("Fișierul sorted_data.txt a fost creat cu succes.")
```

except FileNotFoundError:

```
    print("Eroare: Fișierul data.txt nu există.")
```

Sarcina 8: Citirea și salvarea datelor dintr-un fișier CSV

Scrie un program care:

1. Citește fișierul CSV **students.csv**, care conține informații despre studenți (nume, vârstă, note).
2. Afișează datele citite din fișier sub formă de listă.

```
import csv
```

```
try:
```

```
    # Deschidem fișierul students.csv
```

```
    with open("students.csv", "r", encoding="utf-8") as fisier:
```

```
        reader = csv.reader(fisier)
```

```
        # Salvăm datele într-o listă
```

```
        studenti = []
```

```
        for rand in reader:
```

```
            studenti.append(rand)
```

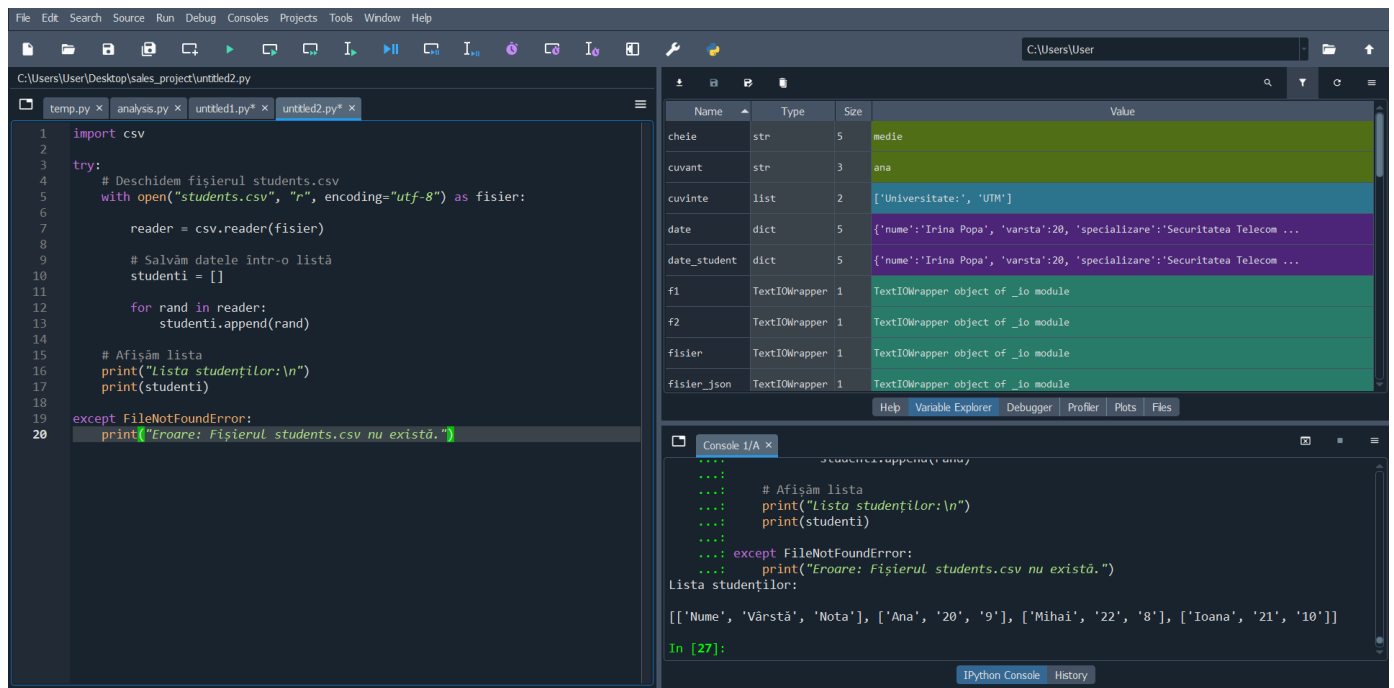
```
    # Afișăm lista
```

```
    print("Lista studenților:\n")
```

```
    print(studenti)
```

```
except FileNotFoundError:
```

```
    print("Eroare: Fișierul students.csv nu există.")
```



Sarcina 9: Conversia datelor din CSV în JSON

Scrie un program care:

1. Citește fișierul CSV **students.csv**.
2. Transformă datele din CSV în format JSON.
3. Salvează rezultatul în fișierul **students.json**.

```
import csv
import json
import os
```

```
print("Folderul curent este:", os.getcwd())
```

```
try:
```

```
    with open("students.csv", "r", encoding="utf-8") as fisier_csv:
        reader = csv.DictReader(fisier_csv)
        studenti = list(reader) # transformă CSV în listă de dicționare
```

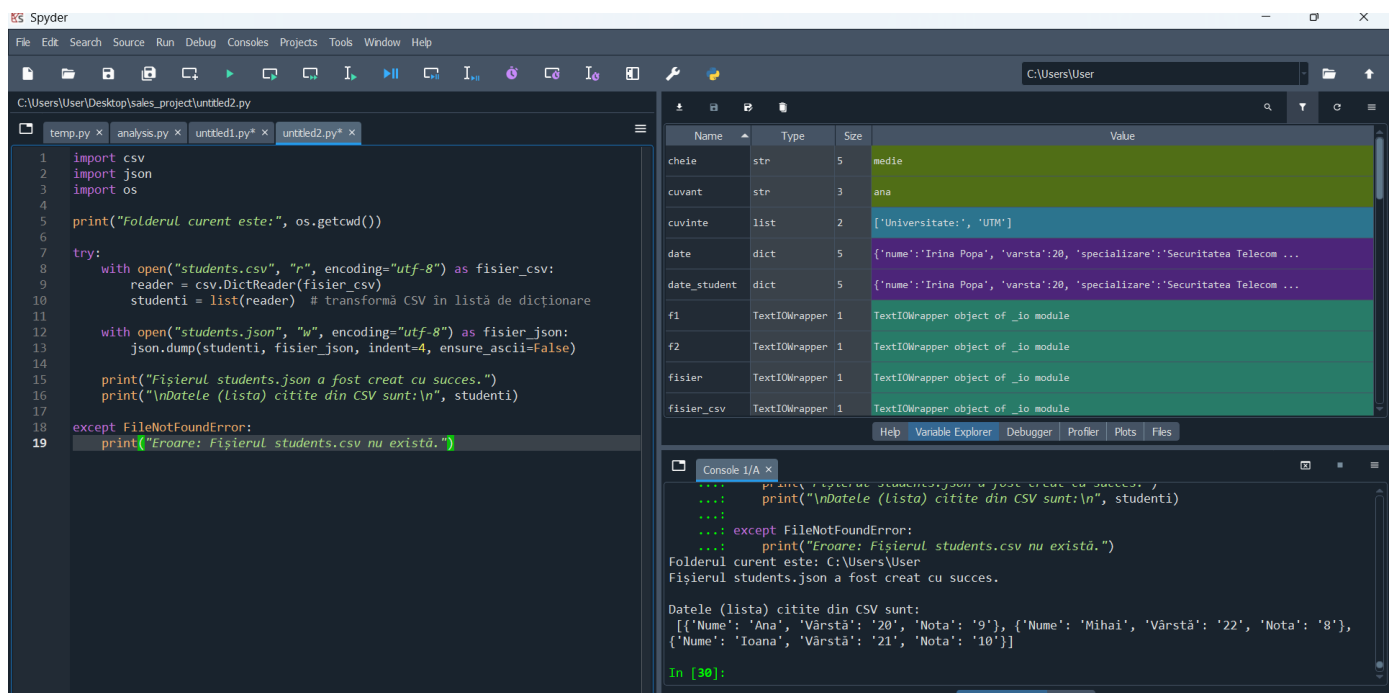
```
    with open("students.json", "w", encoding="utf-8") as fisier_json:
        json.dump(studenti, fisier_json, indent=4, ensure_ascii=False)
```

```
    print("Fișierul students.json a fost creat cu succes.")
    print("\nDatele (lista) citite din CSV sunt:\n", studenti)
```

```
except FileNotFoundError:
```

```
    print("Eroare: Fișierul students.csv nu există.")
```

Creciun Dumitrita GR TSRC-251,PYTHON



Sarcina 10: Îmbinarea mai multor fișiere într-un singur fișier

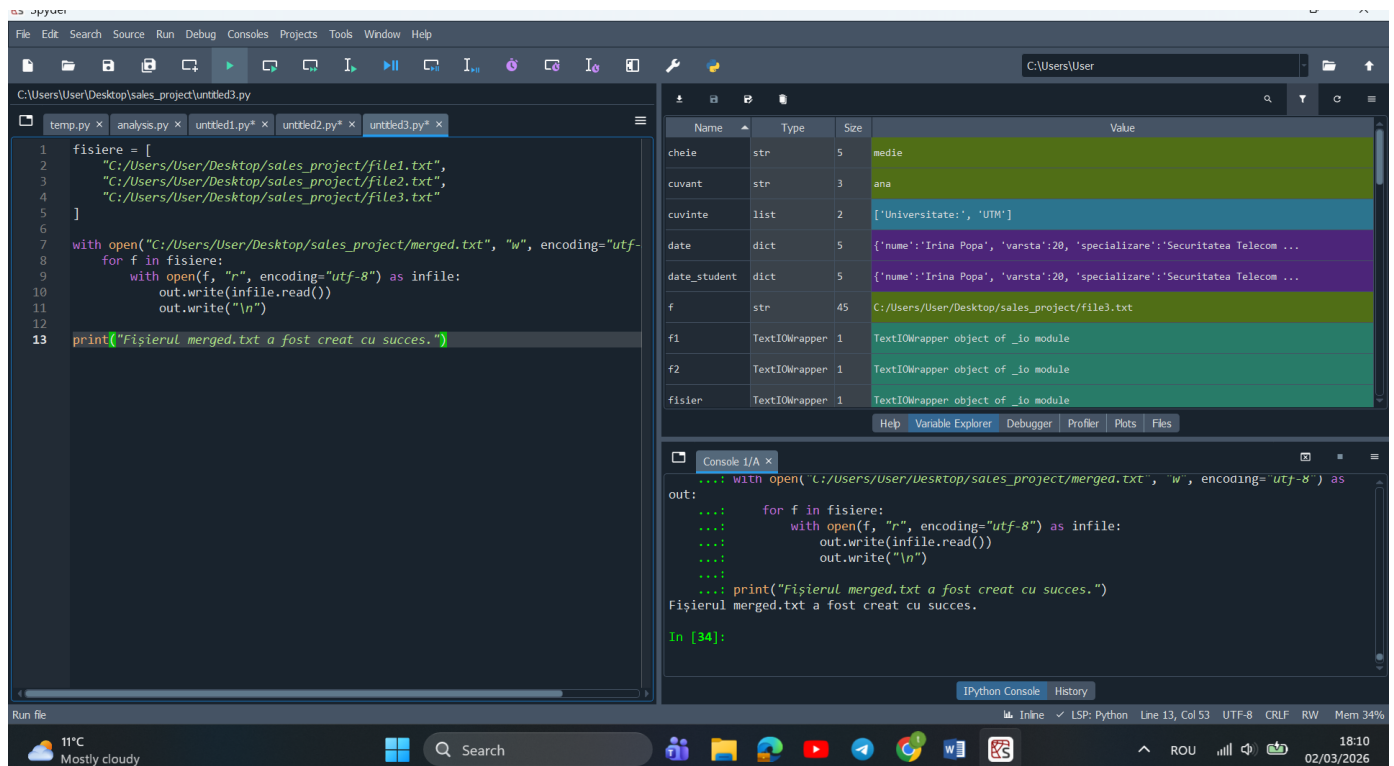
Scrie un program care:

1. Citește fișierele **file1.txt**, **file2.txt**, și **file3.txt**.
2. Îmbină toate conținuturile în fișierul **merged.txt**.

```
fisiere = [
    "C:/Users/User/Desktop/sales_project/file1.txt",
    "C:/Users/User/Desktop/sales_project/file2.txt",
    "C:/Users/User/Desktop/sales_project/file3.txt"
]
```

```
with open("C:/Users/User/Desktop/sales_project/merged.txt", "w", encoding="utf-8") as out:
    for f in fisiere:
        with open(f, "r", encoding="utf-8") as infile:
            out.write(infile.read())
            out.write("\n")
```

```
print("Fișierul merged.txt a fost creat cu succes.")
```



Sarcina 11: Modificarea textului unui fișier

Scrie un program care:

1. Deschide fișierul `text.txt`.
2. Transformă toate literele în majuscule.
3. Scrie textul modificat înapoi în același fișier.

```
import os
```

```
# Setăm manual folderul corect (unde ai fișierele)
```

```
folder = "C:/Users/User/Desktop/sales_project"
```

```
cale_fisier = os.path.join(folder, "data.txt")
```

```
try:
```

```
    with open(cale_fisier, "r", encoding="utf-8") as fisier:
        continut = fisier.read()
```

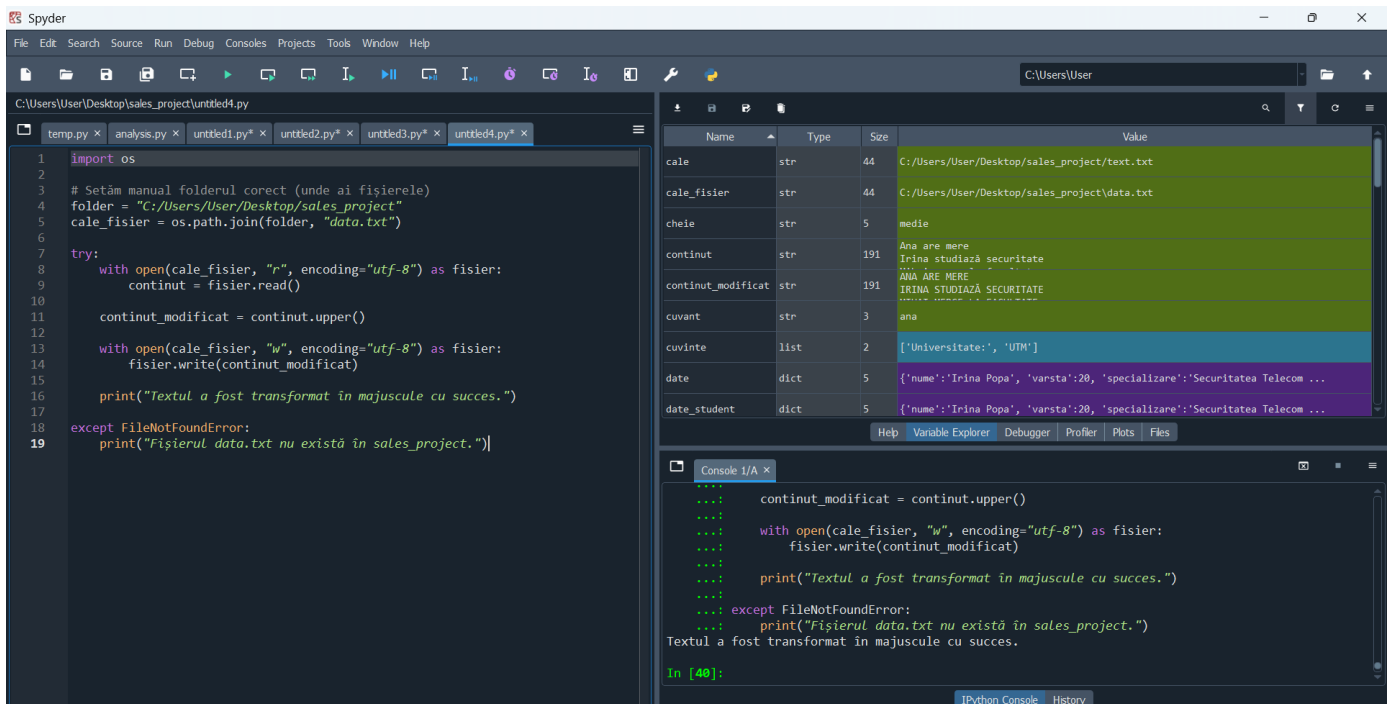
```
    continut_modificat = continut.upper()
```

```
    with open(cale_fisier, "w", encoding="utf-8") as fisier:
        fisier.write(continut_modificat)
```

```
    print("Textul a fost transformat în majuscule cu succes.")
```

```
except FileNotFoundError:
```

```
    print("Fișierul data.txt nu există în sales_project.")
```



Sarcina 12: Copierea unui fișier în altul

Scrie un program care:

1. Copiază conținutul fișierului **source.txt** într-un fișier nou **destination.txt**.
2. Programul trebuie să funcționeze chiar dacă fișierul de destinație există deja (și să îl suprascrie).

```
import os

# Numele fișierelor
source_file = "source.txt"
destination_file = "destination.txt"

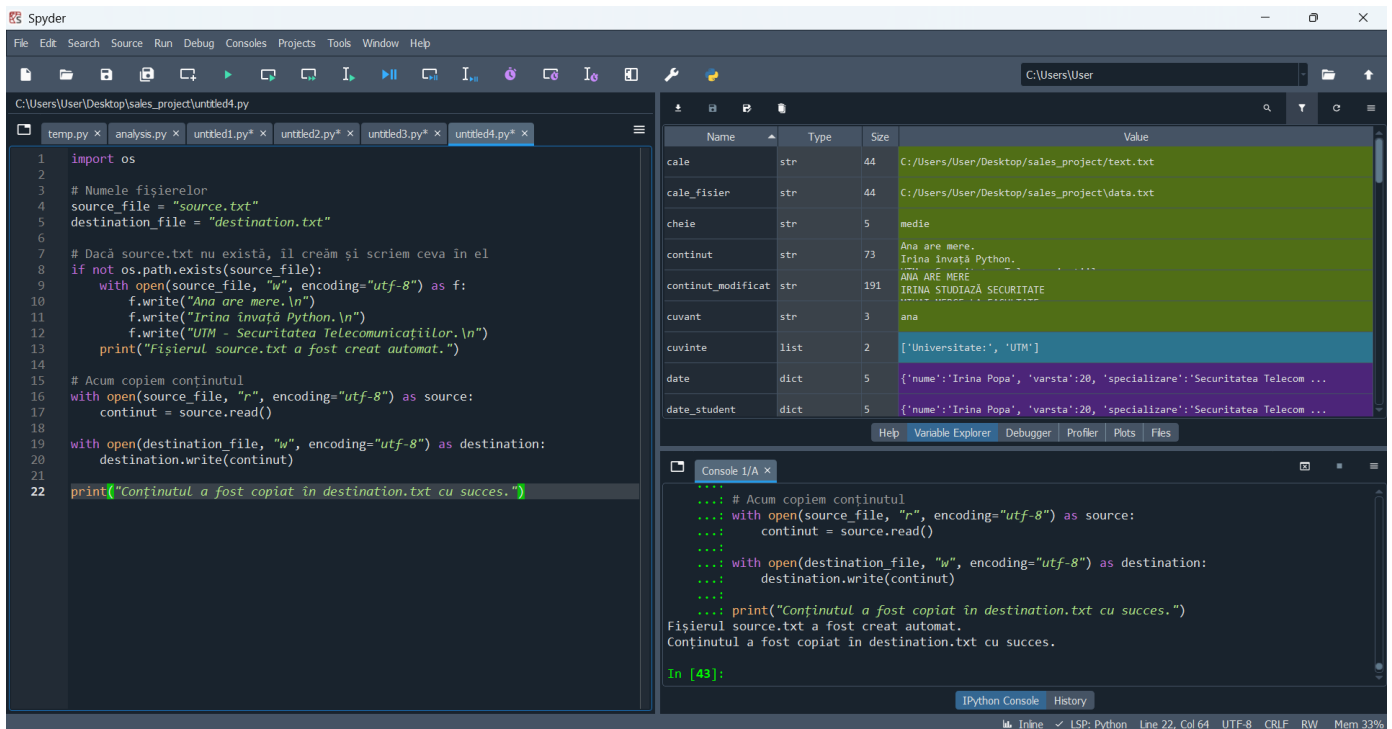
# Dacă source.txt nu există, îl creăm și scriem ceva în el
if not os.path.exists(source_file):
    with open(source_file, "w", encoding="utf-8") as f:
        f.write("Ana are mere.\n")
        f.write("Irina învață Python.\n")
        f.write("UTM - Securitatea Telecomunicațiilor.\n")
    print("Fișierul source.txt a fost creat automat.")

# Acum copiem conținutul
with open(source_file, "r", encoding="utf-8") as source:
    continut = source.read()

with open(destination_file, "w", encoding="utf-8") as destination:
    destination.write(continut)

print("Conținutul a fost copiat în destination.txt cu succes.")
```

Creciun Dumitrita GR TSRC-251,PYTHON



Sarcina 13: Citirea şi scrierea unui fişier binar

Scrie un program care:

1. Scrie mai multe numere întregi într-un fişier binar `numbers.bin`.
2. Citeşte numerele din fişierul binar şi le afişează pe ecran.

```
import struct
```

```
# Lista de numere întregi
numere = [10, 25, 50, 100, 200]
```

```
# 1 Scriem numerele în fişier binar
with open("numbers.bin", "wb") as fisier:
    for numar in numere:
        # "i" = integer (4 bytes)
        fisier.write(struct.pack("i", numar))
```

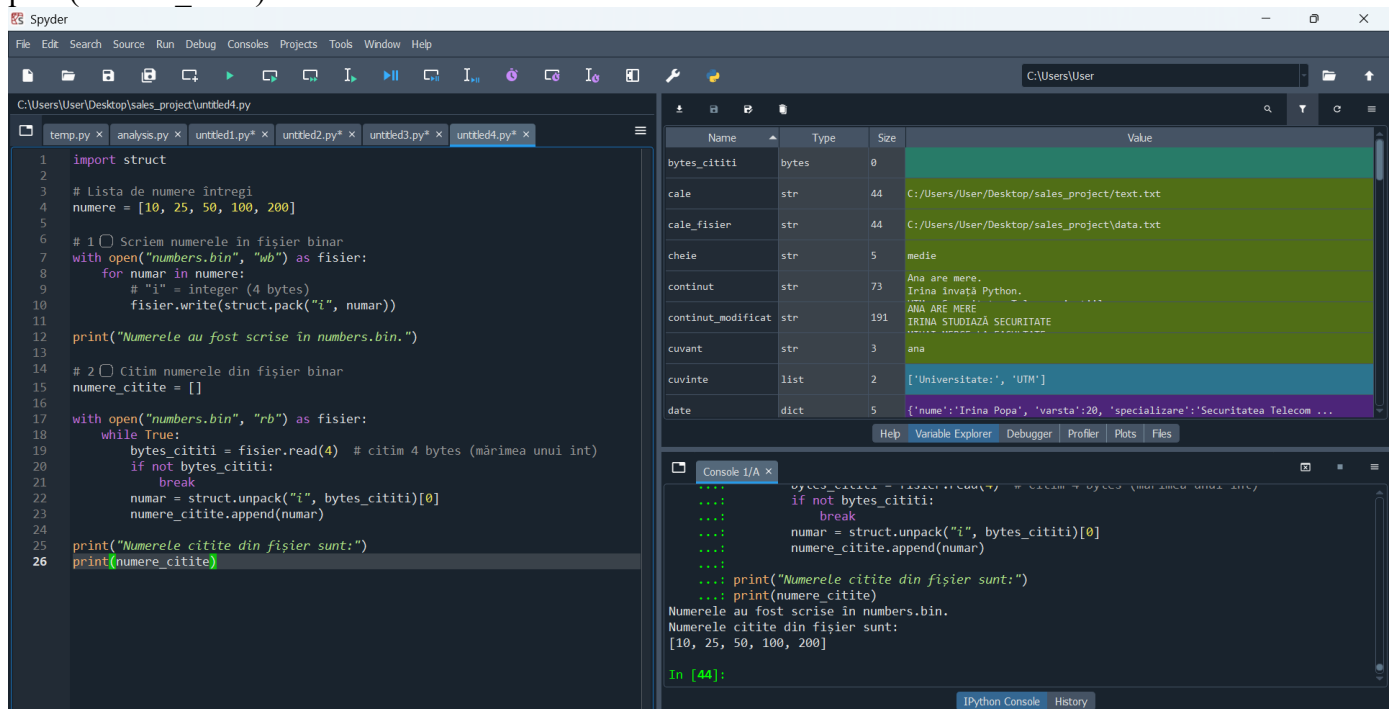
```
print("Numerele au fost scrise în numbers.bin.")
```

```
# 2 Citim numerele din fişier binar
numere_citite = []
```

```
with open("numbers.bin", "rb") as fisier:
    while True:
        bytes_cititi = fisier.read(4) # citim 4 bytes (mărimea unui int)
        if not bytes_cititi:
            break
        numar = struct.unpack("i", bytes_cititi)[0]
        numere_citite.append(numar)
```

```
print("Numerele citite din fişier sunt:")
```

```
print(numere_citite)
```



Sarcina 14: Crearea unei copii de siguranță a unui fișier

Scrie un program care:

1. Creează o copie de siguranță a fișierului **data.txt** într-un fișier numit **data_backup.txt**.
2. Programul trebuie să verifice dacă fișierul de backup există deja și, dacă da, să îl suprascrie.

```
import os
```

```
# Calea completă către folderul tău
folder = "C:/Users/User/Desktop/sales_project"
```

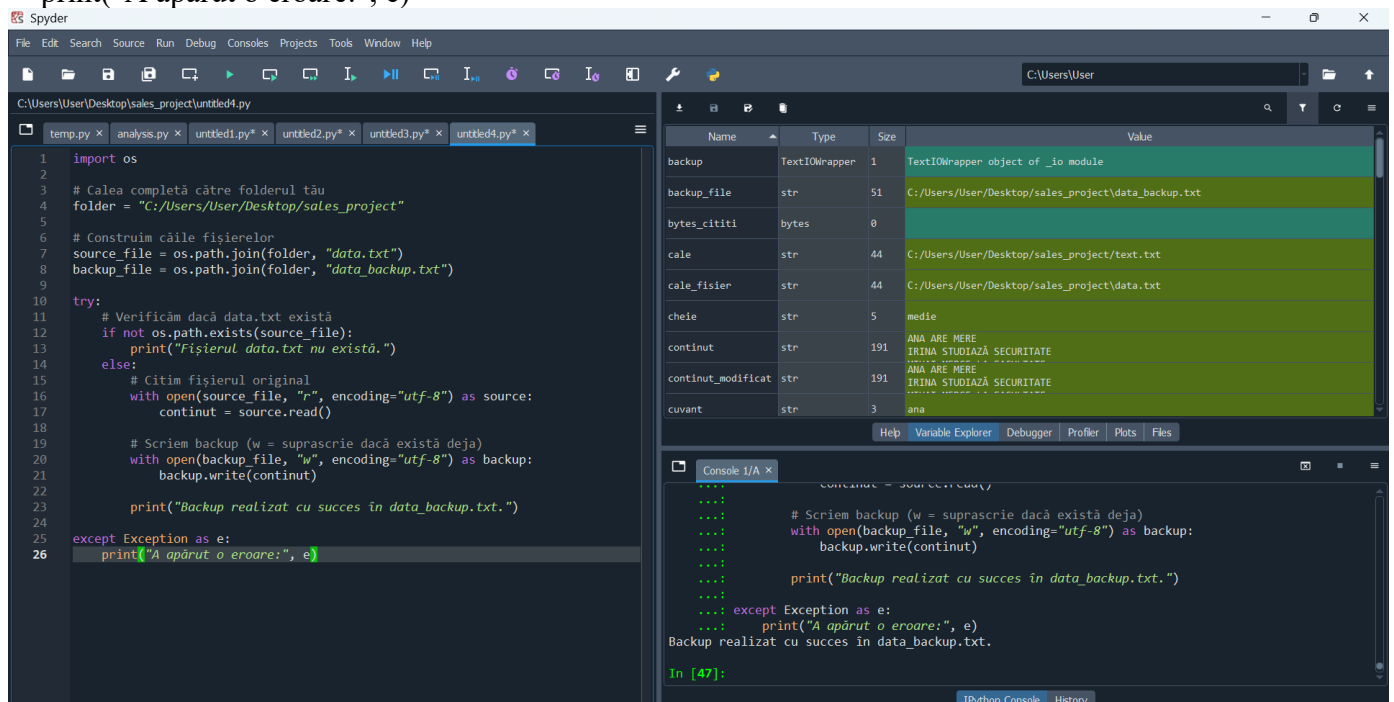
```
# Construim căile fișierelor
source_file = os.path.join(folder, "data.txt")
backup_file = os.path.join(folder, "data_backup.txt")
```

```
try:
    # Verificăm dacă data.txt există
    if not os.path.exists(source_file):
        print("Fișierul data.txt nu există.")
    else:
        # Citim fișierul original
        with open(source_file, "r", encoding="utf-8") as source:
            continut = source.read()

        # Scriem backup (w = suprascrie dacă există deja)
        with open(backup_file, "w", encoding="utf-8") as backup:
            backup.write(continut)

    print("Backup realizat cu succes în data_backup.txt.")
```

```
except Exception as e:
    print("A apărut o eroare:", e)
```



Sarcina 15: Numărarea frecvenței cuvintelor dintr-un fișier

Scrie un program care:

1. Citește fișierul **text.txt**.
2. Numără câte ori apare fiecare cuvânt în fișier.
3. Afișează un raport cu cuvintele și frecvențele lor în fișier.

```
import string
```

```
cale = "C:/Users/User/Desktop/sales_project/text.txt"
```

```
try:
    with open(cale, "r", encoding="utf-8") as f:
        continut = f.read()
```

```
continut = continut.lower()
```

```
for semn in string.punctuation:
    continut = continut.replace(semn, "")
```

```
cuvinte = continut.split()
```

```
frecventa = {}
```

```
for cuvant in cuvinte:
    frecventa[cuvant] = frecventa.get(cuvant, 0) + 1
```

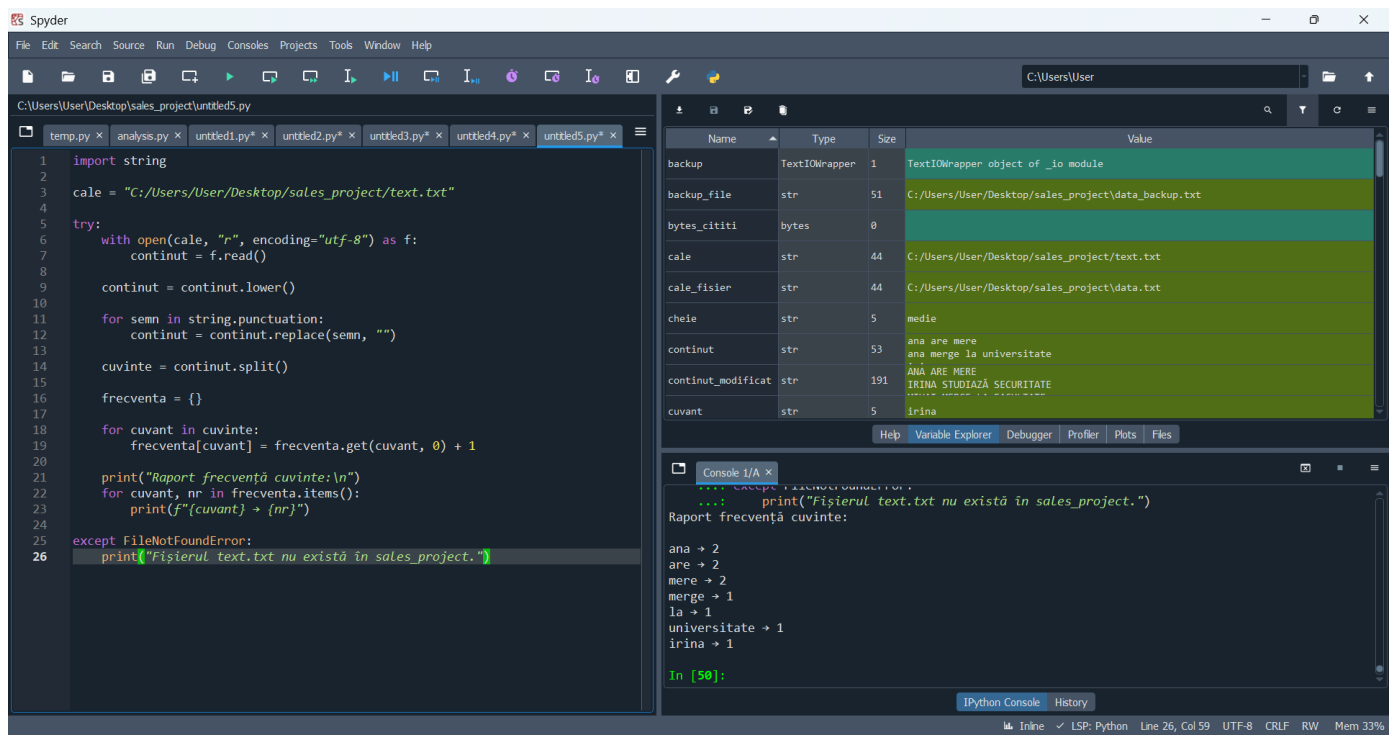
```
print("Raport frecvență cuvinte:\n")
for cuvant, nr in frecventa.items():
```



```
print(f'{cuvant} → {nr}')
```

```
except FileNotFoundError:
```

```
print("Fișierul text.txt nu există în sales_project.")
```



Instrucțiuni pentru realizare:

1. Folosește funcțiile de bază din Python pentru manipularea fișierelor: **open**, **read**, **write**, **writelines**, **readlines**.
2. Încearcă să folosești **with** pentru a deschide fișiere, astfel încât fișierele să fie gestionate corect și să fie închise automat după utilizare.
3. În toate sarcinile care implică scrierea în fișiere, asigură-te că fișierul este deschis în modul corect (de exemplu, **'w'** pentru scriere, **'a'** pentru adăugare).
4. În cazul fișierelor **CSV** și **JSON**, folosește librăriile Python standard (**csv** și **json**).
5. Asigură-te că programele tale gestionează corect erorile, cum ar fi fișierele inexistente sau fișierele goale.

Rezultate așteptate:

Fiecare sarcină trebuie să fie realizată cu succes, programul citind, modificând sau creând fișiere conform cerințelor, iar în final afișând rezultatele corecte.

II. PARTE(40 puncte)

SARCINI LA TEMA "GESTIUNEA FIȘIERELOR ÎN PYTHON"

Sarcina 1: Crearea unui sistem de logare cu rotație de fișiere

Scrie un program care:

1. Creează un fișier de logare (`log.txt`) și scrie mesaje de logare cu timestamp (data și ora).
2. După fiecare 1000 de mesaje scrise, creează un fișier de log nou numit `log_1.txt`, `log_2.txt`, etc.
3. Programul trebuie să asigure că fișierele vechi nu sunt șterse și să păstreze un istoric de loguri.

```
import os
from datetime import datetime

# ===== SETĂRI =====
FOLDER_LOGS = "C:/Users/User/Desktop/sales_project" # schimbă dacă ai alt folder
MESAJ_TOTAL = 5000                                # câte mesaje vrei să scrii în total
LIMITA_PE_FISIER = 1000                            # după câte mesaje trecem la alt fișier

# Ne asigurăm că folderul există
os.makedirs(FOLDER_LOGS, exist_ok=True)

def nume_fisier_log(index: int) -> str:
    """
    index = 0 -> log.txt
    index = 1 -> log_1.txt
    index = 2 -> log_2.txt
    ...
    """
    if index == 0:
        return "log.txt"
    return f"log_{index}.txt"

# ===== SCRIERE LOGURI =====
fisier_index = 0      # 0 = log.txt, apoi 1 = log_1.txt etc.
mesaje_in_fisier = 0  # câte mesaje am scris în fișierul curent

for i in range(1, MESAJ_TOTAL + 1):
    # Timestamp: data + ora (ex: 2026-03-02 18:30:15)
    timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")

    # Construim mesajul de log
    mesaj = f"[{timestamp}] Mesaj log #{i}\n"

    # Calea completă către fișierul curent de log
    cale_log = os.path.join(FOLDER_LOGS, nume_fisier_log(fisier_index))

    # Deschidem în modul "a" (append) ca să NU ștergem ce exista deja
    with open(cale_log, "a", encoding="utf-8") as f:
        f.write(mesaj)

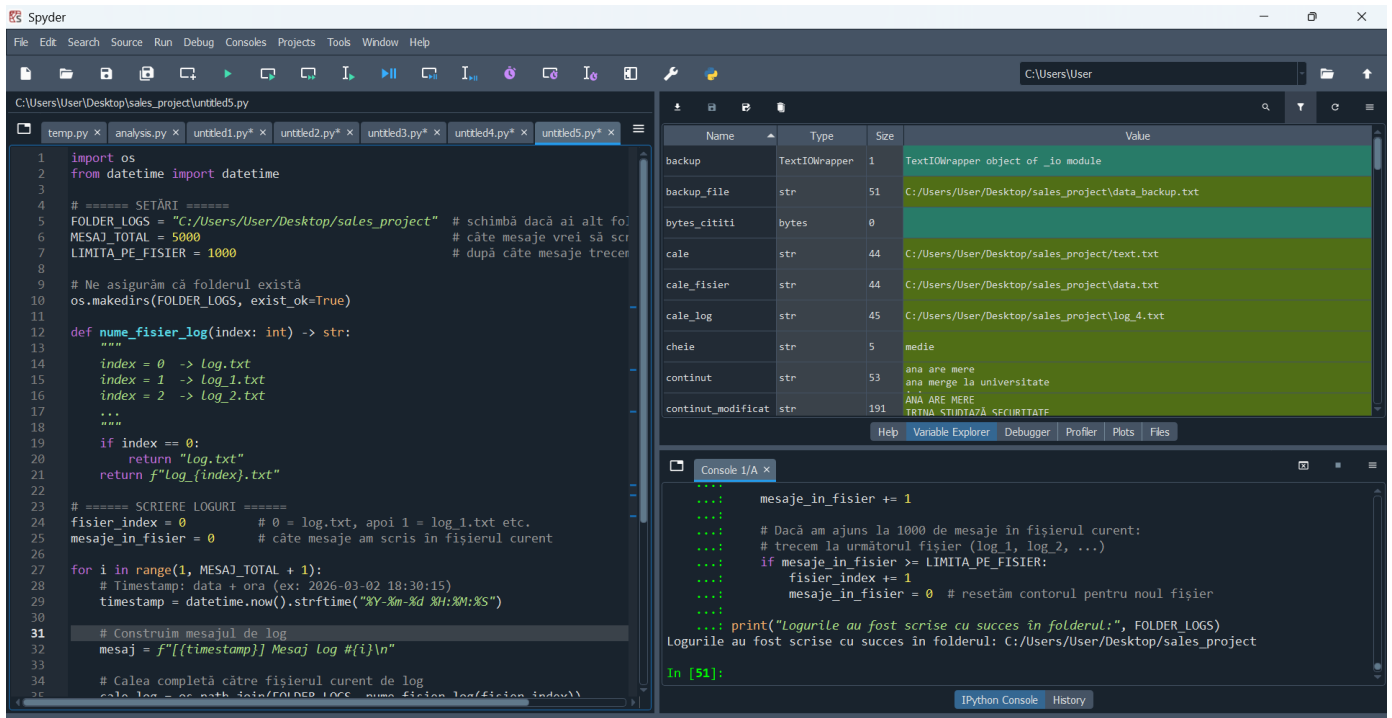
    mesaje_in_fisier += 1

    # Dacă am ajuns la 1000 de mesaje în fișierul curent:
    # trecem la următorul fișier (log_1, log_2, ...)
    if mesaje_in_fisier >= LIMITA_PE_FISIER:
        fisier_index += 1
```

Creciun Dumitrita GR TSRC-251,PYTHON

```
mesaje_in_fisier = 0 # resetăm contorul pentru noul fișier
```

```
print("Logurile au fost scrise cu succes în folderul:", FOLDER_LOGS)
```



Sarcina 2: Criptarea și decriptarea unui fișier

Scrie un program care:

1. Citește un fișier text `data.txt` și criptează conținutul acestuia folosind o metodă de criptare simetrică (de exemplu, AES).
2. Scrie conținutul criptat într-un fișier binar `encrypted_data.bin`.
3. Crează o funcție care decriptează fișierul criptat și scrie rezultatul într-un fișier `decrypted_data.txt`.

Criptare simetrică folosind Fernet (din pachetul cryptography)

Fernet folosește AES + autentificare (HMAC) => criptare sigură și ușor de folosit

```
from cryptography.fernet import Fernet
```

```
import os
```

Folderul tău (ca să nu mai apară problema cu "nu există fișierul")

```
FOLDER = "C:/Users/User/Desktop/sales_project"
```

Fișiere

```
INPUT_TXT = os.path.join(FOLDER, "data.txt")
```

```
KEY_FILE = os.path.join(FOLDER, "secret.key") # cheia se salvează aici
```

```
ENCRYPTED_BIN = os.path.join(FOLDER, "encrypted_data.bin")
```

```
DECRYPTED_TXT = os.path.join(FOLDER, "decrypted_data.txt")
```

```
def get_or_create_key(key_path: str) -> bytes:
```

```
    # Dacă există deja cheia, o citim (ca să poți decripta aceleași date mai târziu)
```

```
if os.path.exists(key_path):
    with open(key_path, "rb") as f:
        return f.read()

# Dacă nu există, generăm o cheie nouă și o salvăm
key = Fernet.generate_key()
with open(key_path, "wb") as f:
    f.write(key)
return key

def encrypt_file(input_path: str, encrypted_path: str, key_path: str) -> None:
    # Luăm cheia (o creăm dacă nu există)
    key = get_or_create_key(key_path)

    # Cream obiectul Fernet cu cheia (criptare simetrică)
    fernet = Fernet(key)

    # Citim textul original din data.txt
    with open(input_path, "rb") as f:
        plaintext_bytes = f.read() # bytes (nu str)

    # Criptăm conținutul (rezultatul este bytes)
    ciphertext_bytes = fernet.encrypt(plaintext_bytes)

    # Scriem conținutul criptat într-un fișier binar
    with open(encrypted_path, "wb") as f:
        f.write(ciphertext_bytes)

def decrypt_file(encrypted_path: str, decrypted_path: str, key_path: str) -> None:
    # Citim cheia (trebuie să fie aceeași folosită la criptare)
    with open(key_path, "rb") as f:
        key = f.read()

    fernet = Fernet(key)

    # Citim datele criptate din fișierul binar
    with open(encrypted_path, "rb") as f:
        ciphertext_bytes = f.read()

    # Decriptăm (obținem bytes-ul original)
    plaintext_bytes = fernet.decrypt(ciphertext_bytes)

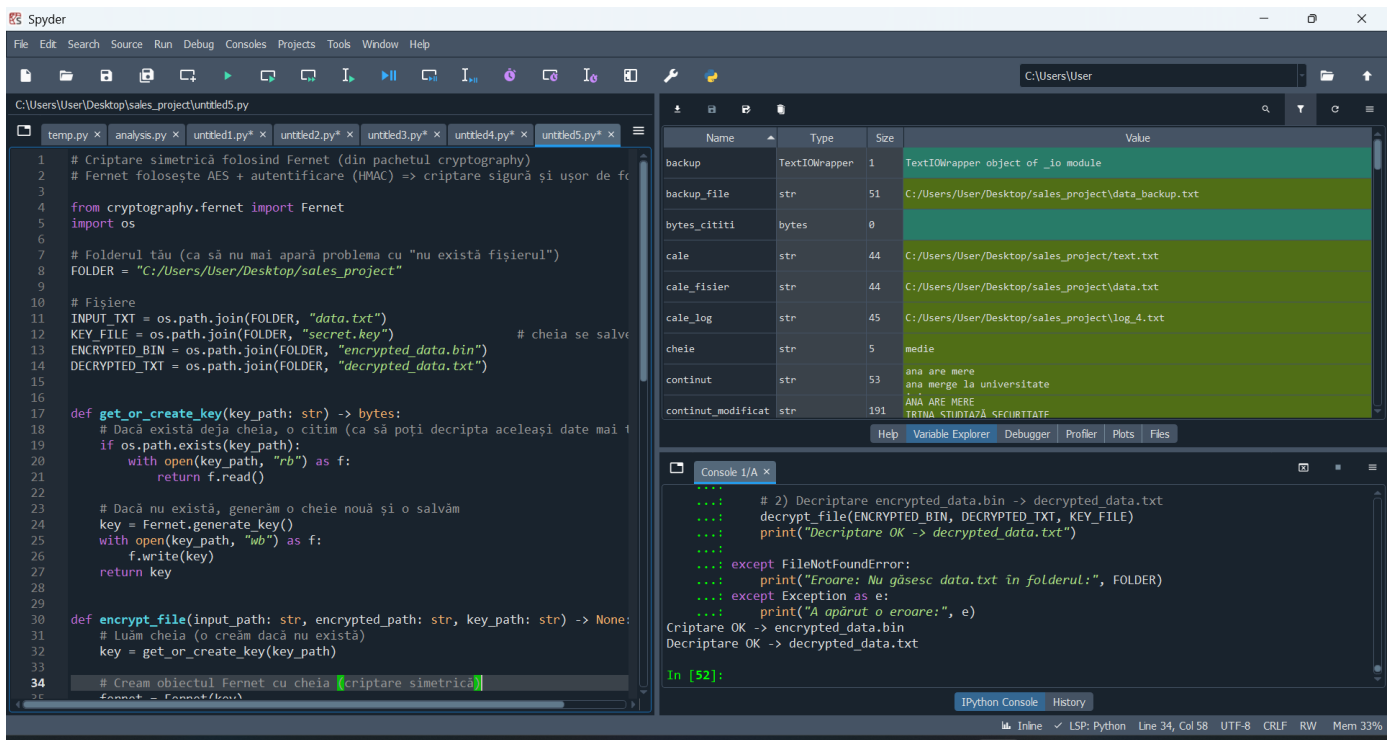
    # Scriem rezultatul decriptat într-un fișier text
    with open(decrypted_path, "wb") as f:
        f.write(plaintext_bytes)

# ===== RULARE =====
try:
    # 1) Criptare data.txt -> encrypted_data.bin
```

```
encrypt_file(INPUT_TXT, ENCRYPTED_BIN, KEY_FILE)
print("Criptare OK -> encrypted_data.bin")
```

```
# 2) Decriptare encrypted_data.bin -> decrypted_data.txt
decrypt_file(ENCRYPTED_BIN, DECRYPTED_TXT, KEY_FILE)
print("Decriptare OK -> decrypted_data.txt")
```

```
except FileNotFoundError:
    print("Eroare: Nu găsesc data.txt în folderul:", FOLDER)
except Exception as e:
    print("A apărut o eroare:", e)
```



Sarcina 3: Citirea și procesarea unui fișier JSON mare

Scrie un program care:

1. Citește un fișier JSON mare (de exemplu, 1 GB sau mai mult) folosind o metodă de citire incrementală (linie cu linie sau chunk-uri de date).
2. Extrage anumite informații din acest fișier (de exemplu, valorile unui anumit câmp) și le salvează într-un fișier CSV.
3. Programul trebuie să fie eficient și să folosească memorie cât mai redusă.

```
import json
import csv
import os
import random
```

```
FOLDER = "C:/Users/User/Desktop/sales_project"
INPUT_JSONL = os.path.join(FOLDER, "big.jsonl")
OUTPUT_CSV = os.path.join(FOLDER, "extracted.csv")
```

```
FIELDS = ["id", "name", "category", "price"]
```

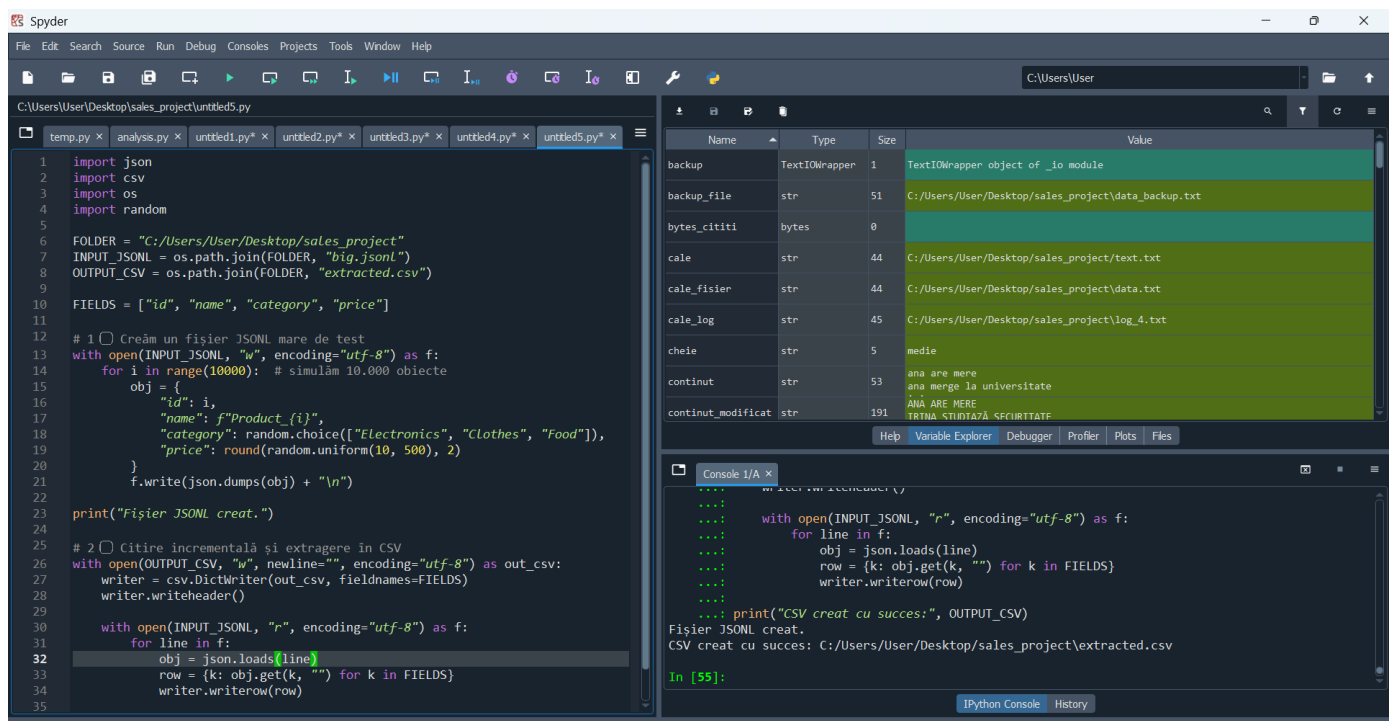
```
# 1 Creăm un fișier JSONL mare de test
with open(INPUT_JSONL, "w", encoding="utf-8") as f:
    for i in range(10000): # simulăm 10.000 obiecte
        obj = {
            "id": i,
            "name": f"Product_{i}",
            "category": random.choice(["Electronics", "Clothes", "Food"]),
            "price": round(random.uniform(10, 500), 2)
        }
        f.write(json.dumps(obj) + "\n")

print("Fișier JSONL creat.")

# 2 Citire incrementală și extragere în CSV
with open(OUTPUT_CSV, "w", newline="", encoding="utf-8") as out_csv:
    writer = csv.DictWriter(out_csv, fieldnames=FIELDS)
    writer.writeheader()

    with open(INPUT_JSONL, "r", encoding="utf-8") as f:
        for line in f:
            obj = json.loads(line)
            row = {k: obj.get(k, "") for k in FIELDS}
            writer.writerow(row)

print("CSV creat cu succes:", OUTPUT_CSV)
```



Sarcina 4: Verificarea integrității fișierelor prin compararea hash-urilor

Scrie un program care:

1. Citește mai multe fișiere dintr-un director dat.

2. Calculează hash-ul (SHA-256) al fiecărui fișier și îl salvează într-un fișier `file_hashes.txt`.
3. Permite utilizatorului să verifice dacă fișierele au fost modificate comparând hash-urile salvate anterior cu noile hash-uri calculate.

```
# Program: Hash SHA-256 pentru fișiere + verificare modificări
# 1) Citește mai multe fișiere dintr-un director
# 2) Calculează hash SHA-256 pentru fiecare fișier și salvează în file_hashes.txt
# 3) Permite verificarea dacă fișierele s-au modificat, comparând hash-urile vechi cu cele noi
```

```
import os          # lucrăm cu directoare și fișiere (listare, căi)
import hashlib     # calculăm hash SHA-256
from typing import Dict # tipuri (pentru claritate)
```

```
# Numele fișierului unde salvăm hash-urile
HASH_FILE = "file_hashes.txt"
```

```
def sha256_of_file(file_path: str, chunk_size: int = 1024 * 1024) -> str:
```

```
    """
```

```
    Calculează SHA-256 pentru un fișier, citind în bucăți (chunk-uri).
```

```
    chunk_size = 1 MB (ca să fie eficient și pentru fișiere mari)
```

```
    """
```

```
    # Cream un obiect hash SHA-256 (la început gol)
```

```
    hasher = hashlib.sha256()
```

```
    # Deschidem fișierul în mod binar (rb) ca să citim bytes corect
```

```
    with open(file_path, "rb") as f:
```

```
        while True:
```

```
            # Citim o bucată de date din fișier
```

```
            chunk = f.read(chunk_size)
```

```
            # Dacă nu mai avem date, ieșim din buclă
```

```
            if not chunk:
```

```
                break
```

```
            # Actualizăm hash-ul cu bucata citită
```

```
            hasher.update(chunk)
```

```
    # Returnăm hash-ul final în format hex (text)
```

```
    return hasher.hexdigest()
```

```
def get_files_from_directory(directory: str) -> list:
```

```
    """
```

```
    Returnează lista fișierelor (căi complete) din director.
```

```
    Ignoră sub-directoarele și ignoră file_hashes.txt (ca să nu se includă în verificare).
```

```
    """
```

```
    files = []
```

```
    # Parcurgem tot ce este în director
```

```
    for name in os.listdir(directory):
```

```
        # Construim calea completă: director + nume fișier
```

```
        full_path = os.path.join(directory, name)
```

```
# Dacă este fișier (nu folder) și nu este HASH_FILE, îl adăugăm
if os.path.isfile(full_path) and name != HASH_FILE:
    files.append(full_path)

return files

def compute_hashes(directory: str) -> Dict[str, str]:
    """
    Calculează hash-urile pentru toate fișierele din director.
    Returnează un dicționar: {nume_fișier: hash}
    """
    hashes = {}

    # Luăm fișierele din director
    files = get_files_from_directory(directory)

    # Pentru fiecare fișier calculăm SHA-256
    for file_path in files:
        filename = os.path.basename(file_path) # doar numele fișierului
        file_hash = sha256_of_file(file_path) # hash SHA-256
        hashes[filename] = file_hash # salvăm în dicționar

    return hashes

def save_hashes(directory: str, hashes: Dict[str, str]) -> None:
    """
    Salvează hash-urile în file_hashes.txt în directorul dat.
    Format pe linie: nume_fișier|hash
    """
    # Construim calea completă către fișierul de hash-uri
    out_path = os.path.join(directory, HASH_FILE)

    # Deschidem fișierul în modul scriere (w) => suprascrie dacă există
    with open(out_path, "w", encoding="utf-8") as out:
        # Scriem fiecare fișier și hash-ul lui pe o linie
        for filename, h in sorted(hashes.items()):
            out.write(f'{filename}|{h}\n')

    print(f"✓ Hash-urile au fost salvate în: {out_path}")

def load_hashes(directory: str) -> Dict[str, str]:
    """
    Încarcă hash-urile salvate anterior din file_hashes.txt.
    Returnează {nume_fișier: hash}.
    Dacă fișierul nu există, returnează dicționar gol.
    """
    path = os.path.join(directory, HASH_FILE)

    # Dacă nu există fișierul de hash-uri, nu avem ce încărca
```



```
if not os.path.exists(path):
    return {}

hashes = {}

# Deschidem fișierul în modul citire
with open(path, "r", encoding="utf-8") as f:
    for line in f:
        line = line.strip()
        # Sărim peste liniile goale
        if not line:
            continue

        # Împărțim linia în: nume_fișier și hash
        # Split la primul "|" (în caz că un nume conține accidental "|", foarte rar)
        filename, h = line.split("|", 1)
        hashes[filename] = h

return hashes

def compare_hashes(old_hashes: Dict[str, str], new_hashes: Dict[str, str]) -> None:
    """
    Compară hash-urile vechi cu cele noi și afișează:
    - fișiere modificate
    - fișiere noi
    - fișiere șterse
    - fișiere nemodificate
    """
    old_files = set(old_hashes.keys())
    new_files = set(new_hashes.keys())

    # Fișiere noi: apar acum, dar nu existau înainte
    added = new_files - old_files

    # Fișiere șterse: existau înainte, dar acum lipsesc
    removed = old_files - new_files

    # Fișiere comune: existau și înainte și acum
    common = old_files & new_files

    modified = []
    unchanged = []

    # Pentru fișierele comune, verificăm dacă hash-ul e diferit
    for filename in sorted(common):
        if old_hashes[filename] != new_hashes[filename]:
            modified.append(filename)
        else:
            unchanged.append(filename)

    print("\n===== REZULTAT VERIFICARE =====")
```

if modified:

```
print("\n● Fișiere MODIFICATE:")
for f in modified:
    print(" -", f)
```

else:

```
print("\n✓ Nu există fișiere modificate.")
```

if added:

```
print("\n● Fișiere NOI:")
for f in sorted(added):
    print(" -", f)
```

if removed:

```
print("\n● Fișiere ȘTERSE:")
for f in sorted(removed):
    print(" -", f)
```

if unchanged:

```
print("\n● Fișiere NEMODIFICATE:")
for f in unchanged:
    print(" -", f)
```

```
print("\n=====")
```

def main():

Cerem utilizatorului directorul în care sunt fișierele

```
directory = input("Introdu calea către director (ex: C:/Users/User/Desktop/sales_project): ").strip()
```

Dacă utilizatorul nu dă nimic, folosim directorul curent

if not directory:

```
    directory = os.getcwd()
```

Verificăm dacă directorul există

if not os.path.isdir(directory):

```
    print("✗ Directorul introdus nu există.")
    return
```

```
print("\nAlege o opțiune:")
```

```
print("1) Generează și salvează hash-urile (prima rulare / după actualizare)")
```

```
print("2) Verifică dacă fișierele au fost modificate (compară cu hash-urile salvate)")
```

```
opt = input("Opțiunea ta (1/2): ").strip()
```

if opt == "1":

Calculăm hash-uri noi și le salvăm

```
new_hashes = compute_hashes(directory)
```

```
save_hashes(directory, new_hashes)
```

elif opt == "2":

```

# Încărcăm hash-urile vechi
old_hashes = load_hashes(directory)

# Dacă nu există hash-uri salvate, nu putem compara
if not old_hashes:
    print(f"✗ Nu există {HASH_FILE} în acest director. Rulează opțiunea 1 mai întâi.")
    return

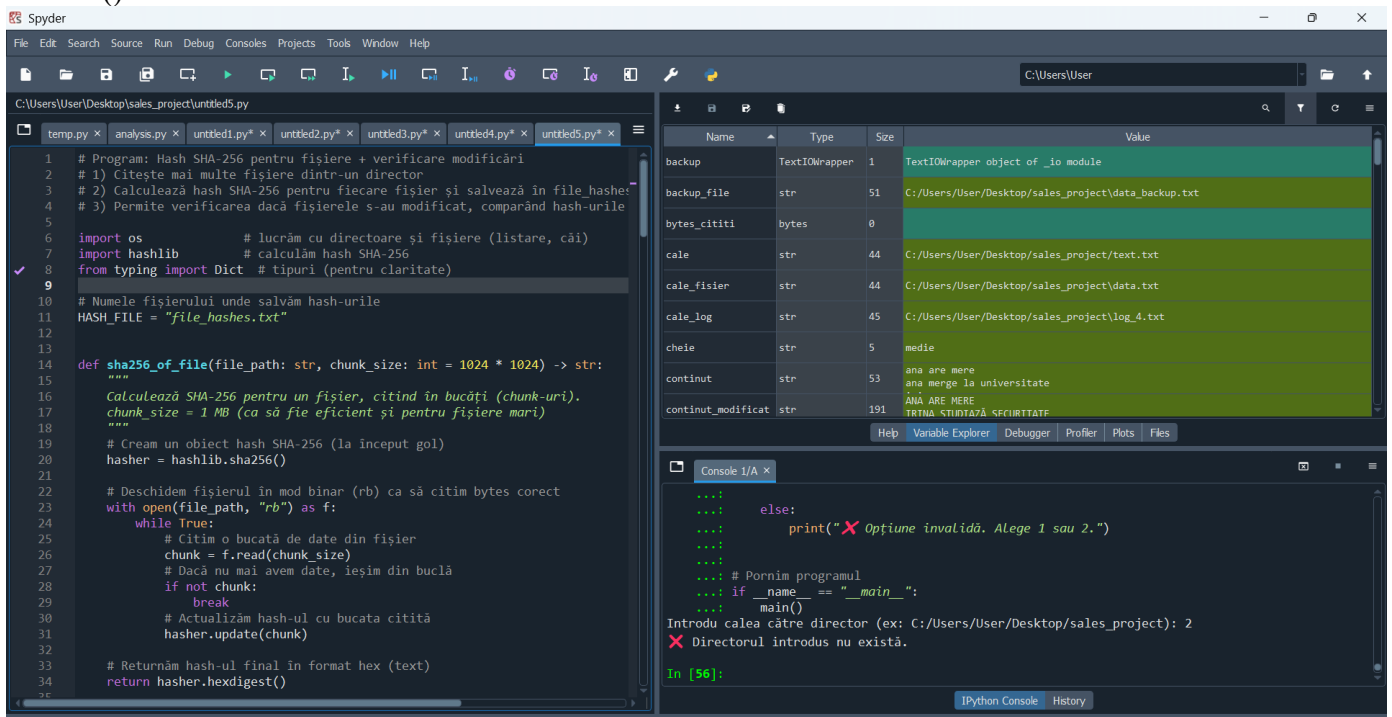
# Calculăm hash-uri noi
new_hashes = compute_hashes(directory)

# Comparăm și afișăm rezultatul
compare_hashes(old_hashes, new_hashes)

else:
    print("✗ Opțiune invalidă. Alege 1 sau 2.")

# Pornim programul
if __name__ == "__main__":
    main()

```



Sarcina 5: Generarea unui raport din fișiere de jurnal

Scrie un program care:

1. Citește fișiere de jurnal (log1.txt, log2.txt, etc.) care conțin erori și mesaje de avertizare.
2. Extrage doar erorile din fiecare fișier de jurnal și le salvează într-un raport în format CSV, cu coloane pentru data și mesajul de eroare.
3. Raportul trebuie să conțină și statistici precum numărul total de erori și avertizări.

```

# Program: Extrage ERORI din log-uri și face raport CSV + statistici
# 1) Citește fișiere de jurnal: log1.txt, log2.txt, ... (dintr-un folder ales)

```

2) Extrage doar liniile cu ERORI și le salvează în errors_report.csv (data, mesaj)

3) Calculează statistici: total erori și total avertizări (warnings)

```
import os          # pentru lucru cu foldere/fișiere (listare, căi)
import csv         # pentru scriere raport CSV
import re         # pentru a extrage data și mesajul cu expresii regulate
from glob import glob # pentru a găsi fișiere după model (log*.txt)
```

=== Setări raport ===

OUTPUT_CSV = "errors_report.csv" # numele raportului CSV

OUTPUT_STATS = "log_stats.txt" # fișier text cu statistici (opțional)

```
def parse_log_line(line: str):
```

```
    """
```

Primește o linie din log și încearcă să extragă:

- data/ora (timestamp)

- nivelul (ERROR / WARNING / etc.)

- mesajul

Returnează tuple: (timestamp, level, message) sau (None, None, None) dacă nu se pot extrage.

```
    """
```

Ștergem spații la capete + \n

line = line.strip()

Dacă linia este goală, nu avem ce procesa

if not line:

return None, None, None

Model general pentru log-uri de tip:

2026-03-02 18:10:02 ERROR Mesaj...

2026-03-02 18:10:02 WARNING Mesaj...

#

Grupuri:

1) data + ora

2) level (ERROR/WARNING)

3) mesajul

pattern = r"^(\d{4}-\d{2}-\d{2}[T]\d{2}:\d{2}:\d{2})\s+(ERROR|WARNING)\s+(.*)\$"

m = re.match(pattern, line, flags=re.IGNORECASE)

if m:

timestamp = m.group(1)

level = m.group(2).upper()

message = m.group(3)

return timestamp, level, message

Dacă nu se potrivește modelul de mai sus, încercăm un alt format comun:

[2026-03-02 18:10:02] ERROR: Mesaj...

pattern2 = r"^[(\d{4}-\d{2}-\d{2}[T]\d{2}:\d{2}:\d{2})]\s*(ERROR|WARNING)\s*[:\-\]\s*(.*)\$"

m2 = re.match(pattern2, line, flags=re.IGNORECASE)

```
if m2:
    timestamp = m2.group(1)
    level = m2.group(2).upper()
    message = m2.group(3)
    return timestamp, level, message

# Dacă nici așa nu merge, returnăm None (linie necunoscută / alt format)
return None, None, None
```

```
def extract_errors_to_csv(folder: str):
    """
```

```
    Citește log*.txt din folder, extrage ERORILE și:
```

```
    - scrie errors_report.csv
```

```
    - calculează statistici (total erori / avertizări)
```

```
    """
```

```
# 1) Găsim fișierele log: log1.txt, log2.txt, log_something.txt etc.
```

```
# model: log*.txt
```

```
pattern = os.path.join(folder, "log*.txt")
```

```
log_files = sorted(glob(pattern))
```

```
# Dacă nu găsim fișiere, anunțăm și ieșim
```

```
if not log_files:
```

```
    print("✗ Nu am găsit fișiere de tip log*.txt în folderul:", folder)
```

```
    return
```

```
# 2) Pregătim lista unde salvăm erorile găsite (rânduri pentru CSV)
```

```
error_rows = [] # fiecare element va fi: [timestamp, message, file_name]
```

```
# 3) Contoare statistici
```

```
total_errors = 0
```

```
total_warnings = 0
```

```
# 4) Parcurgem fiecare fișier log
```

```
for log_path in log_files:
```

```
    file_name = os.path.basename(log_path)
```

```
# Deschidem fișierul în mod citire (text) cu utf-8
```

```
with open(log_path, "r", encoding="utf-8", errors="replace") as f:
```

```
    for line in f:
```

```
        timestamp, level, message = parse_log_line(line)
```

```
# Dacă linia nu are format recunoscut, o ignorăm
```

```
if timestamp is None:
```

```
    continue
```

```
# Dacă este WARNING, numărăm warning
```

```
if level == "WARNING":
```

```
    total_warnings += 1
```

```
# Dacă este ERROR, numărăm și salvăm în raport
```

```
    if level == "ERROR":
        total_errors += 1
        error_rows.append([timestamp, message, file_name])
```

5) Scriem raportul CSV cu ERORI

```
output_csv_path = os.path.join(folder, OUTPUT_CSV)
```

newline="" e recomandat la CSV pe Windows

```
with open(output_csv_path, "w", newline="", encoding="utf-8") as out_csv:
    writer = csv.writer(out_csv)
```

Scriem antetul (coloanele)

```
writer.writerow(["data", "mesaj_eroare", "fisier_log"])
```

Scriem fiecare eroare găsită

```
writer.writerows(error_rows)
```

6) Salvăm statistici într-un fișier text (opțional, dar util)

```
stats_path = os.path.join(folder, OUTPUT_STATS)
```

```
with open(stats_path, "w", encoding="utf-8") as out_stats:
```

```
    out_stats.write("STATISTICI LOGURI\n")
```

```
    out_stats.write("=====\n")
```

```
    out_stats.write(f'Total fișiere log citite: {len(log_files)}\n')
```

```
    out_stats.write(f'Total ERORI: {total_errors}\n')
```

```
    out_stats.write(f'Total AVERTIZĂRI (WARNING): {total_warnings}\n')
```

```
    out_stats.write(f'Total linii de erori salvate în CSV: {len(error_rows)}\n')
```

7) Afișăm pe ecran un rezumat

```
print("✓ Gata!")
```

```
print("Raport CSV cu erori:", output_csv_path)
```

```
print("Statistici:", stats_path)
```

```
print(f'Total ERORI: {total_errors}')
```

```
print(f'Total WARNING: {total_warnings}')
```

```
def main():
```

Cerem folderul cu log-uri

```
folder = input("Introdu calea folderului cu log-uri (ex: C:/Users/User/Desktop/sales_project): ").strip()
```

Dacă nu introduci nimic, folosim folderul curent

if not folder:

```
    folder = os.getcwd()
```

Verificăm dacă folderul există

if not os.path.isdir(folder):

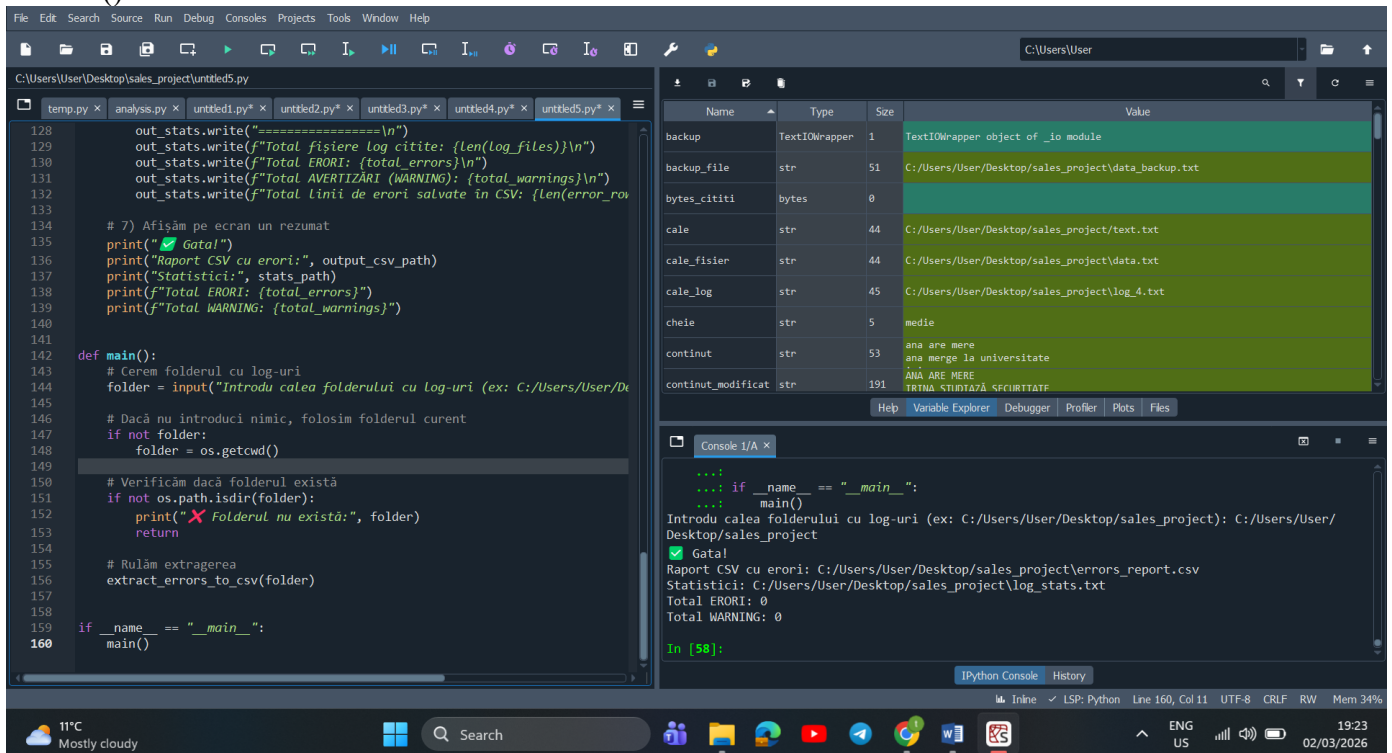
```
    print("✗ Folderul nu există:", folder)
```

```
    return
```

Rulăm extragerea

```
extract_errors_to_csv(folder)
```

```
if __name__ == "__main__":
    main()
```



Sarcina 6: Crearea unui sistem de backup incremental

Scrie un program care:

1. Creează un backup incremental al unui director (`source_directory`) într-un director de backup (`backup_directory`).
2. La fiecare rulare, programul trebuie să copieze doar fișierele care s-au modificat de la ultima rulare (folosind timestamp-uri sau hash-uri pentru verificare).
3. Fiecare backup trebuie să fie salvat într-un folder cu o dată specifică (de exemplu, `backup_2023_03_21`).

```
import os
import shutil
import json
from datetime import datetime
```

```
# =====
# SETĂRI (schimbă aici)
# =====
```

```
SOURCE_DIRECTORY = r"C:\Users\User\Desktop\sales_project" # folderul de copiat
BACKUP_DIRECTORY = r"C:\Users\User\Desktop\sales_project_backup" # unde salvăm backup-urile
```

```
# Numele fișierului unde memorăm ce am copiat ultima dată (manifest)
MANIFEST_PATH = os.path.join(BACKUP_DIRECTORY, "manifest.json")
```

```
def load_manifest(manifest_path: str) -> dict:
    """
```

Citește manifestul (istoricul) cu timpii ultimului backup.

Dacă nu există, întoarce dict gol.

"""

Dacă manifestul nu există, e prima rulare -> nu avem istoric

if not os.path.exists(manifest_path):

return {}

Deschidem manifestul în citire și încărcăm JSON-ul într-un dicționar Python

with open(manifest_path, "r", encoding="utf-8") as f:

return json.load(f)

def save_manifest(manifest_path: str, manifest: dict) -> None:

"""

Salvează manifestul actualizat pe disc (suprascrie fișierul).

"""

Asigurăm că folderul de backup există

os.makedirs(os.path.dirname(manifest_path), exist_ok=True)

Scriem dict-ul ca JSON (indent pentru a fi ușor de citit)

with open(manifest_path, "w", encoding="utf-8") as f:

json.dump(manifest, f, indent=4, ensure_ascii=False)

def get_all_files(root_dir: str) -> list:

"""

Returnează o listă cu TOATE fișierele din root_dir (și subfoldere),
ca căi relative (ex: 'subfolder\\fișier.txt').

"""

files = []

os.walk parcurge toate folderele și fișierele recursiv

for current_folder, _, filenames in os.walk(root_dir):

for name in filenames:

full_path = os.path.join(current_folder, name)

Calea relativă față de root_dir (o folosim în manifest)

rel_path = os.path.relpath(full_path, root_dir)

files.append(rel_path)

return files

def incremental_backup(source_dir: str, backup_dir: str) -> None:

"""

Creează un backup incremental:

- creează folder backup_<data>

- copiază doar fișiere noi/modificate

- actualizează manifest.json

"""

Verificăm că sursa există

if not os.path.isdir(source_dir):


```
print(f'Eroare: folderul sursă nu există: {source_dir}')
return

# Creăm folderul de backup principal dacă nu există
os.makedirs(backup_dir, exist_ok=True)

# Încărcăm manifestul (istoricul) din rulare anterioară
manifest = load_manifest(MANIFEST_PATH)

# Creăm numele folderului backup cu dată+oră (ca să fie unic)
timestamp = datetime.now().strftime("%Y_%m_%d_%H%M%S")
backup_run_folder = os.path.join(backup_dir, f'backup_{timestamp}')

# Creăm folderul pentru această rulare
os.makedirs(backup_run_folder, exist_ok=True)

# Luăm toate fișierele din sursă (relativ)
rel_files = get_all_files(source_dir)

# Contori pentru raport
copied = 0
skipped = 0

# Parcurgem fiecare fișier din sursă
for rel_path in rel_files:
    source_file = os.path.join(source_dir, rel_path)

    # Luăm timpul de modificare (mtime) al fișierului din sursă
    mtime = os.path.getmtime(source_file)

    # Dacă fișierul NU exista în manifest sau s-a modificat -> îl copiem
    # manifest ține: { "rel_path": last_mtime }
    if (rel_path not in manifest) or (mtime > manifest[rel_path]):
        dest_file = os.path.join(backup_run_folder, rel_path)

        # Creăm subfolderele necesare în destinație (dacă există)
        os.makedirs(os.path.dirname(dest_file), exist_ok=True)

        # Copiem fișierul (copy2 păstrează metadata: dată, etc.)
        shutil.copy2(source_file, dest_file)

        # Actualizăm manifestul cu mtime-ul nou
        manifest[rel_path] = mtime
        copied += 1
    else:
        # Nu s-a schimbat față de ultima rulare -> îl ignorăm
        skipped += 1

# Salvăm manifestul actualizat după ce am terminat
save_manifest(MANIFEST_PATH, manifest)

# Afișăm raport
```

```

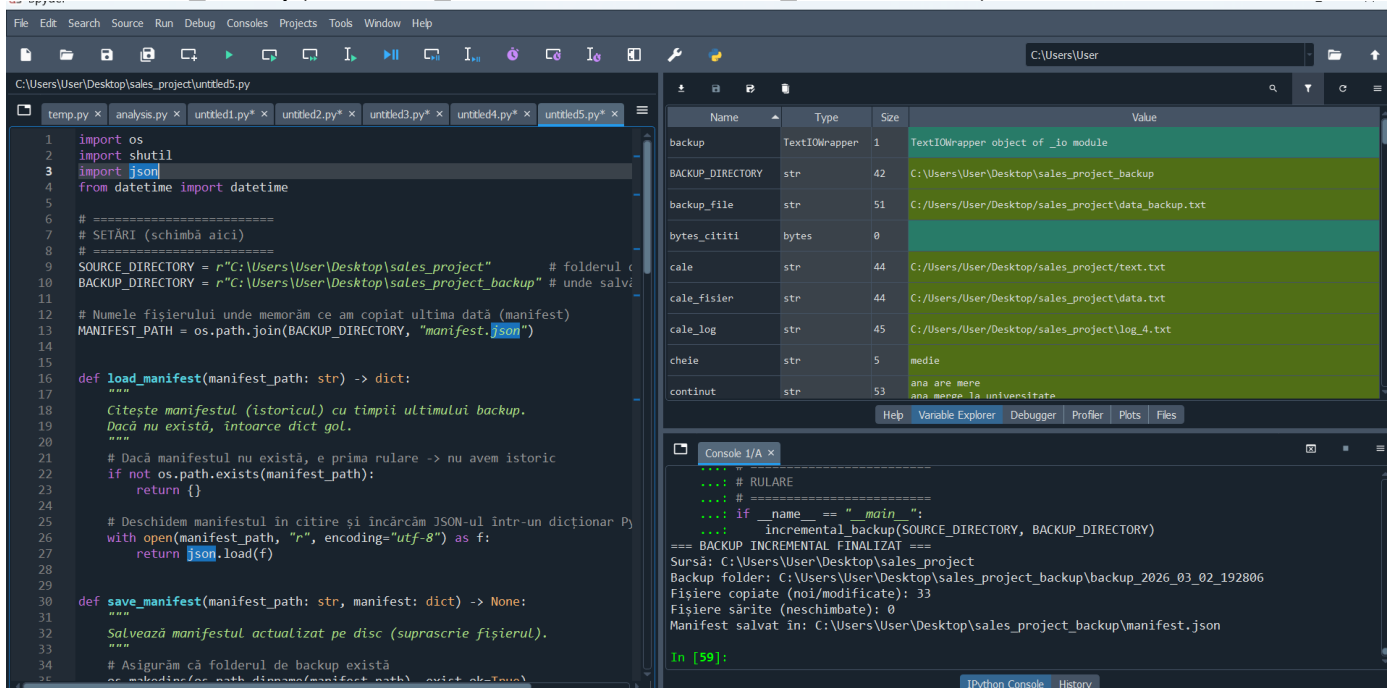
print("=== BACKUP INCREMENTAL FINALIZAT ===")
print("Sursă:", source_dir)
print("Backup folder:", backup_run_folder)
print("Fișiere copiate (noi/modificate):", copied)
print("Fișiere sărite (neschimbate):", skipped)
print("Manifest salvat în:", MANIFEST_PATH)

```

```

# =====
# RULARE
# =====
if __name__ == "__main__":
    incremental_backup(SOURCE_DIRECTORY, BACKUP_DIRECTORY)

```



Sarcina 7: Compararea conținutului a două fișiere mari

Scrie un program care:

1. Compara două fișiere mari (file1.txt și file2.txt).
2. Afișează pe ecran toate diferențele dintre cele două fișiere, fie linie cu linie, fie pe blocuri de text.
3. Programul trebuie să fie eficient și să nu încarce întregul fișier în memorie.

```

def compare_large_files(file1_path, file2_path):
    """

```

```

    Compară două fișiere mari linie cu linie.
    Nu încarcă tot fișierul în memorie.
    """

```

```

try:
    with open(file1_path, "r", encoding="utf-8") as f1, \
        open(file2_path, "r", encoding="utf-8") as f2:

```

```

        line_number = 1
        differences_found = False

```

```

while True:
    line1 = f1.readline()
    line2 = f2.readline()

    # Dacă am ajuns la final în ambele fișiere
    if not line1 and not line2:
        break

    # Dacă liniile sunt diferite
    if line1 != line2:
        differences_found = True
        print(f'\nDiferență la linia {line_number}:')
        print(f'File1: {line1.strip()}')
        print(f'File2: {line2.strip()}')

    line_number += 1

if not differences_found:
    print("Fișierele sunt identice.")

except FileNotFoundError as e:
    print("Eroare:", e)

```

=== RULARE ===

```

file1 = input("Introdu calea către file1.txt: ").strip()
file2 = input("Introdu calea către file2.txt: ").strip()

```

compare_large_files(file1, file2)

The screenshot displays the Spyder Python IDE interface. The left pane shows a Python script named `untitled5.py` with the following code:

```

8   with open(file1_path, "r", encoding="utf-8") as f1, \
9       open(file2_path, "r", encoding="utf-8") as f2:
10
11       line_number = 1
12       differences_found = False
13
14       while True:
15           line1 = f1.readline()
16           line2 = f2.readline()
17
18           # Dacă am ajuns la final în ambele fișiere
19           if not line1 and not line2:
20               break
21
22           # Dacă liniile sunt diferite
23           if line1 != line2:
24               differences_found = True
25               print(f'\nDiferență la linia {line_number}:')
26               print(f'File1: {line1.strip()}')
27               print(f'File2: {line2.strip()}')
28
29           line_number += 1
30
31       if not differences_found:
32           print("Fișierele sunt identice.")
33
34   except FileNotFoundError as e:
35       print("Eroare:", e)
36
37
38   # === RULARE ===
39   file1 = input("Introdu calea către file1.txt: ").strip()
40   file2 = input("Introdu calea către file2.txt: ").strip()
41
42   compare_large_files(file1, file2)

```

The right pane shows the Variable Explorer with the following variables and values:

Name	Type	Size	Value
backup	TextIOWrapper	1	TextIOWrapper object of _io module
BACKUP_DIRECTORY	str	42	C:\Users\User\Desktop\sales_project_backup
backup_file	str	51	C:\Users\User\Desktop\sales_project\data_backup.txt
bytes_cititi	bytes	0	
cale	str	44	C:\Users\User\Desktop\sales_project\text.txt
cale_fisier	str	44	C:\Users\User\Desktop\sales_project\data.txt
cale_log	str	45	C:\Users\User\Desktop\sales_project\log_4.txt
cheie	str	5	medie
continut	str	53	ana are mere ana merge la universitate

The bottom pane shows the IPython Console with the following output:

```

In [60]:
Introdu calea către file1.txt: C:\Users\User\Desktop\sales_project\file1.txt
Introdu calea către file2.txt: C:\Users\User\Desktop\sales_project\file2.txt

Diferență la linia 1:
File1: Ana
File2: Mihai

Diferență la linia 2:
File1: Irina
File2: Bogdan

```

Sarcina 8: Crearea unui fișier ZIP cu mai multe tipuri de fișiere

Scrie un program care:

1. Creează un fișier ZIP care conține atât fișiere text, imagini și fișiere binare dintr-un director dat.
2. Programul trebuie să păstreze structura de directoare și să poată adăuga fișiere multiple de diferite tipuri.
3. La final, programul trebuie să extragă fișierele din arhivă într-un director temporar.

```
import os
import zipfile
import tempfile
```

```
# =====
# PROGRAM: ZIP + PĂSTRARE STRUCTURĂ + EXTRAGERE
# =====
```

```
# 1) AICI pui folderul tău (directorul pe care vrei să-l arhivezi)
```

```
# Exemplu din pozele tale:
```

```
# SOURCE_DIR = r"C:\Users\User\Desktop\sales_project"
```

```
SOURCE_DIR = r"C:\Users\User\Desktop\sales_project" # <- schimbă dacă ai alt folder
```

```
# 2) Numele arhivei ZIP care va fi creată (în același folder ca scriptul sau cu cale completă)
```

```
ZIP_NAME = "arhiva_proiect.zip"
```

```
def create_zip_from_directory(source_dir: str, zip_name: str) -> str:
```

```
    """
```

```
    Creează un fișier ZIP cu toate fișierele din source_dir,
    păstrând structura de directoare (foldere/subfoldere).
```

```
    """
```

```
    # Verificăm dacă folderul există
```

```
    if not os.path.isdir(source_dir):
```

```
        raise FileNotFoundError(f"Folderul nu există: {source_dir}")
```

```
    # Deschidem/creăm arhiva ZIP în mod scriere (w) => suprascrie dacă există
```

```
    with zipfile.ZipFile(zip_name, "w", compression=zipfile.ZIP_DEFLATED) as z:
```

```
        # os.walk parcurge recursiv folderul și subfolderele
```

```
        for root, dirs, files in os.walk(source_dir):
```

```
            for filename in files:
```

```
                # Calea completă a fișierului pe disc
```

```
                full_path = os.path.join(root, filename)
```

```
                # Calea relativă față de source_dir => asta păstrează structura în ZIP
```

```
                relative_path = os.path.relpath(full_path, source_dir)
```

```
                # Adăugăm fișierul în ZIP (cu numele relativ)
```

```
                z.write(full_path, arcname=relative_path)
```

```
    return zip_name
```

```
def extract_zip_to_temp(zip_name: str) -> str:
```

```
    """
```

```
    Extrage arhiva ZIP într-un director temporar și returnează calea acelui director.
```

```
    """
```

```
    # Verificăm dacă arhiva există
```

```
if not os.path.isfile(zip_name):
    raise FileNotFoundError(f"Arhiva nu există: {zip_name}")

# Creăm un folder temporar (Windows/Linux) unde extragem fișierele
temp_dir = tempfile.mkdtemp(prefix="extract_")

# Extragem tot conținutul ZIP în acel folder
with zipfile.ZipFile(zip_name, "r") as z:
    z.extractall(temp_dir)

return temp_dir

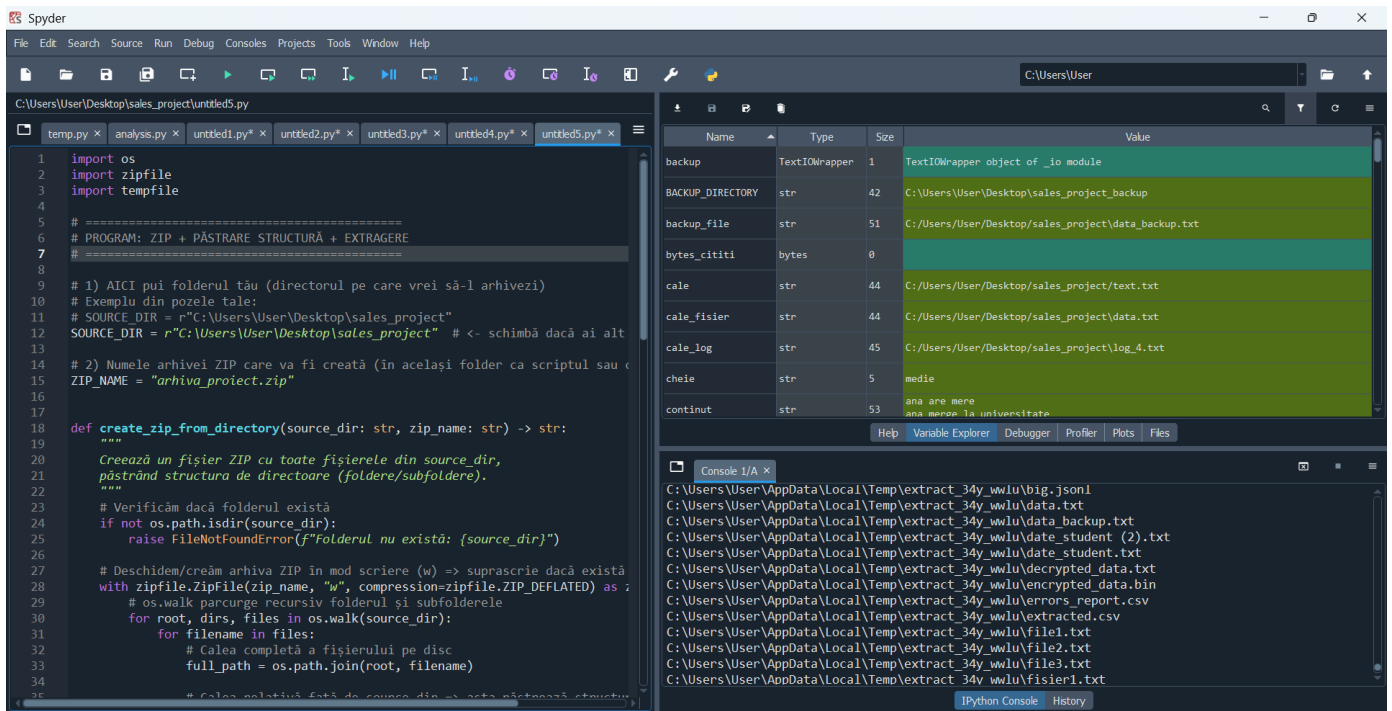
def main():
    try:
        # 1) Creăm ZIP-ul din folder
        zip_created = create_zip_from_directory(SOURCE_DIR, ZIP_NAME)
        print(f"✓ Arhiva ZIP a fost creată: {zip_created}")

        # 2) Extragem ZIP-ul într-un folder temporar
        extracted_folder = extract_zip_to_temp(zip_created)
        print(f"✓ Arhiva a fost extrasă în folderul temporar: {extracted_folder}")

        # (opțional) listăm câteva fișiere extrase
        print("\n📁 Fișiere extrase (primele 20):")
        count = 0
        for root, dirs, files in os.walk(extracted_folder):
            for f in files:
                print(os.path.join(root, f))
                count += 1
                if count >= 20:
                    return

    except FileNotFoundError as e:
        print("✗ Eroare:", e)
        print("🔍 Verifică dacă SOURCE_DIR este corect (calea către folder).")
    except PermissionError as e:
        print("✗ Eroare de permisiuni:", e)
        print("🔒 Închide fișierele deschise (Word/Excel/poze) și rulează iar.")
    except Exception as e:
        print("✗ Eroare neașteptată:", e)

if __name__ == "__main__":
    main()
```



Sarcina 9: Prelucrarea unui fișier XML mare

Scrie un program care:

1. Citește un fișier XML mare (`large_file.xml`) linie cu linie.
2. Extrage toate elementele dintr-un anumit tag XML (de exemplu, `<student>`) și le salvează într-un fișier CSV cu informațiile relevante (nume, vârstă, nota).
3. Programul trebuie să fie eficient și să utilizeze tehnici de procesare incrementală pentru a manipula fișierele mari.

```

import os
import csv
import xml.etree.ElementTree as ET

# ===== SETĂRI =====
FOLDER = "C:/Users/User/Desktop/sales_project"
XML_FILE = os.path.join(FOLDER, "large_file.xml")
CSV_FILE = os.path.join(FOLDER, "students_from_xml.csv")

```

```
TAG_TARGET = "student" # tag-ul pe care îl extragem
```

```
def extract_students_xml_to_csv(xml_path: str, csv_path: str, target_tag: str = "student") -> None:
    """
```

Citește XML incremental (streaming) și extrage fiecare element `<student>`.

Scrie rezultatul în CSV fără să țină tot XML-ul în memorie.

```
    """
```

```
# Deschidem CSV pentru scriere
```

```
with open(csv_path, "w", newline="", encoding="utf-8") as out_csv:
    writer = csv.writer(out_csv)
```

```
# Antet CSV
writer.writerow(["nume", "varsta", "nota"])

# iterpase citește XML pe bucăți (incremental)
# "end" => procesăm elementul când s-a terminat de citit
context = ET.iterparse(xml_path, events=("end",))

for event, elem in context:
    # Când găsim un element <student> complet citit
    if elem.tag == target_tag:
        # Căutăm subtag-urile: <name>, <age>, <grade>
        # (dacă XML-ul tău are alte nume, le schimbi aici)
        name_el = elem.find("name")
        age_el = elem.find("age")
        grade_el = elem.find("grade")

        nume = name_el.text.strip() if (name_el is not None and name_el.text) else ""
        varsta = age_el.text.strip() if (age_el is not None and age_el.text) else ""
        nota = grade_el.text.strip() if (grade_el is not None and grade_el.text) else ""

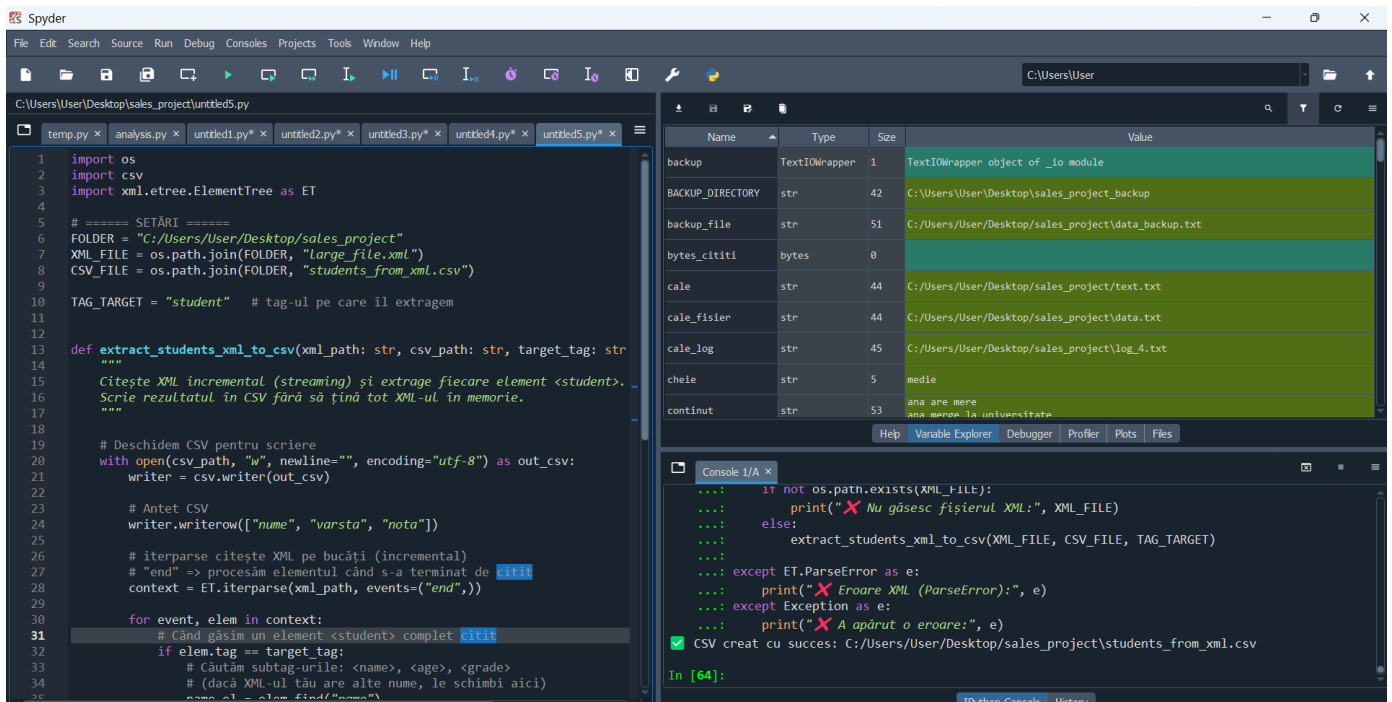
        # Scriem rândul în CSV
        writer.writerow([nume, varsta, nota])

        # IMPORTANT: eliberăm memoria pentru elementul procesat
        elem.clear()

print("✓ CSV creat cu succes:", csv_path)

# ===== RULARE =====
try:
    if not os.path.exists(XML_FILE):
        print("✗ Nu găsesc fișierul XML:", XML_FILE)
    else:
        extract_students_xml_to_csv(XML_FILE, CSV_FILE, TAG_TARGET)

except ET.ParseError as e:
    print("✗ Eroare XML (ParseError):", e)
except Exception as e:
    print("✗ A apărut o eroare:", e)
```



Sarcina 10: Implementarea unui sistem de gestionare a fișierelor media

Scrie un program care:

1. Scanează un director pentru fișiere media (imagini, videoclipuri, muzică) și salvează informații despre aceste fișiere într-o bază de date JSON sau CSV.
2. Informațiile trebuie să includă numele fișierului, tipul (formatul) și dimensiunea.
3. Programul trebuie să permită utilizatorului să caute fișiere media după tipul fișierului sau dimensiune.

```
import os
import csv
import json
```

```
# =====
# SETĂRI
# =====
```

```
FOLDER_DEFAULT = "C:/Users/User/Desktop/sales_project" # schimbă dacă vrei alt folder
CSV_OUT = "media_index.csv"
JSON_OUT = "media_index.json"
```

```
# Extensii media (poți adăuga ce vrei)
IMAGE_EXT = {".jpg", ".jpeg", ".png", ".gif", ".bmp", ".webp", ".tiff"}
VIDEO_EXT = {".mp4", ".mkv", ".avi", ".mov", ".wmv", ".flv", ".webm"}
AUDIO_EXT = {".mp3", ".wav", ".flac", ".aac", ".ogg", ".m4a"}
```

```
def media_type_from_ext(ext: str) -> str:
    """Determină tipul media după extensie."""
    ext = ext.lower()
    if ext in IMAGE_EXT:
        return "image"
    if ext in VIDEO_EXT:
        return "video"
```



```
if ext in AUDIO_EXT:
    return "audio"
return "other"
```

```
def scan_media_files(folder: str) -> list[dict]:
    """
    Scanează recursiv folderul și returnează o listă de dicționare:
    {name, extension, type, size_bytes, path}
    """
    results = []

    for root, _, files in os.walk(folder):
        for fname in files:
            full_path = os.path.join(root, fname)
            ext = os.path.splitext(fname)[1].lower()

            # luăm doar fișiere media (image/video/audio)
            mtype = media_type_from_ext(ext)
            if mtype == "other":
                continue

            size_bytes = os.path.getsize(full_path)

            results.append({
                "name": fname,
                "extension": ext,
                "type": mtype,
                "size_bytes": size_bytes,
                "path": full_path
            })

    return results
```

```
def save_to_csv(items: list[dict], csv_path: str) -> None:
    """Salvează lista în CSV."""
    with open(csv_path, "w", newline="", encoding="utf-8") as f:
        writer = csv.DictWriter(f, fieldnames=["name", "extension", "type", "size_bytes", "path"])
        writer.writeheader()
        writer.writerows(items)
```

```
def save_to_json(items: list[dict], json_path: str) -> None:
    """Salvează lista în JSON."""
    with open(json_path, "w", encoding="utf-8") as f:
        json.dump(items, f, indent=4, ensure_ascii=False)
```

```
def bytes_from_user(text: str) -> int:
    """
    Convertește input de tip:
    1000      (bytes)
    200KB     (kilobytes)
    5MB       (megabytes)
    1.5GB     (gigabytes)
    """
```

```
"""
s = text.strip().upper().replace(" ", "")
if s.endswith("KB"):
    return int(float(s[:-2]) * 1024)
if s.endswith("MB"):
    return int(float(s[:-2]) * 1024 * 1024)
if s.endswith("GB"):
    return int(float(s[:-2]) * 1024 * 1024 * 1024)
return int(float(s)) # bytes

def search_items(items: list[dict]) -> None:
    """
    Permite căutare după:
    - tip: image/video/audio
    - extensie: .jpg .mp4 etc.
    - dimensiune minimă/maximă
    """
    print("\n--- CĂUTARE ---")
    print("1) Caută după tip (image/video/audio)")
    print("2) Caută după extensie (ex: .jpg, .mp4, .mp3)")
    print("3) Caută după dimensiune (min/max) ex: 5MB, 200KB")
    opt = input("Alege opțiunea (1/2/3): ").strip()

    found = []

    if opt == "1":
        t = input("Introdu tipul (image/video/audio): ").strip().lower()
        found = [x for x in items if x["type"] == t]

    elif opt == "2":
        ext = input("Introdu extensia (ex: .jpg): ").strip().lower()
        if not ext.startswith("."):
            ext = "." + ext
        found = [x for x in items if x["extension"] == ext]

    elif opt == "3":
        min_s = input("Dimensiune MIN (ex: 0, 200KB, 5MB): ").strip()
        max_s = input("Dimensiune MAX (ex: 10MB, 1GB): ").strip()

        min_b = bytes_from_user(min_s) if min_s else 0
        max_b = bytes_from_user(max_s) if max_s else 10**18

        found = [x for x in items if min_b <= x["size_bytes"] <= max_b]

    else:
        print("Opțiune invalidă.")
        return

    print(f"\nRezultate găsite: {len(found)}")
    # Afișăm primele 30 rezultate ca să nu umple consola
    for x in found[:30]:
        size_kb = x["size_bytes"] / 1024
```

```
print(f'- {x['name']} | {x['type']} | {x['extension']} | {size_kb:.1f} KB | {x['path']}')

if len(found) > 30:
    print("... (mai multe rezultate, vezi CSV/JSON)")

def main():
    folder = input(f"Introdu calea folderului (Enter = {FOLDER_DEFAULT}): ").strip()
    if not folder:
        folder = FOLDER_DEFAULT

    if not os.path.isdir(folder):
        print("✗ Folderul nu există:", folder)
        return

    # 1) Scanare
    items = scan_media_files(folder)

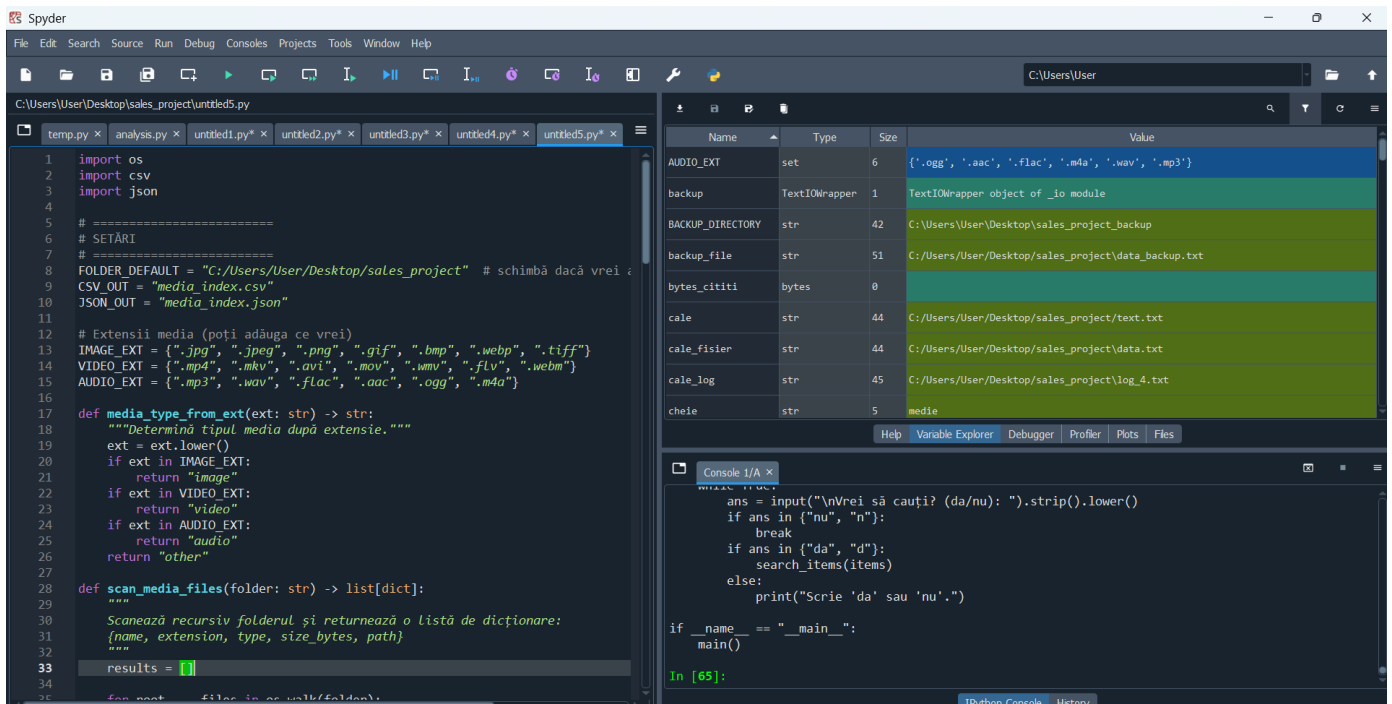
    # 2) Salvare în CSV și JSON în același folder scanat
    csv_path = os.path.join(folder, CSV_OUT)
    json_path = os.path.join(folder, JSON_OUT)

    save_to_csv(items, csv_path)
    save_to_json(items, json_path)

    print("\n✓ Scanare finalizată!")
    print("Fișiere media găsite:", len(items))
    print("CSV salvat:", csv_path)
    print("JSON salvat:", json_path)

    # 3) Căutare
    while True:
        ans = input("\nVrei să cauți? (da/nu): ").strip().lower()
        if ans in {"nu", "n"}:
            break
        if ans in {"da", "d"}:
            search_items(items)
        else:
            print("Scrie 'da' sau 'nu'.")

if __name__ == "__main__":
    main()
```



Sarcina 11: Comprimarea și decompimarea fișierelor mari folosind multithreading

Scrie un program care:

1. Comprimă fișiere mari dintr-un director folosind algoritmi de compresie eficienți (de exemplu, gzip sau bzip2).
2. Folosește multithreading pentru a comprima fișierele simultan, astfel încât să economisească timp.
3. Programul trebuie să permită și decompimarea fișierelor folosind multithreading.

```

import os
import gzip
import bz2
import shutil
from concurrent.futures import ThreadPoolExecutor, as_completed

```

```

# =====
# SETĂRI
# =====

```

```

FOLDER_DEFAULT = r"C:\Users\User\Desktop\sales_project" # schimbă dacă vrei alt folder
CHUNK_SIZE = 1024 * 1024 # 1 MB per chunk (eficient pentru fișiere mari)

```

```

def compress_one_file(input_path: str, algo: str) -> str:
    """

```

```

    Comprimă un fișier folosind gzip sau bzip2.
    Returnează calea fișierului comprimat.
    """

```

```

    if algo == "gzip":

```

```

        output_path = input_path + ".gz" # ex: data.txt.gz

```

```

        # Deschidem output gzip în scriere binară

```

```

        with open(input_path, "rb") as fin, gzip.open(output_path, "wb") as fout:

```

```

            shutil.copyfileobj(fin, fout, length=CHUNK_SIZE) # copiere pe bucăți

```

```

        return output_path

```

```
if algo == "bzip2":
    output_path = input_path + ".bz2" # ex: data.txt.bz2
    # Deschidem output bzip2 în scriere binară
    with open(input_path, "rb") as fin, bz2.open(output_path, "wb") as fout:
        shutil.copyfileobj(fin, fout, length=CHUNK_SIZE)
    return output_path

raise ValueError("Algoritm invalid. Alege: gzip sau bzip2.")
```

```
def decompress_one_file(input_path: str) -> str:
    """
    Decomprimă un fișier .gz sau .bz2.
    Returnează calea fișierului decomprimat.
    """
    if input_path.endswith(".gz"):
        output_path = input_path[:-3] # scoatem .gz
        with gzip.open(input_path, "rb") as fin, open(output_path, "wb") as fout:
            shutil.copyfileobj(fin, fout, length=CHUNK_SIZE)
        return output_path

    if input_path.endswith(".bz2"):
        output_path = input_path[:-4] # scoatem .bz2
        with bz2.open(input_path, "rb") as fin, open(output_path, "wb") as fout:
            shutil.copyfileobj(fin, fout, length=CHUNK_SIZE)
        return output_path

    raise ValueError("Fișierul nu este .gz sau .bz2 (nu știi ce să decomprim).")
```

```
def list_files_for_compress(folder: str, min_size_mb: int) -> list[str]:
    """
    Returnează fișierele din folder (recursiv) mai mari decât min_size_mb.
    Ignoră fișierele deja comprimate.
    """
    results = []
    min_bytes = min_size_mb * 1024 * 1024

    for root, _, files in os.walk(folder):
        for name in files:
            path = os.path.join(root, name)

            # Ignorăm fișierele deja comprimate
            if path.endswith(".gz") or path.endswith(".bz2"):
                continue

            # Luăm dimensiunea și filtrăm doar fișierele mari
            if os.path.getsize(path) >= min_bytes:
                results.append(path)

    return results
```

```
def list_files_for_decompress(folder: str) -> list[str]:
    """Returnează toate fişierele .gz şi .bz2 din folder (recursiv)."""
    results = []
    for root, _, files in os.walk(folder):
        for name in files:
            path = os.path.join(root, name)
            if path.endswith(".gz") or path.endswith(".bz2"):
                results.append(path)
    return results

def run_compress(folder: str, algo: str, min_size_mb: int, workers: int) -> None:
    files = list_files_for_compress(folder, min_size_mb)
    print(f'Fişiere de comprimat (>{min_size_mb} MB): {len(files)}')

    if not files:
        return

    with ThreadPoolExecutor(max_workers=workers) as executor:
        futures = [executor.submit(compress_one_file, f, algo) for f in files]

        for fut in as_completed(futures):
            try:
                out_path = fut.result()
                print("✓ Comprimat:", out_path)
            except Exception as e:
                print("✗ Eroare la comprimare:", e)

def run_decompress(folder: str, workers: int) -> None:
    files = list_files_for_decompress(folder)
    print(f'Fişiere de decompimat: {len(files)}')

    if not files:
        return

    with ThreadPoolExecutor(max_workers=workers) as executor:
        futures = [executor.submit(decompress_one_file, f) for f in files]

        for fut in as_completed(futures):
            try:
                out_path = fut.result()
                print("✓ Decomprimat:", out_path)
            except Exception as e:
                print("✗ Eroare la decomprimare:", e)

def main():
    folder = input(f'Folder (Enter = {FOLDER_DEFAULT}): ').strip()
```

if not folder:

folder = FOLDER_DEFAULT

if not os.path.isdir(folder):

print("✗ Folderul nu există:", folder)

return

action = input("Alege acțiune: compress / decompress: ").strip().lower()

workers_str = input("Câte thread-uri? (ex: 4): ").strip()

workers = int(workers_str) if workers_str else 4

if action == "compress":

algo = input("Algoritm (gzip/bzip2): ").strip().lower()

min_mb_str = input("Dimensiune minimă (MB) (ex: 5): ").strip()

min_mb = int(min_mb_str) if min_mb_str else 5

run_compress(folder, algo, min_mb, workers)

elif action == "decompress":

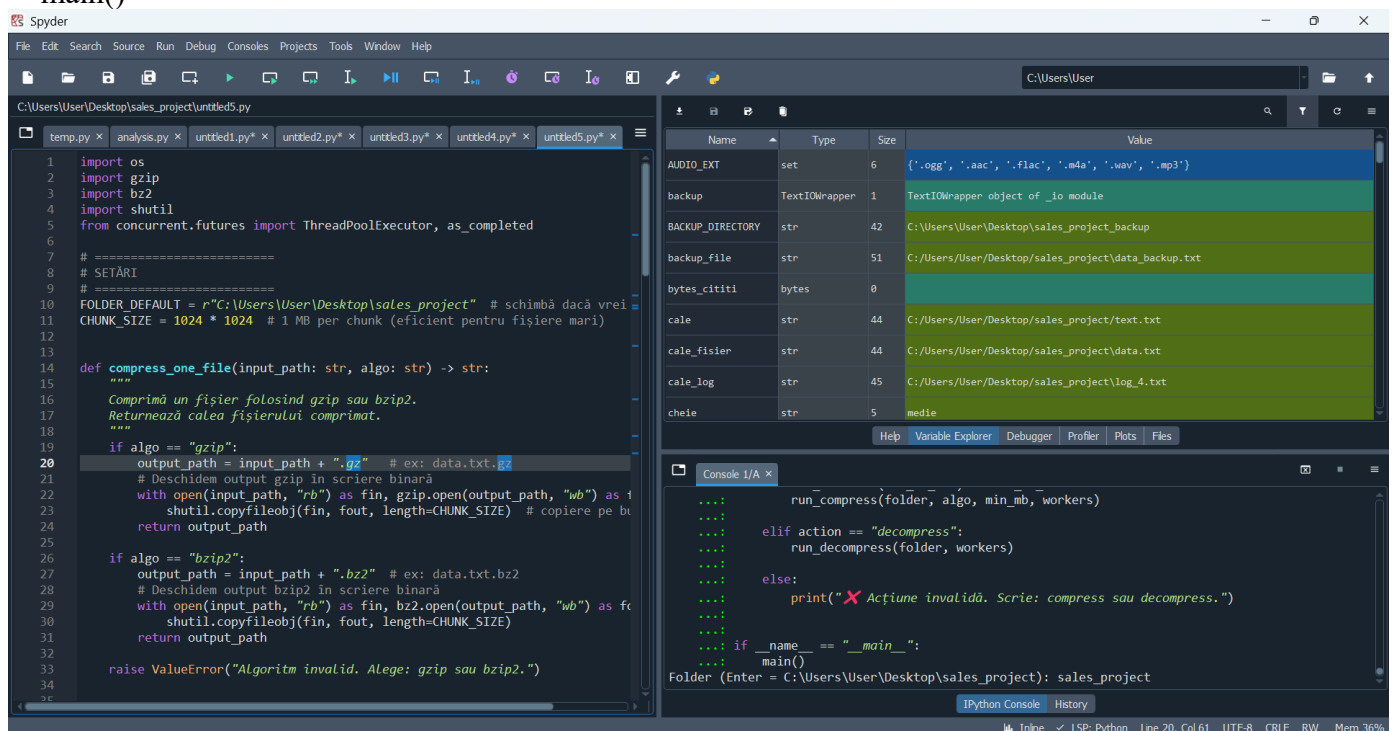
run_decompress(folder, workers)

else:

print("✗ Acțiune invalidă. Scrie: compress sau decompress.")

if __name__ == "__main__":

main()



Sarcina 12: Generarea unui raport de utilizare a discului

Scrie un program care:

1. Scanează un director și generează un raport de utilizare a discului, afișând dimensiunea fiecărui fișier și director.
2. Salvează raportul într-un fișier CSV sau JSON cu coloane pentru numele fișierului și dimensiunea.
3. La final, raportează totalul utilizat pe disc și procente pentru fiecare fișier sau director.

```
import os
import gzip
import bz2
import shutil
from concurrent.futures import ThreadPoolExecutor, as_completed

# =====
# SETĂRI
# =====
FOLDER_DEFAULT = r"C:\Users\User\Desktop\sales_project" # schimbă dacă vrei alt folder
CHUNK_SIZE = 1024 * 1024 # 1 MB per chunk (eficient pentru fișiere mari)
```

```
def compress_one_file(input_path: str, algo: str) -> str:
    """
    Comprimă un fișier folosind gzip sau bz2.
    Returnează calea fișierului comprimat.
    """
    if algo == "gzip":
        output_path = input_path + ".gz" # ex: data.txt.gz
        # Deschidem output gzip în scriere binară
        with open(input_path, "rb") as fin, gzip.open(output_path, "wb") as fout:
            shutil.copyfileobj(fin, fout, length=CHUNK_SIZE) # copiere pe bucăți
        return output_path

    if algo == "bz2":
        output_path = input_path + ".bz2" # ex: data.txt.bz2
        # Deschidem output bz2 în scriere binară
        with open(input_path, "rb") as fin, bz2.open(output_path, "wb") as fout:
            shutil.copyfileobj(fin, fout, length=CHUNK_SIZE)
        return output_path

    raise ValueError("Algoritm invalid. Alege: gzip sau bz2.")
```

```
def decompress_one_file(input_path: str) -> str:
    """
    Decomprimă un fișier .gz sau .bz2.
    Returnează calea fișierului decomprimat.
    """
    if input_path.endswith(".gz"):
        output_path = input_path[:-3] # scoatem .gz
        with gzip.open(input_path, "rb") as fin, open(output_path, "wb") as fout:
            shutil.copyfileobj(fin, fout, length=CHUNK_SIZE)
        return output_path

    if input_path.endswith(".bz2"):
```



```
    output_path = input_path[:-4] # scoatem .bz2
    with bz2.open(input_path, "rb") as fin, open(output_path, "wb") as fout:
        shutil.copyfileobj(fin, fout, length=CHUNK_SIZE)
    return output_path

    raise ValueError("Fișierul nu este .gz sau .bz2 (nu știu ce să decompres).")

def list_files_for_compress(folder: str, min_size_mb: int) -> list[str]:
    """
    Returnează fișierele din folder (recursiv) mai mari decât min_size_mb.
    Ignoră fișierele deja comprimate.
    """
    results = []
    min_bytes = min_size_mb * 1024 * 1024

    for root, _, files in os.walk(folder):
        for name in files:
            path = os.path.join(root, name)

            # Ignorăm fișierele deja comprimate
            if path.endswith(".gz") or path.endswith(".bz2"):
                continue

            # Luăm dimensiunea și filtrăm doar fișierele mari
            if os.path.getsize(path) >= min_bytes:
                results.append(path)

    return results

def list_files_for_decompress(folder: str) -> list[str]:
    """Returnează toate fișierele .gz și .bz2 din folder (recursiv)."""
    results = []
    for root, _, files in os.walk(folder):
        for name in files:
            path = os.path.join(root, name)
            if path.endswith(".gz") or path.endswith(".bz2"):
                results.append(path)
    return results

def run_compress(folder: str, algo: str, min_size_mb: int, workers: int) -> None:
    files = list_files_for_compress(folder, min_size_mb)
    print(f'Fișiere de comprimat (>{min_size_mb} MB): {len(files)}')

    if not files:
        return

    with ThreadPoolExecutor(max_workers=workers) as executor:
        futures = [executor.submit(compress_one_file, f, algo) for f in files]
```

```
    for fut in as_completed(futures):
        try:
            out_path = fut.result()
            print("✓ Comprimat:", out_path)
        except Exception as e:
            print("✗ Eroare la comprimare:", e)

def run_decompress(folder: str, workers: int) -> None:
    files = list_files_for_decompress(folder)
    print(f"Fișiere de decompimat: {len(files)}")

    if not files:
        return

    with ThreadPoolExecutor(max_workers=workers) as executor:
        futures = [executor.submit(decompress_one_file, f) for f in files]

    for fut in as_completed(futures):
        try:
            out_path = fut.result()
            print("✓ Decomprimat:", out_path)
        except Exception as e:
            print("✗ Eroare la decomprimare:", e)

def main():
    folder = input(f"Folder (Enter = {FOLDER_DEFAULT}): ").strip()
    if not folder:
        folder = FOLDER_DEFAULT

    if not os.path.isdir(folder):
        print("✗ Folderul nu există:", folder)
        return

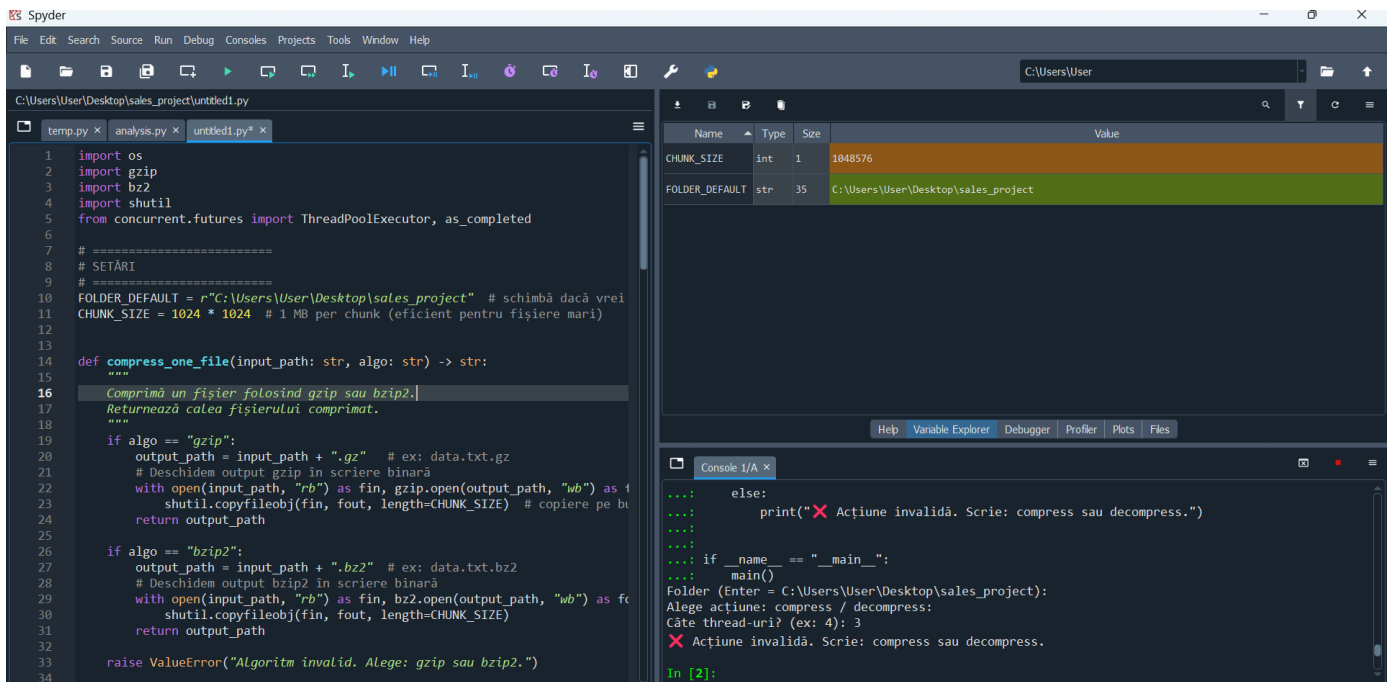
    action = input("Alege acțiune: compress / decompress: ").strip().lower()
    workers_str = input("Câte thread-uri? (ex: 4): ").strip()
    workers = int(workers_str) if workers_str else 4

    if action == "compress":
        algo = input("Algoritm (gzip/bzip2): ").strip().lower()
        min_mb_str = input("Dimensiune minimă (MB) (ex: 5): ").strip()
        min_mb = int(min_mb_str) if min_mb_str else 5
        run_compress(folder, algo, min_mb, workers)

    elif action == "decompress":
        run_decompress(folder, workers)

    else:
        print("✗ Acțiune invalidă. Scrie: compress sau decompress.")
```

```
if __name__ == "__main__":
    main()
```



Sarcina 13: Crearea unui fișier de indexare a datelor

Scrie un program care:

1. Citește un fișier text mare `large_data.txt` și creează un fișier de indexare (`index.txt`).
2. Indexează fiecare cuvânt din fișierul de date, creând o listă de locații (numere de linie și poziții) unde cuvintele apar.
3. Programul trebuie să fie eficient și să permită căutarea rapidă a unui cuvânt din fișierul indexat.

```
import os
import re

# =====
# SETĂRI
# =====
FOLDER = "C:/Users/User/Desktop/sales_project"
DATA_FILE = os.path.join(FOLDER, "large_data.txt")
INDEX_FILE = os.path.join(FOLDER, "index.txt")

# Regex pentru a extrage cuvinte (litere/cifre + diacritice)
WORD_RE = re.compile(r"[A-Za-z0-9ĂÂÎȘȚăâîșț]+", re.UNICODE)
```

```
def build_index(data_path: str, index_path: str) -> None:
    """
```

- 1) Citește `large_data.txt` linie cu linie
- 2) Indexează fiecare cuvânt -> listă de locații (linie, poziție)
- 3) Scrie indexul în `index.txt`

Format index.txt (simplu):

cuvant: linie-pozitie, linie-pozitie, ...

ex: ana: 1-1, 2-3

"""

Dicționar: cuvânt -> listă de apariții ("linie-pozitie")

index = {}

with open(data_path, "r", encoding="utf-8", errors="replace") as f:

for line_no, line in enumerate(f, start=1):

Extragem cuvintele din linie

words = WORD_RE.findall(line.lower())

poziția = ordinea cuvântului în linia respectivă (1,2,3...)

for pos, w in enumerate(words, start=1):

loc = f"{line_no}-{pos}"

adăugăm locația în listă

if w in index:

index[w].append(loc)

else:

index[w] = [loc]

Scriem indexul în fișier (ordonat alfabetic)

with open(index_path, "w", encoding="utf-8") as out:

for w in sorted(index.keys()):

out.write(w + ": " + ", ".join(index[w]) + "\n")

print("✓ Index creat:", index_path)

print("Cuvinte indexate:", len(index))

def search_word_in_index(index_path: str, word: str) -> None:

"""

Caută rapid un cuvânt în index.txt.

Pentru simplitate, căutăm linia care începe cu 'cuvant:'.

"""

word = word.strip().lower()

if not word:

print("✗ Cuvânt gol.")

return

Citim indexul linie cu linie și găsim cuvântul

with open(index_path, "r", encoding="utf-8", errors="replace") as f:

prefix = word + ":",

for line in f:

if line.startswith(prefix):

print("✓ Găsit:", line.strip())

return

print("Nu am găsit cuvântul în index.")

```

def main():
    print("1) Construiește index (index.txt)")
    print("2) Caută cuvânt în index")
    opt = input("Alege (1/2): ").strip()

    if opt == "1":
        if not os.path.exists(DATA_FILE):
            print("✗ Nu găsesc fișierul:", DATA_FILE)
            return
        build_index(DATA_FILE, INDEX_FILE)

    elif opt == "2":
        if not os.path.exists(INDEX_FILE):
            print("✗ Nu există index.txt. Rulează opțiunea 1 mai întâi.")
            return
        w = input("Introdu cuvântul de căutat: ")
        search_word_in_index(INDEX_FILE, w)

    else:
        print("✗ Opțiune invalidă.")

if __name__ == "__main__":
    main()

```

The screenshot shows the Spyder Python IDE interface. The left pane displays the code editor with the following Python code:

```

1 import os
2 import re
3
4 # =====
5 # SETĂRI
6 # =====
7 FOLDER = "C:/Users/User/Desktop/sales_project"
8 DATA_FILE = os.path.join(FOLDER, "large_data.txt")
9 INDEX_FILE = os.path.join(FOLDER, "index.txt")
10
11 # Regex pentru a extrage cuvinte (litere/cifre + diacritice)
12 WORD_RE = re.compile(r"[A-Za-z0-9ĂÂÎȘȚăăîșț]+", re.UNICODE)
13
14
15 def build_index(data_path: str, index_path: str) -> None:
16     """
17     1) Citește large_data.txt linie cu linie
18     2) Indexează fiecare cuvânt -> listă de locații (linie, poziție)
19     3) Scrie indexul în index.txt
20
21     Format index.txt (simplu):
22     cuvânt: linie-pozitie, linie-pozitie, ...
23     ex: ana: 1-1, 2-3
24     """
25
26     # Dicționar: cuvânt -> listă de apariții ("linie-pozitie")
27     index = {}
28
29     with open(data_path, "r", encoding="utf-8", errors="replace") as f:
30         for line_no, line in enumerate(f, start=1):
31             # Extragem cuvintele din linie
32             words = WORD_RE.findall(line.lower())
33
34             # poziția = ordinea cuvântului în linia respectivă (1,2,3...)
35             for pos, w in enumerate(words, start=1):

```

The right pane shows the Variable Explorer with the following variables:

Name	Type	Size	Value
CHUNK_SIZE	Int	1	1048576
DATA_FILE	str	50	C:/Users/User/Desktop/sales_project/large_data.txt
FOLDER	str	35	C:/Users/User/Desktop/sales_project
FOLDER_DEFAULT	str	35	C:/Users/User/Desktop/sales_project
INDEX_FILE	str	45	C:/Users/User/Desktop/sales_project/index.txt
WORD_RE	Pattern	1	Pattern object of re module

The bottom pane shows the IPython Console with the following output:

```

...: print("✗ Opțiune invalidă.")
...:
...: if __name__ == "__main__":
...:     main()
...:
1) Construiește index (index.txt)
2) Caută cuvânt în index
Alege (1/2): 1
✓ Index creat: C:/Users/User/Desktop/sales_project/index.txt
Cuvinte indexate: 12

In [4]:

```

Sarcina 14: Procesarea fișierelor CSV complexe

Scrie un program care:

1. Citește un fișier CSV complex (`data.csv`) care conține mai multe coloane și date de tip variabil (de exemplu, stringuri, date, numere).
2. Procesează și salvează în fișiere separate datele din fiecare coloană.
3. Programul trebuie să permită și filtrele de căutare pentru valorile din fiecare coloană.

```
import csv
import os
from datetime import datetime

# =====
# SETĂRI (schimbă dacă vrei)
# =====

# Folderul unde ai fișierele (după poze, la tine e sales_project)
FOLDER = r"C:\Users\User\Desktop\sales_project"

# Numele fișierului CSV de intrare
INPUT_CSV = os.path.join(FOLDER, "data.csv")

# Folder unde salvăm coloanele separat
OUTPUT_COLUMNS_DIR = os.path.join(FOLDER, "coloane_separate")

# Fișier unde salvăm rezultatul filtrat
FILTERED_OUT = os.path.join(FOLDER, "filtered_results.csv")

# =====
# FUNCȚII AJUTĂTOARE: detectare tipuri (număr / dată)
# =====

def try_parse_number(value: str):
    """
    Încearcă să transforme un text în număr (float).
    Acceptă și virgula ca separator zecimal (ex: "12,5").
    Returnează float sau None dacă nu se poate.
    """
    if value is None:
        return None
    value = value.strip()
    if value == "":
        return None
    value = value.replace(",", ".")
    try:
        return float(value)
    except ValueError:
        return None

def try_parse_date(value: str):
    """
    Încearcă să transforme un text în dată.
    Acceptă formate frecvente: YYYY-MM-DD, DD.MM.YYYY, DD/MM/YYYY.
```

Returnează datetime sau None dacă nu se poate.

```
"""
```

```
if value is None:
```

```
    return None
```

```
value = value.strip()
```

```
if value == "":
```

```
    return None
```

```
formats = ["%Y-%m-%d", "%d.%m.%Y", "%d/%m/%Y", "%Y/%m/%d"]
```

```
for fmt in formats:
```

```
    try:
```

```
        return datetime.strptime(value, fmt)
```

```
    except ValueError:
```

```
        pass
```

```
return None
```

```
# =====
```

```
# 1) CITIRE CSV + SALVARE COLOANE SEPARAT
```

```
# =====
```

```
def read_csv_as_dicts(csv_path: str):
```

```
    """
```

```
    Citește CSV și întoarce:
```

```
    - rows: listă de dicționare (cheie = nume coloană)
```

```
    - headers: lista de coloane
```

```
    """
```

```
    with open(csv_path, "r", encoding="utf-8", newline="") as f:
```

```
        reader = csv.DictReader(f)
```

```
        headers = reader.fieldnames # numele coloanelor
```

```
        rows = list(reader)        # toate rândurile ca dicționare
```

```
    return rows, headers
```

```
def save_each_column(rows, headers, out_dir: str):
```

```
    """
```

```
    Salvează fiecare coloană într-un fișier separat:
```

```
    ex: coloana "name" -> coloane_separate/name.txt
```

```
    """
```

```
    # Dacă folderul nu există, îl creăm
```

```
    os.makedirs(out_dir, exist_ok=True)
```

```
    for col in headers:
```

```
        out_path = os.path.join(out_dir, f"{col}.txt")
```

```
    # Deschidem fișierul coloanei în mod scriere (suprascrie dacă există)
```

```
    with open(out_path, "w", encoding="utf-8") as out_f:
```

```
        # Scriem fiecare valoare din acea coloană pe linie separată
```

```
        for r in rows:
```

```
            value = r.get(col, "")
```

```
            out_f.write(str(value) + "\n")
```

```
# =====
# 2) FILTRARE / CĂUTARE ÎN FUNCȚIE DE COLOANĂ
# =====
```

```
def filter_rows(rows, column: str, mode: str, value1: str, value2: str = ""):
```

```
    """
```

```
    Filtrează rândurile după:
```

```
    mode:
```

- "equals" : exact egal (text)
- "contains" : conține (text, case-insensitive)
- "gt" : mai mare (număr / dată)
- "lt" : mai mic (număr / dată)
- "between" : între value1 și value2 (număr / dată)

```
    """
```

```
    results = []
```

```
    for r in rows:
```

```
        cell = r.get(column, "")
```

```
        cell_str = str(cell).strip()
```

```
        # -----
```

```
        # Filtre text
```

```
        # -----
```

```
        if mode == "equals":
```

```
            if cell_str == value1:
```

```
                results.append(r)
```

```
        elif mode == "contains":
```

```
            if value1.lower() in cell_str.lower():
```

```
                results.append(r)
```

```
        # -----
```

```
        # Filtre numeric / date
```

```
        # -----
```

```
        elif mode in ("gt", "lt", "between"):
```

```
            # Încercăm să interpretăm atât celula cât și valoarea ca NUMĂR
```

```
            cell_num = try_parse_number(cell_str)
```

```
            v1_num = try_parse_number(value1)
```

```
            v2_num = try_parse_number(value2) if value2 else None
```

```
            # Dacă nu sunt numere, încercăm ca DATE
```

```
            cell_date = None
```

```
            v1_date = None
```

```
            v2_date = None
```

```
            if cell_num is None or v1_num is None:
```

```
                cell_date = try_parse_date(cell_str)
```

```
                v1_date = try_parse_date(value1)
```

```
                v2_date = try_parse_date(value2) if value2 else None
```

```
            # 1) Dacă avem numere, comparăm ca numere
```

```
            if cell_num is not None and v1_num is not None:
```



```
        if mode == "gt" and cell_num > v1_num:
            results.append(r)
        elif mode == "lt" and cell_num < v1_num:
            results.append(r)
        elif mode == "between" and v2_num is not None and v1_num <= cell_num <= v2_num:
            results.append(r)

# 2) Altfel, dacă avem date, comparăm ca date
elif cell_date is not None and v1_date is not None:
    if mode == "gt" and cell_date > v1_date:
        results.append(r)
    elif mode == "lt" and cell_date < v1_date:
        results.append(r)
    elif mode == "between" and v2_date is not None and v1_date <= cell_date <= v2_date:
        results.append(r)

# Dacă nu e nici număr, nici dată => nu îl includem
else:
    pass

else:
    raise ValueError("Mod de filtrare invalid!")

return results

def save_rows_to_csv(rows, headers, out_path: str):
    """
    Salvează o listă de rânduri (dict) într-un CSV.
    """
    with open(out_path, "w", encoding="utf-8", newline="") as f:
        writer = csv.DictWriter(f, fieldnames=headers)
        writer.writeheader()
        writer.writerows(rows)

# =====
# PROGRAM PRINCIPAL
# =====

def main():
    # 0) Verificăm că fișierul există
    if not os.path.exists(INPUT_CSV):
        print("Eroare: Nu găsesc fișierul:", INPUT_CSV)
        print("Pune data.csv în folderul:", FOLDER)
        return

    # 1) Citim CSV
    rows, headers = read_csv_as_dicts(INPUT_CSV)
    print("Am citit", len(rows), "rânduri.")
    print("Coloane:", headers)
```

```
# 2) Salvăm fiecare coloană separat
save_each_column(rows, headers, OUTPUT_COLUMNS_DIR)
print("Coloanele au fost salvate în:", OUTPUT_COLUMNS_DIR)
```

```
# 3) Filtrare (interactiv)
print("\n--- FILTRARE / CĂUTARE ---")
print("Alege coloana exact cum apare mai sus.")
column = input("Coloana: ").strip()
```

```
if column not in headers:
    print("Eroare: coloana nu există!")
    return
```

```
print("\nAlege modul:")
print("1) equals (text exact)")
print("2) contains (text conține)")
print("3) gt (>) număr/dată")
print("4) lt (<) număr/dată")
print("5) between (între) număr/dată")
choice = input("Alegere (1-5): ").strip()
```

```
mode_map = {
    "1": "equals",
    "2": "contains",
    "3": "gt",
    "4": "lt",
    "5": "between",
}
```

```
if choice not in mode_map:
    print("Eroare: alegere invalidă!")
    return
```

```
mode = mode_map[choice]
```

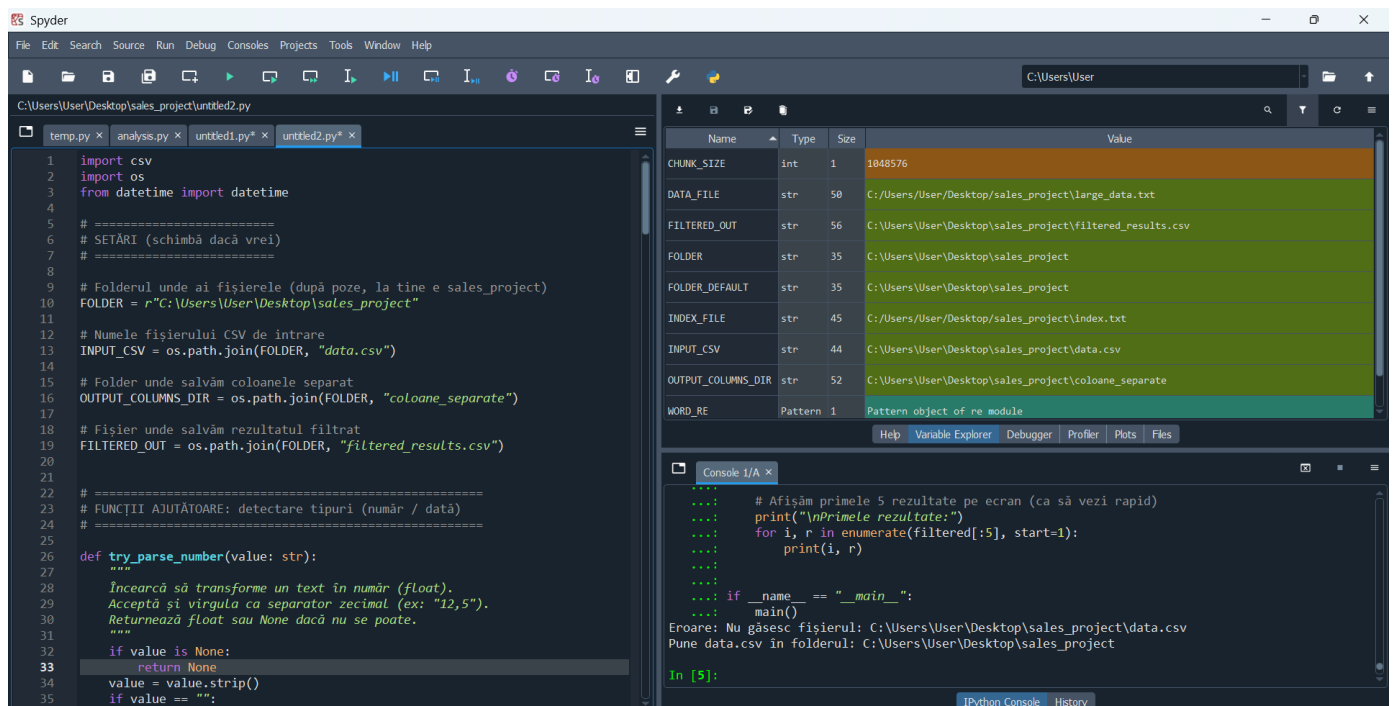
```
value1 = input("Valoare 1: ").strip()
value2 = ""
if mode == "between":
    value2 = input("Valoare 2: ").strip()
```

```
filtered = filter_rows(rows, column, mode, value1, value2)
print("\nRezultate găsite:", len(filtered))
```

```
# Salvăm rezultatul filtrat într-un CSV
save_rows_to_csv(filtered, headers, FILTERED_OUT)
print("Rezultatul filtrat a fost salvat în:", FILTERED_OUT)
```

```
# Afișăm primele 5 rezultate pe ecran (ca să vezi rapid)
print("\nPrimele rezultate:")
for i, r in enumerate(filtered[:5], start=1):
    print(i, r)
```

```
if __name__ == "__main__":
    main()
```



Sarcina 15: Crearea unui sistem de sincronizare a fișierelor între două directoare

Scrie un program care:

1. Compară două directoare și sincronizează fișierele între ele.
2. Dacă un fișier este prezent într-un director, dar nu și în celălalt, programul trebuie să-l copieze.
3. Dacă un fișier este modificat într-un director, programul trebuie să-l înlocuiască pe cel vechi din celălalt director.

```
import os
import shutil
import hashlib
```

```
# =====
# FUNCȚII AJUTĂTOARE
# =====
```

```
def file_sha256(path: str, chunk_size: int = 1024 * 1024) -> str:
    """
```

Calculează SHA-256 pentru un fișier, citit pe bucăți (eficient pentru fișiere mari).

```
    """
```

```
    h = hashlib.sha256()
```

```
    with open(path, "rb") as f:
```

```
        while True:
```

```
            chunk = f.read(chunk_size)
```

```
            if not chunk:
```

```
                break
```

```
    h.update(chunk)
    return h.hexdigest()
```

```
def list_files_recursive(root: str):
```

```
    """
    Returnează un dicționar:
    { "cale_relativa": "cale_completa" }
    pentru toate fișierele din root (recursiv).
    """
    files = {}
    for folder, _, filenames in os.walk(root):
        for name in filenames:
            full_path = os.path.join(folder, name)
            rel_path = os.path.relpath(full_path, root)
            files[rel_path] = full_path
    return files
```

```
def ensure_parent_folder(dest_file: str):
```

```
    """
    Creează folderul părinte pentru un fișier destinație, dacă nu există.
    """
    parent = os.path.dirname(dest_file)
    os.makedirs(parent, exist_ok=True)
```

```
def copy_file(src: str, dest: str):
```

```
    """
    Copiază fișierul src în dest și păstrează metadata (timp modificare).
    """
    ensure_parent_folder(dest)
    shutil.copy2(src, dest) # copy2 păstrează timestamp-ul
    print(f"COPIAT: {src} -> {dest}")
```

```
# =====
# SINCRONIZARE DIRECTOARE
# =====
```

```
def sync_directories(dir_a: str, dir_b: str, use_hash: bool = True):
```

```
    """
    Sincronizează 2 directoare (A <-> B).
    - use_hash=True: compară și hash (mai sigur, dar mai lent)
    - use_hash=False: compară doar mtime + size (mai rapid)
    """
    if not os.path.isdir(dir_a):
        print("Eroare: Directorul A nu există:", dir_a)
        return
    if not os.path.isdir(dir_b):
        print("Eroare: Directorul B nu există:", dir_b)
        return
```

```
files_a = list_files_recursive(dir_a)
files_b = list_files_recursive(dir_b)

all_rel_paths = sorted(set(files_a.keys()) | set(files_b.keys()))

updated = 0
copied_missing = 0
same = 0

for rel in all_rel_paths:
    a_path = files_a.get(rel)
    b_path = files_b.get(rel)

    # 1) Există doar în A -> copiem în B
    if a_path and not b_path:
        dest = os.path.join(dir_b, rel)
        copy_file(a_path, dest)
        copied_missing += 1
        continue

    # 2) Există doar în B -> copiem în A
    if b_path and not a_path:
        dest = os.path.join(dir_a, rel)
        copy_file(b_path, dest)
        copied_missing += 1
        continue

    # 3) Există în ambele -> comparăm
    a_stat = os.stat(a_path)
    b_stat = os.stat(b_path)

    # Comparare rapidă: size + timp modificare
    a_mtime = a_stat.st_mtime
    b_mtime = b_stat.st_mtime
    a_size = a_stat.st_size
    b_size = b_stat.st_size

    # Dacă e identic (rapid)
    if (a_size == b_size) and (int(a_mtime) == int(b_mtime)) and not use_hash:
        same += 1
        continue

    # Dacă folosim hash, verificăm sigur dacă sunt identice
    if use_hash and a_size == b_size:
        a_hash = file_sha256(a_path)
        b_hash = file_sha256(b_path)
        if a_hash == b_hash:
            same += 1
            continue

    # Dacă sunt diferite -> copiem varianta mai nouă peste cea veche
```

```

if a_mtime > b_mtime:
    copy_file(a_path, b_path) # suprascrie în B
    updated += 1
elif b_mtime > a_mtime:
    copy_file(b_path, a_path) # suprascrie în A
    updated += 1
else:
    # Dacă au același timp dar sunt diferite (posibil),
    # alegem să copiem din A în B (arbitrar)
    copy_file(a_path, b_path)
    updated += 1

print("\n=== REZUMAT ===")
print("Fișiere copiate (lipsă):", copied_missing)
print("Fișiere actualizate (înlocuite):", updated)
print("Fișiere identice:", same)

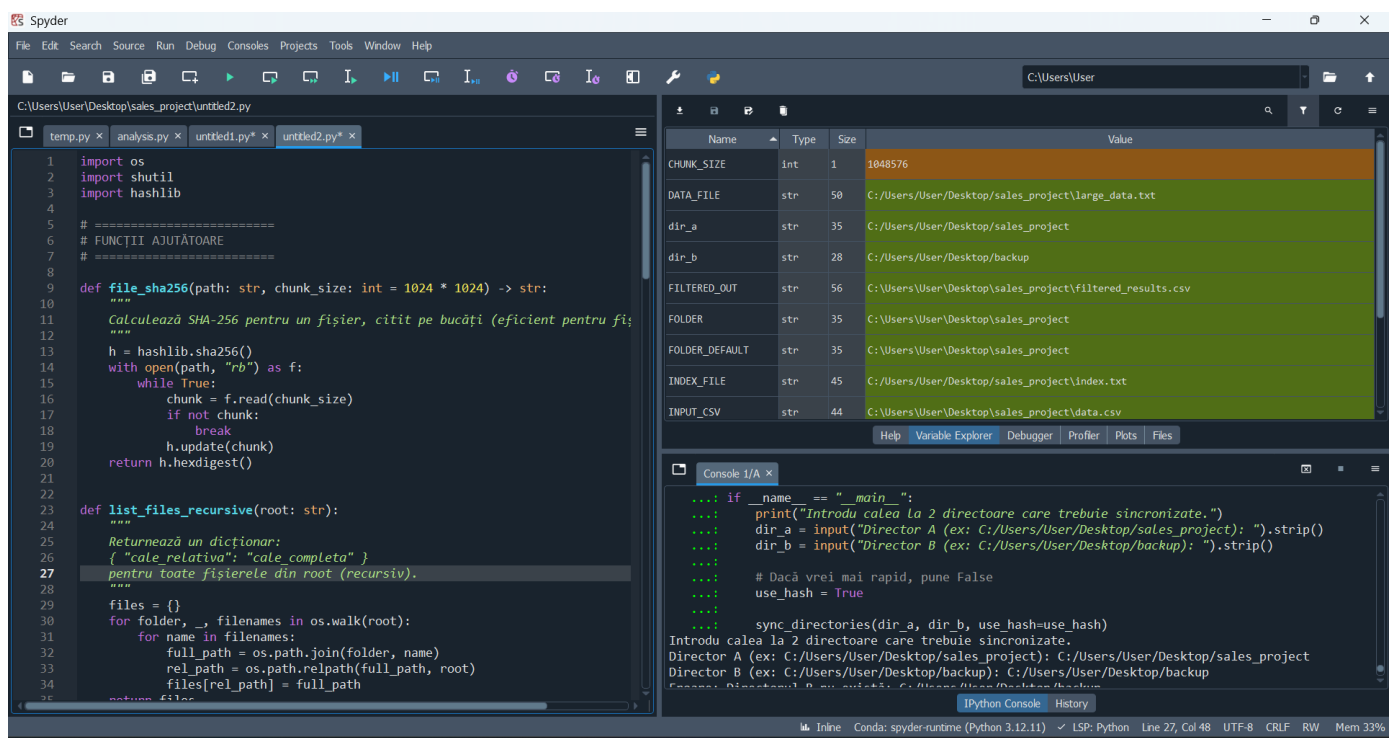
# =====
# MAIN
# =====

if __name__ == "__main__":
    print("Introdu calea la 2 directoare care trebuie sincronizate.")
    dir_a = input("Director A (ex: C:/Users/User/Desktop/sales_project): ").strip()
    dir_b = input("Director B (ex: C:/Users/User/Desktop/backup): ").strip()

    # Dacă vrei mai rapid, pune False
    use_hash = True

    sync_directories(dir_a, dir_b, use_hash=use_hash)

```



Instrucțiuni pentru realizare:

1. Utilizează tehnici de procesare eficientă a fișierelor mari (citirea incrementală, multithreading, manipularea fișierelor binare etc.).
2. Asigură-te că programele tale sunt eficiente din punct de vedere al utilizării memoriei și al vitezei.
3. Folosește librării Python standard precum `os`, `shutil`, `hashlib`, `zipfile`, `csv`, `json`, `cryptography`, `xml.etree.ElementTree`, `threading` și altele pentru a implementa soluțiile.
4. Încearcă să gestionezi corect erorile (fișiere lipsă, permisiuni insuficiente, fișiere corupte etc.).
5. Fii atent la optimizarea I/O-ului de fișiere, deoarece poate fi un factor limitativ important pentru fișiere mari.

Rezultate așteptate:

Fiecare sarcină trebuie să fie realizată cu succes, asigurându-se că datele sunt gestionate corect și eficient, și că rezultatele sunt prezentate într-un format corect și acces